Session: 1

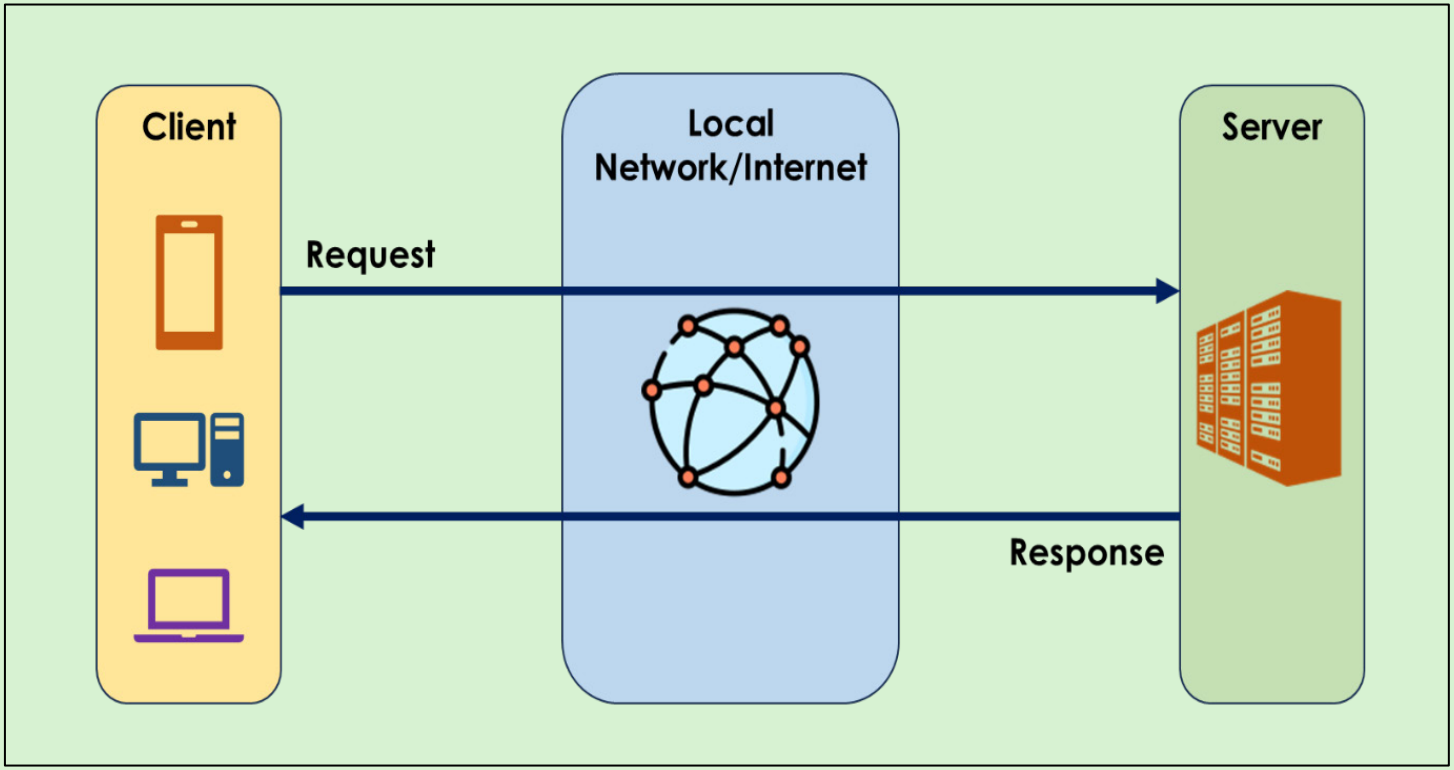
Introduction to Server-Side Development

Server-side Development with Node.js

Objectives

- Outline the client-server architecture
- Explain the significance of server-side programming
- Explain a Web server and its architecture
- Describe Node.js, its features, and its installation process

What is Client-Server Architecture?



Server-side Programming

Server-side programming is developing various programs that run on server to perform the tasks, which are received as instructions from the users through the client.

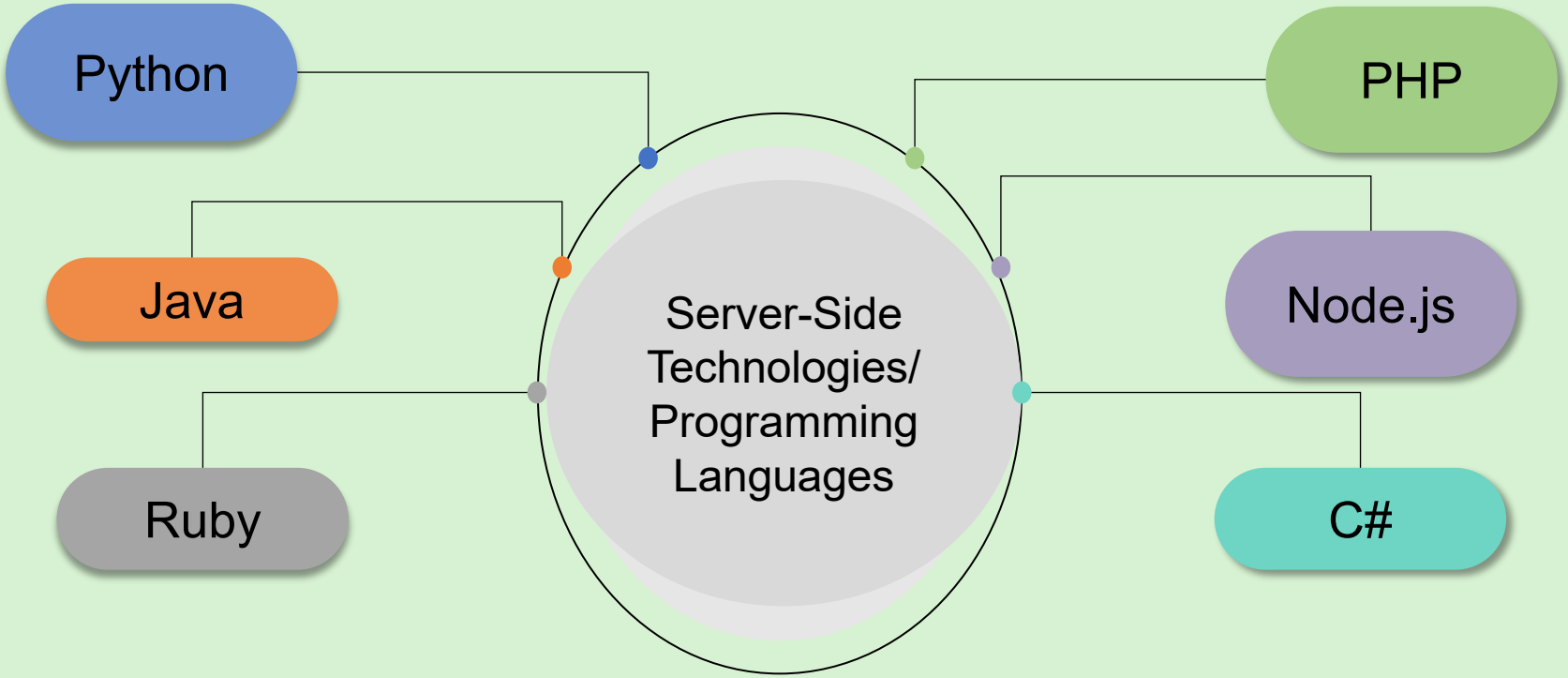
Dynamic Websites

- Storing and retrieving data from a database
- Generating dynamic responses based on user inputs
- Authenticating and authorizing users
- Sending and receiving emails

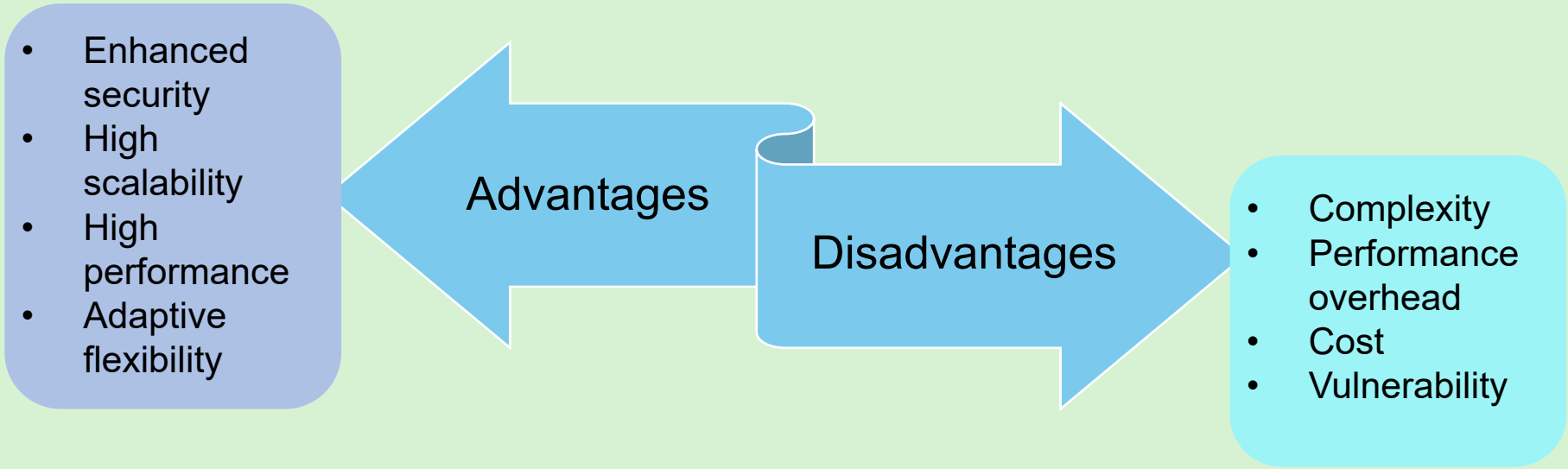
Static Websites

- Displaying the:
  - Current date and time on a Web page
  - Latest posts or news on a Website
- Recommending products based on the user's browsing and purchasing history
- Processing payments on e-commerce Websites

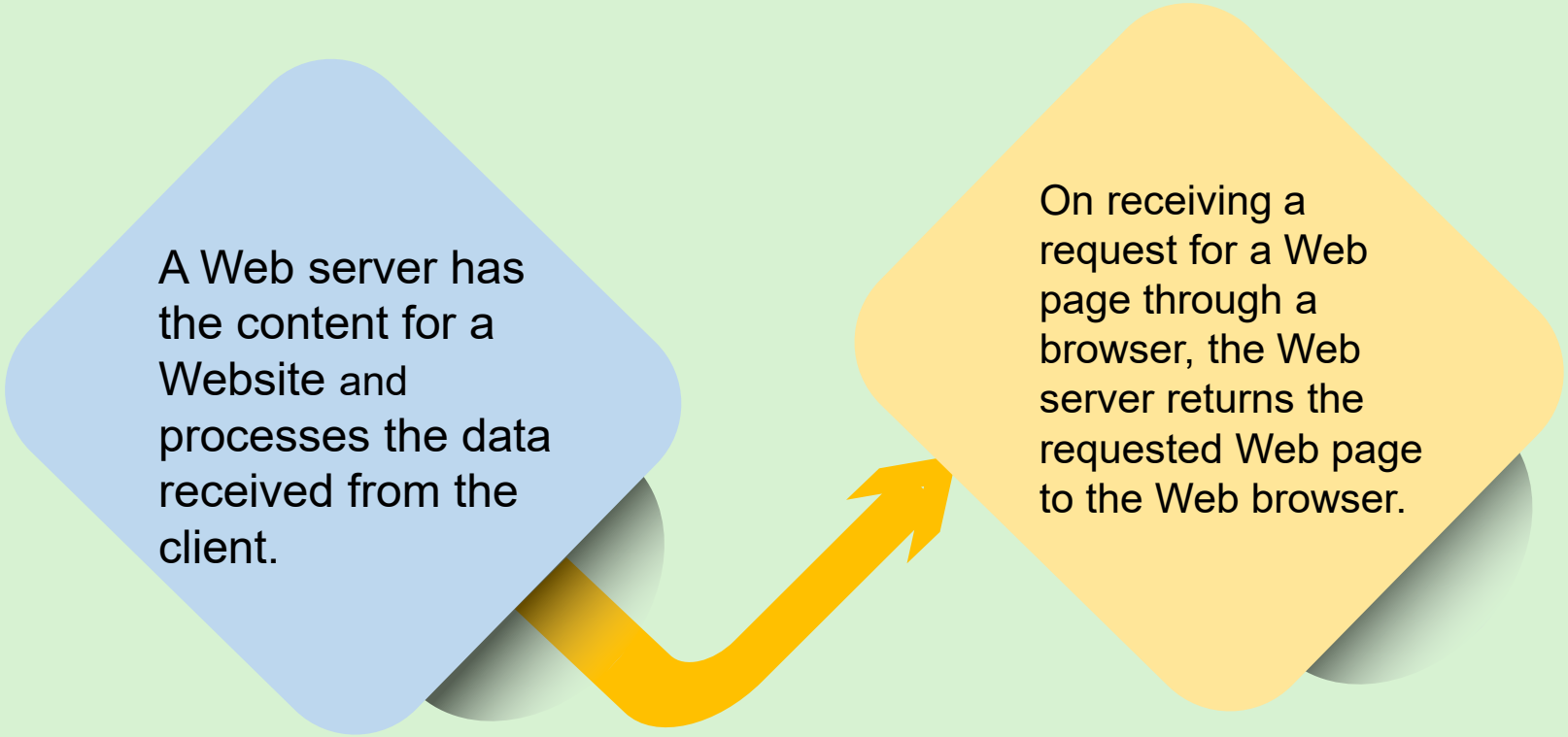
Server-side Technologies/Programming Languages



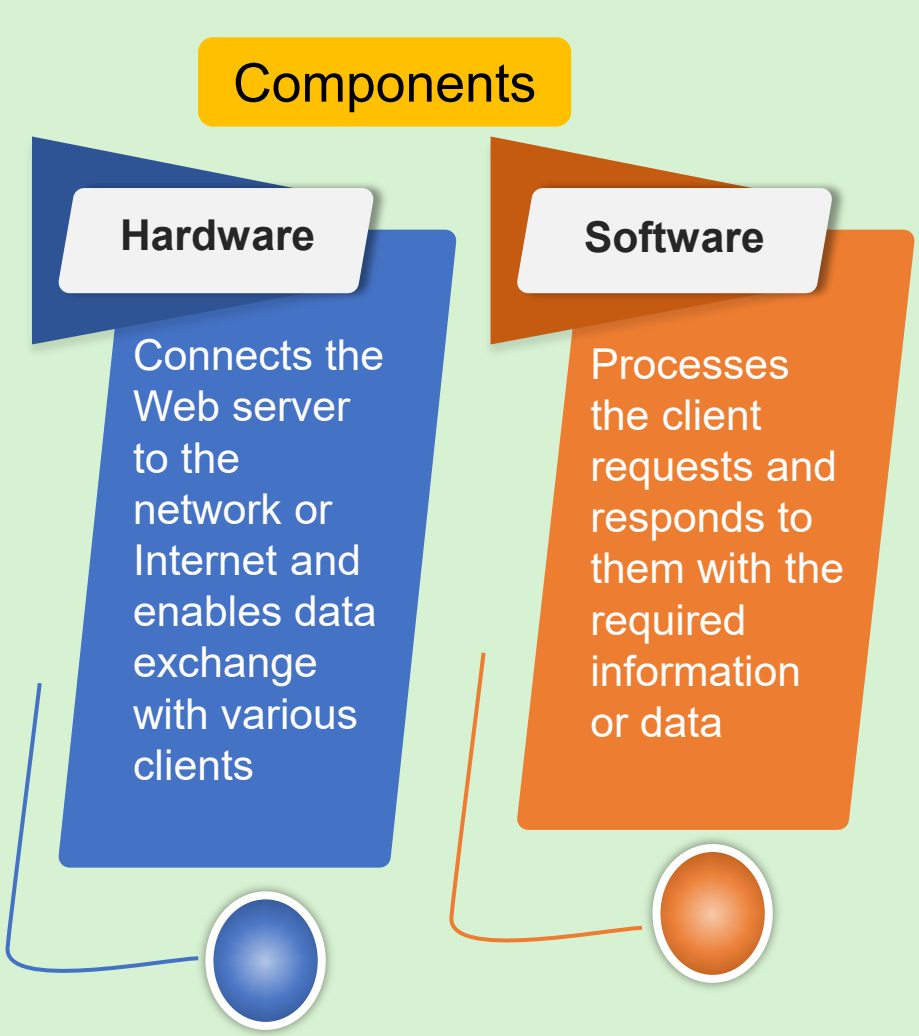
Advantages and Disadvantages of Server-side Programming

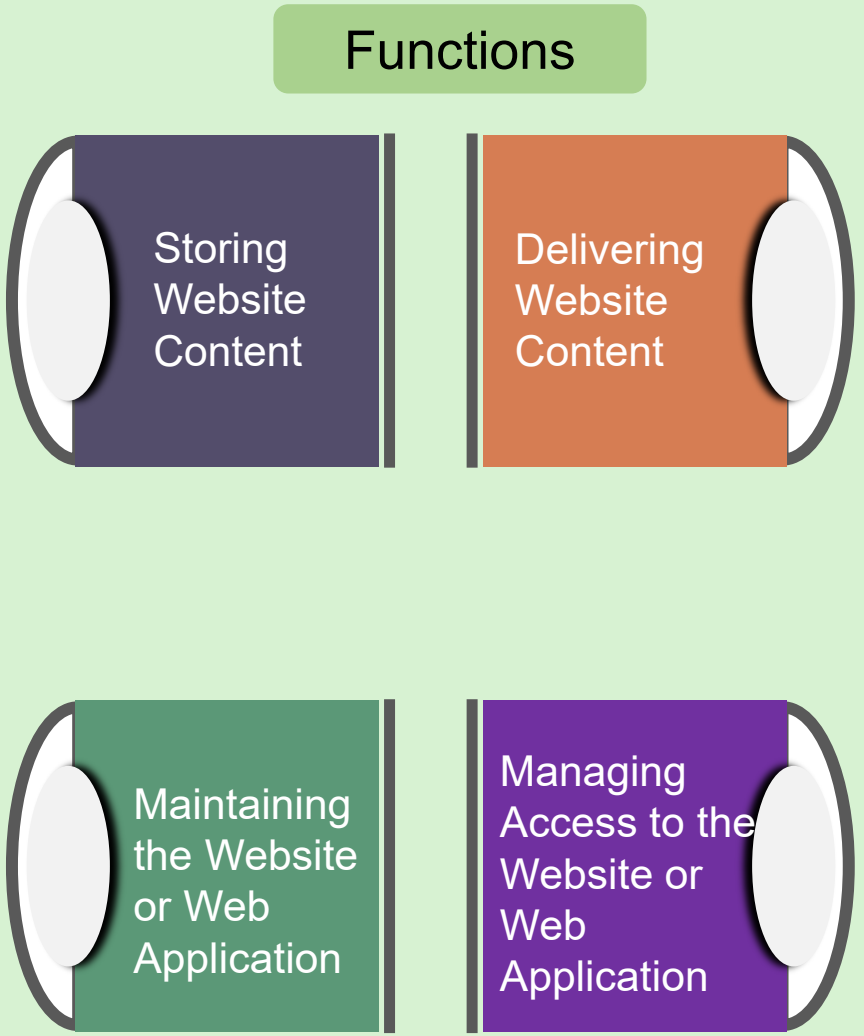


Introduction to Web Server



Components and Functions of a Web Server





Types of Web Server Architecture

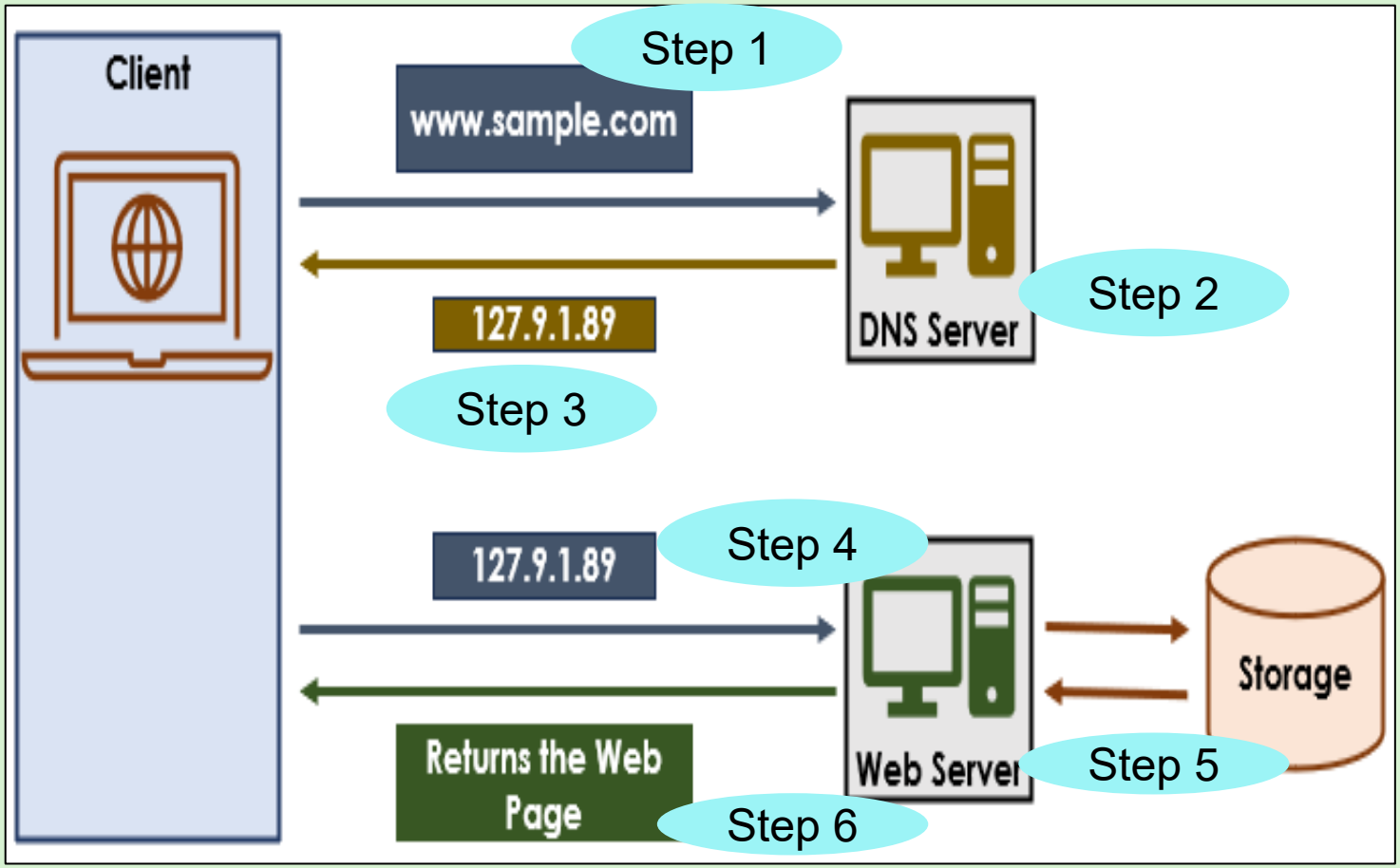
Concurrent Approach

- The Web server creates a thread for each request it receives.
- Every thread facilitates simultaneous responses to multiple requests.
- This approach is best suited for Websites that experience heavy user traffic.

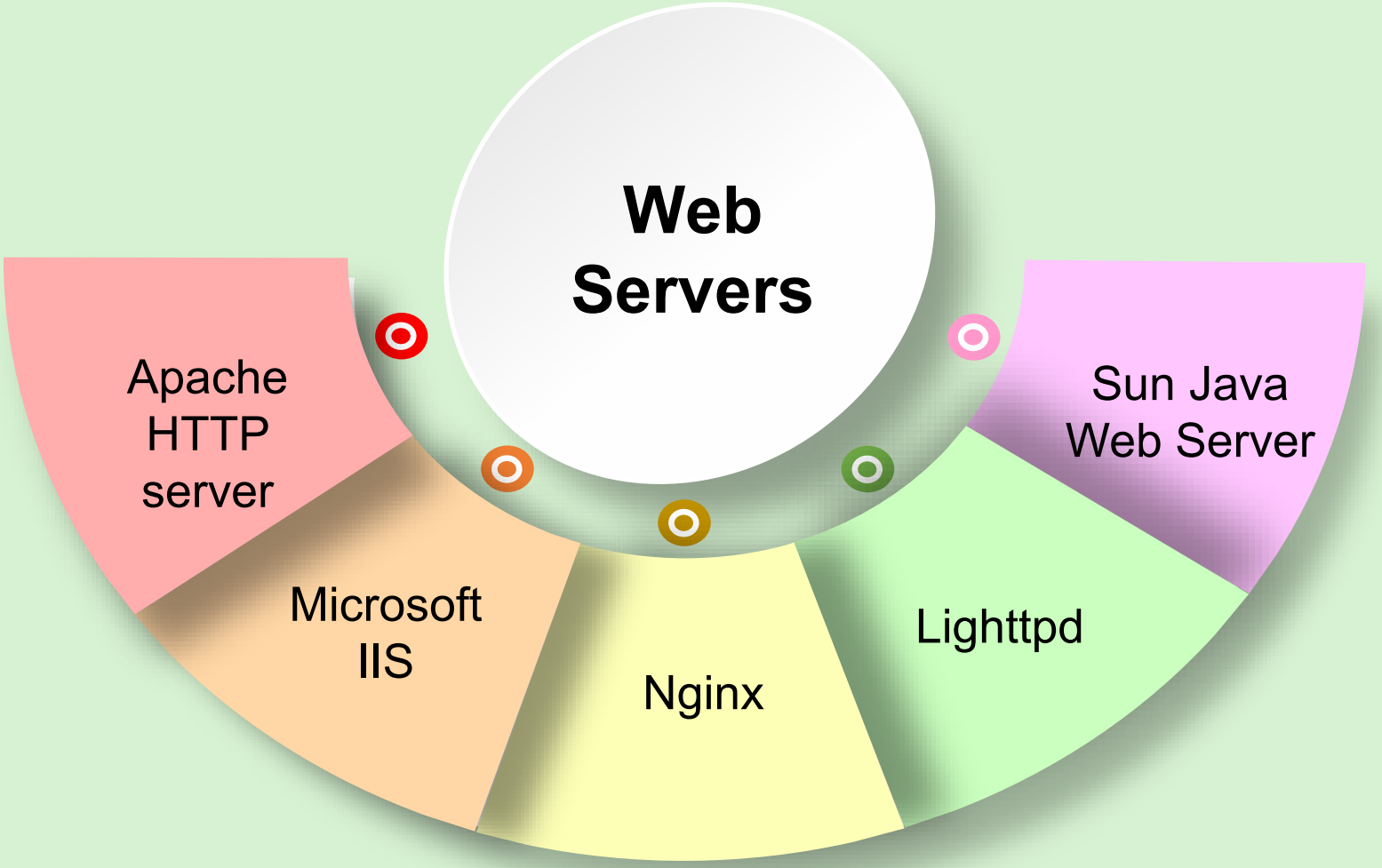
Single-Process-Event-Driven Approach

- The Web server creates a single thread for all the requests it receives.
- The thread handles the requests in a non-blocking manner.
- This approach is best suited for Websites that experience low user traffic.

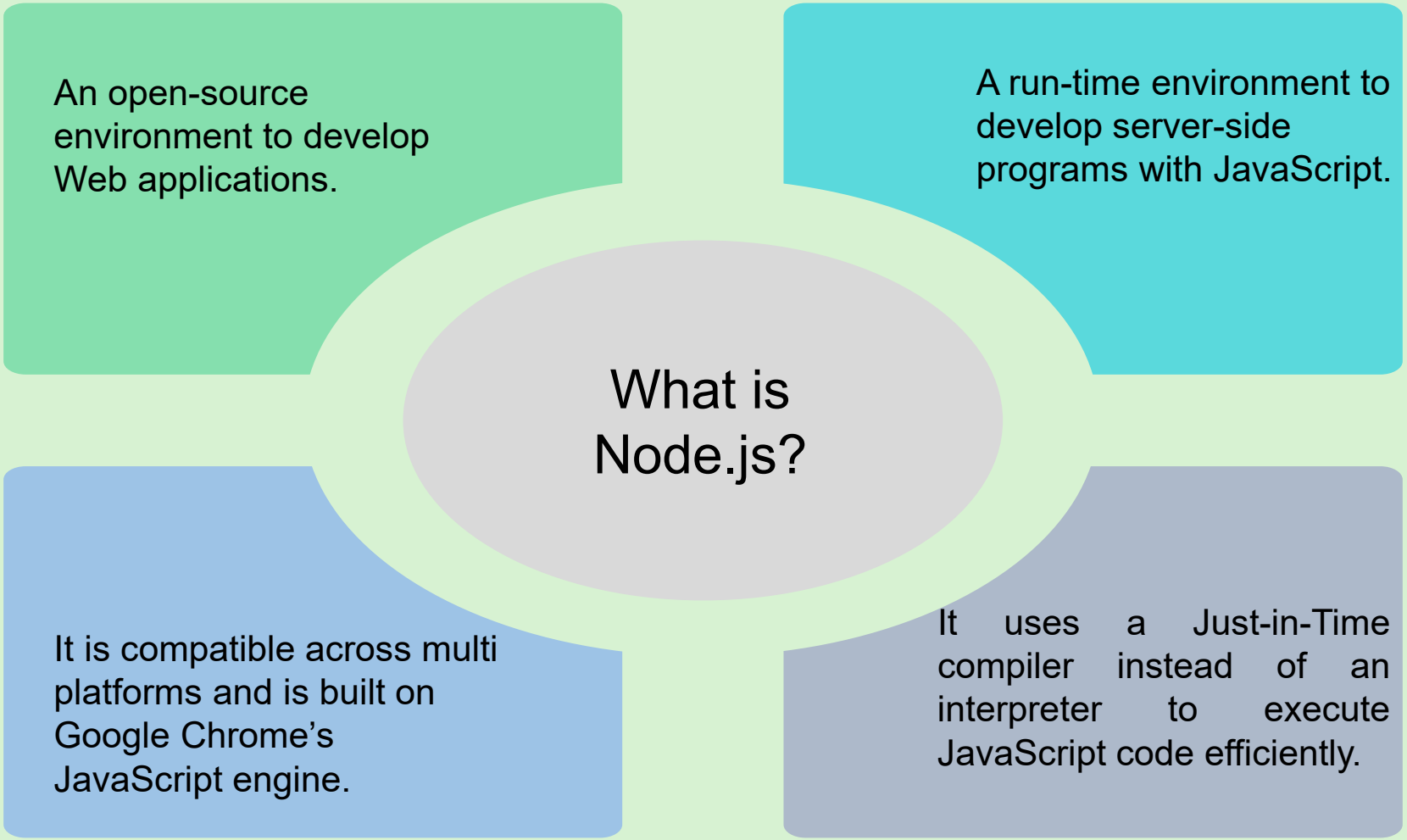
Working of Web Servers



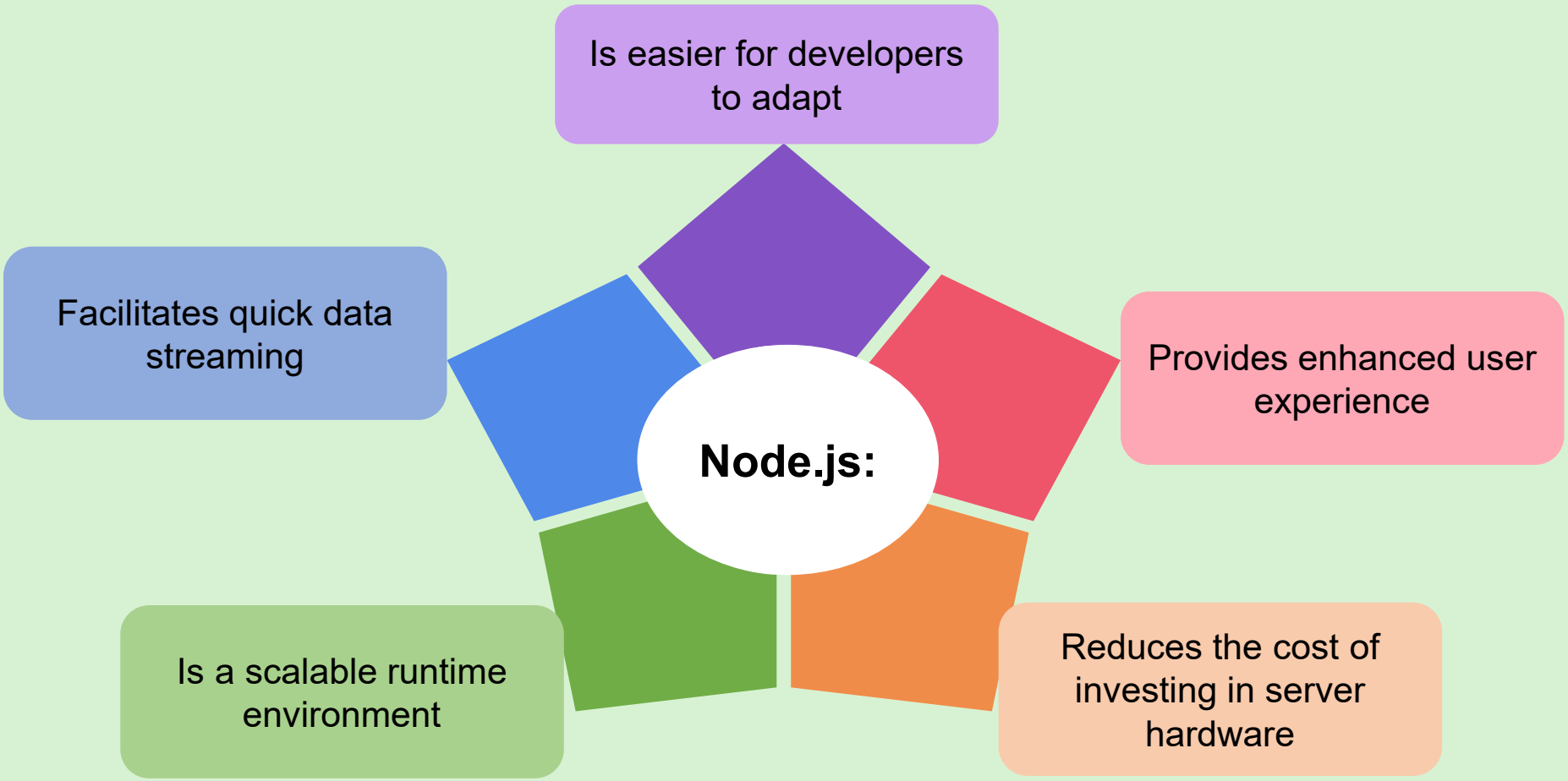
Types of Web Servers



Introduction to Node.js



Features of Node.js



Node.js Installation

1. Open a browser and navigate to: https://nodejs.org/en/download
2. Download the associated .msi file by clicking it.
3. Navigate to the folder where the file is downloaded and double-click it.
4. Complete the steps on the Node.js Setup wizard.
5. To install Node.js, click **Install**.
6. To close the wizard, click **Finish**.
7. To verify that Node.js has been installed, open a Command Prompt window.
8. At the command prompt, type the command as `node -v`.

Summary

- Client-server architecture allows the server to serve requests from multiple clients.
- Client, server, and network are the components of the client-server architecture.
- The DNS server and Web server are the two servers that help in fetching the Web page when a request is made.
- Server-side scripting allows data processing to be done on the server side, thereby keeping the processing part hidden from the client.
- Java, PHP, Python, Node.js, Ruby, and C# are some of the server-side scripting tools.
- Node.js is a single-threaded, run-time environment for running JavaScript.
- Ease of working with JavaScript, scalability, and fast streaming of data makes Node.js a preferred runtime environment for Web applications.
- Visual Studio Code can be used as an IDE for Node.js applications.

Session: 2

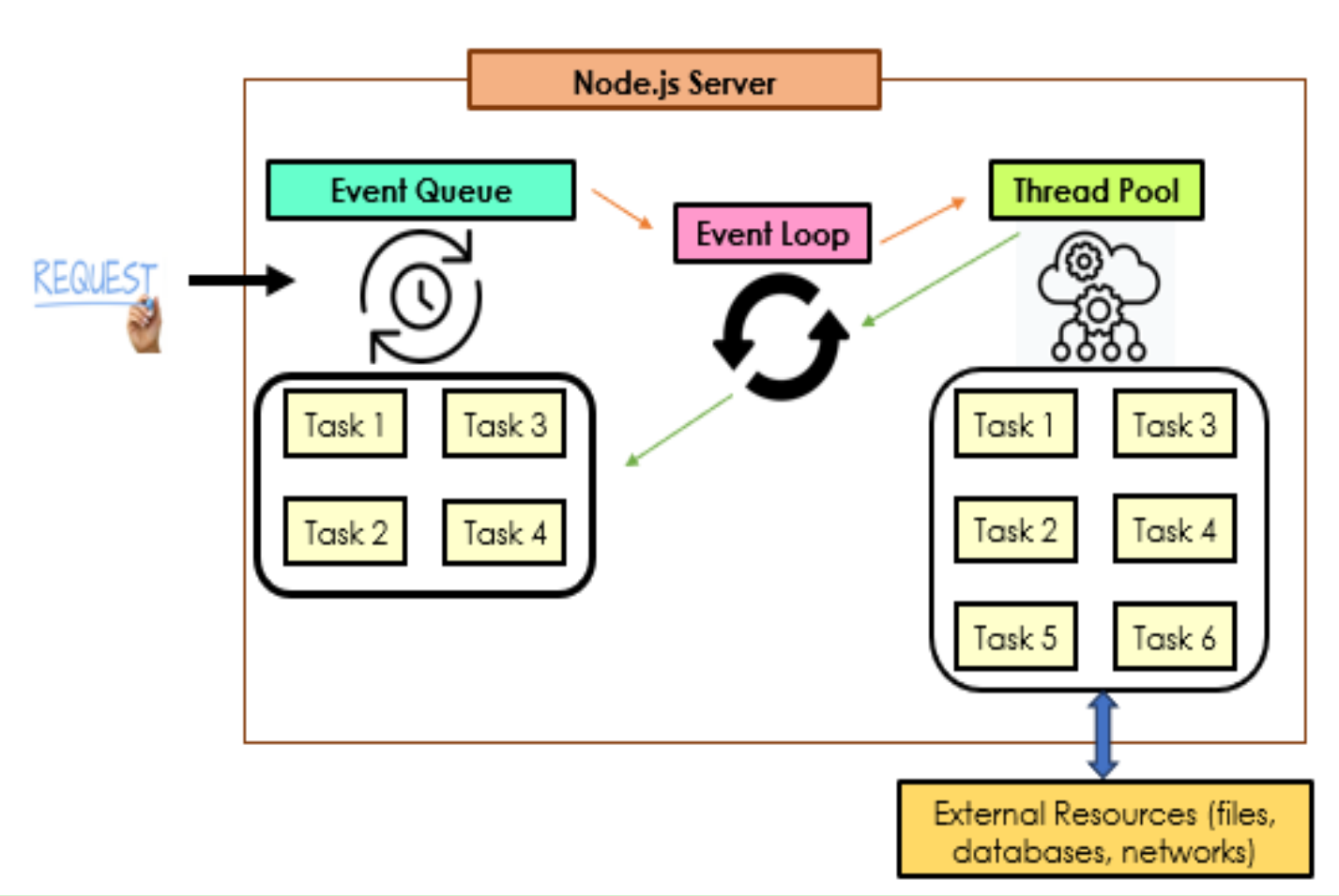
Node.js Essentials

Server-side Development with Node.js

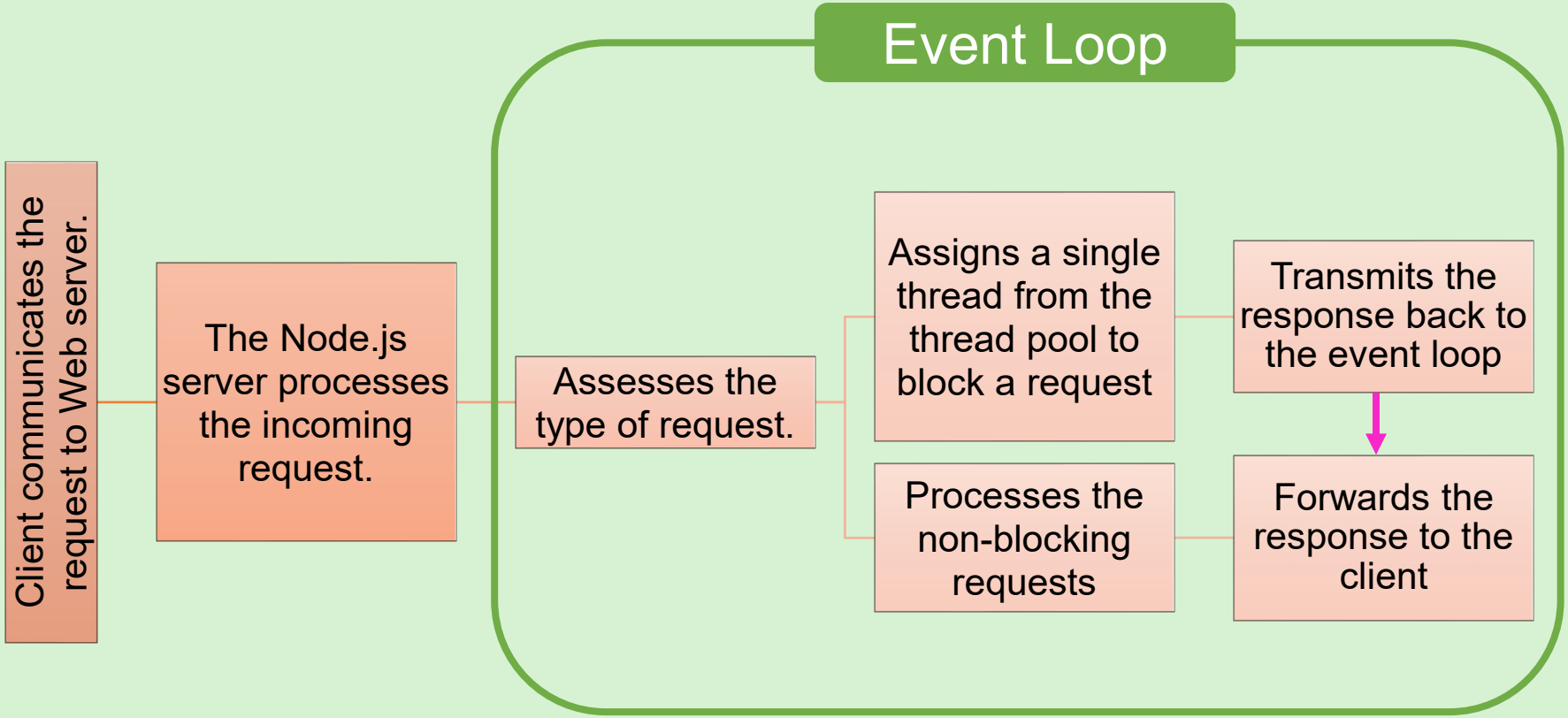
Objectives

- Describe the architecture of Node.js
- List benefits and limitations of Node.js applications
- Explain the concept of console-based applications in Node.js
- Explain major programming constructs in Node.js
- Compare the roles of Node Package Manager (NPM) and Node Version Manager (NVM) in Node.js
- Describe global objects and their use in Node.js applications

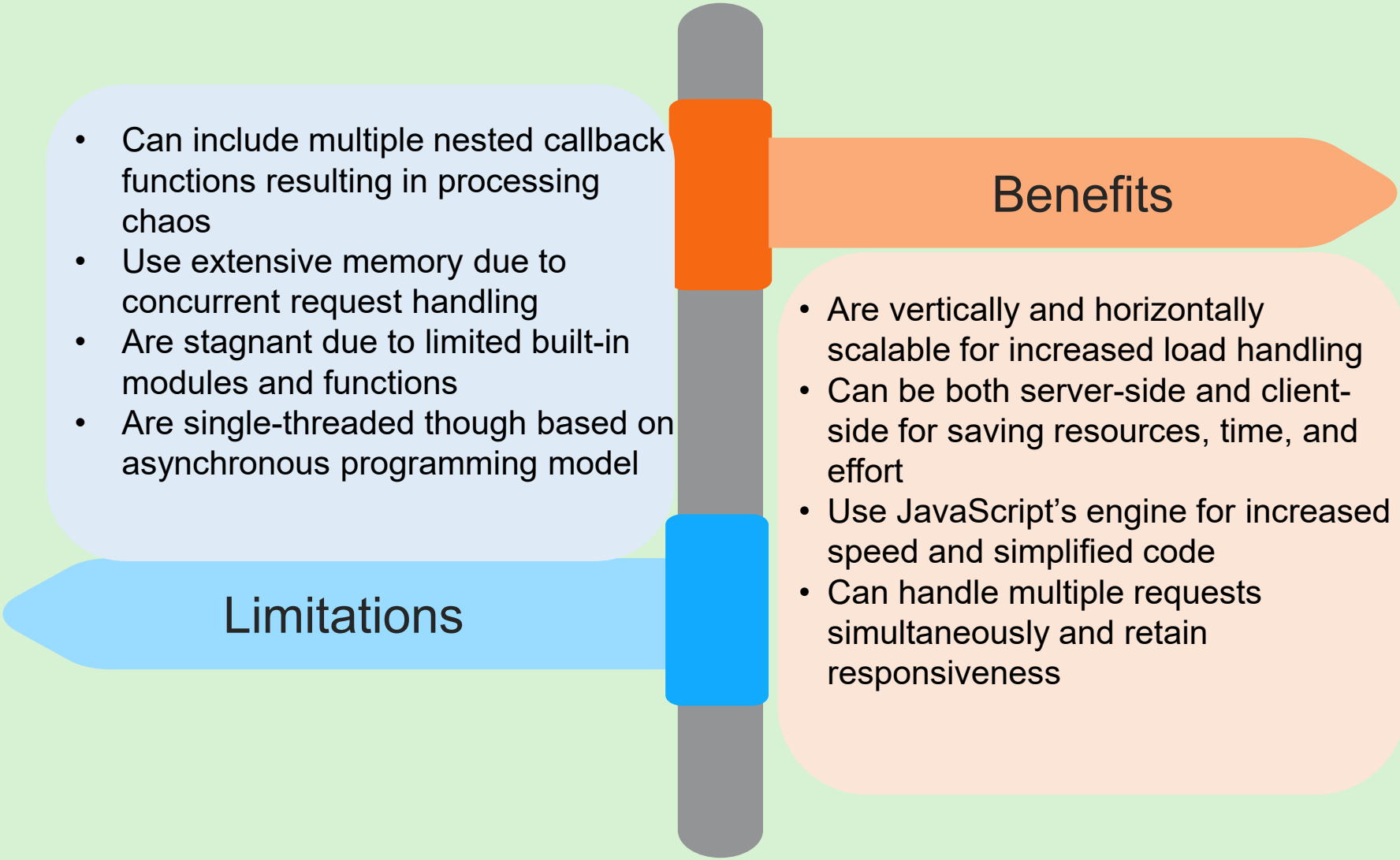
Introduction to Node.js Architecture



Workflow of Node.js

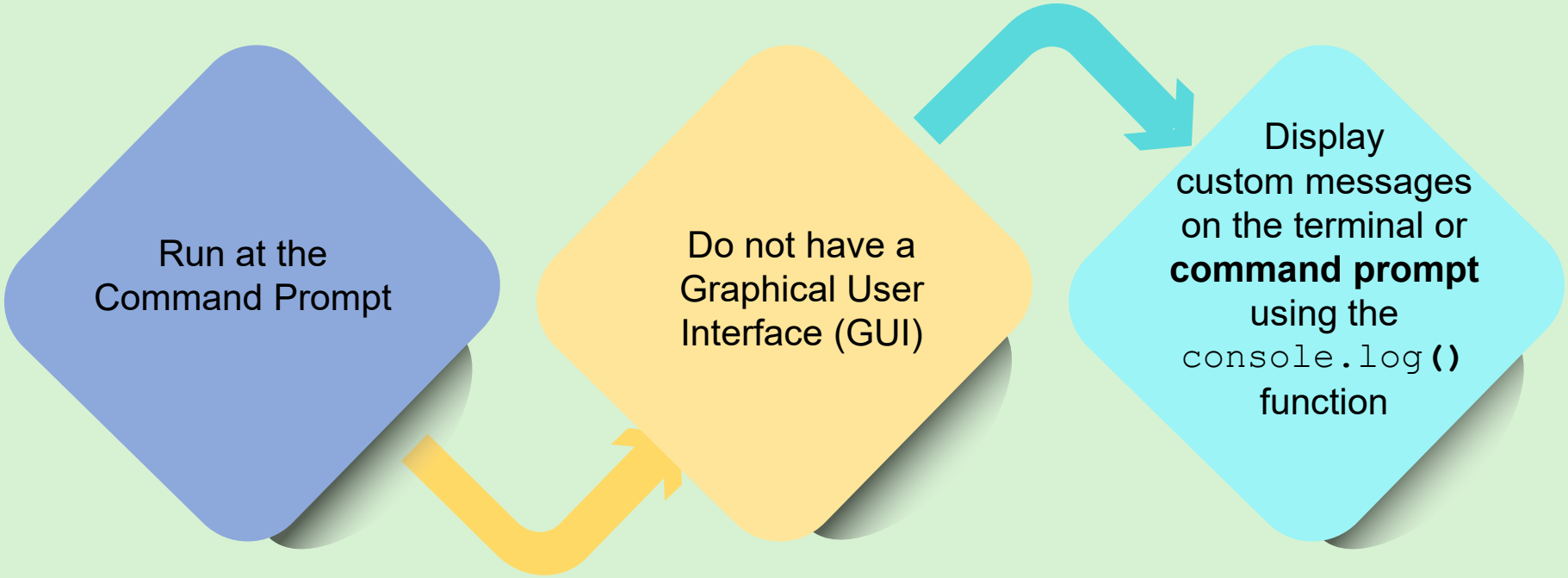


Benefits and Limitations of Node.js Applications

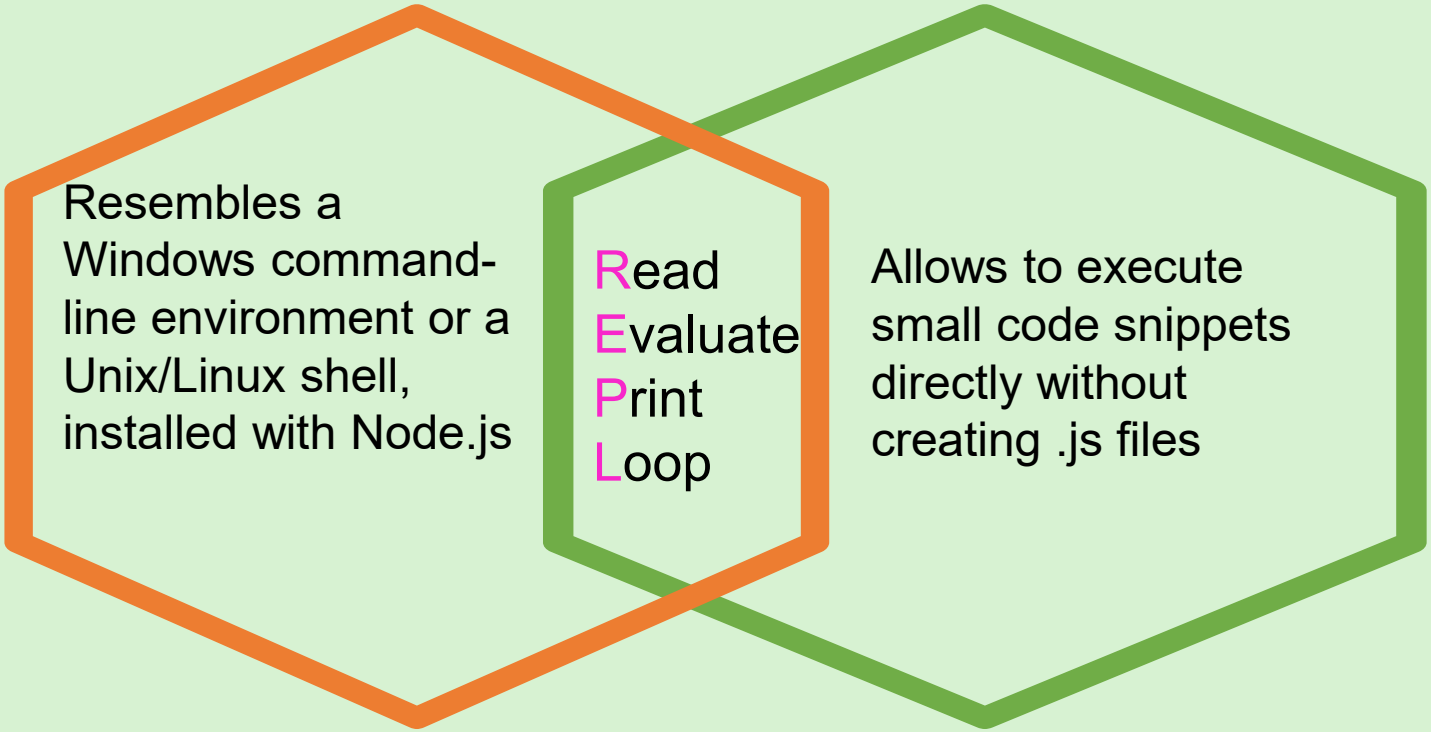


Console-based Application in Node.js

Console-based Applications:

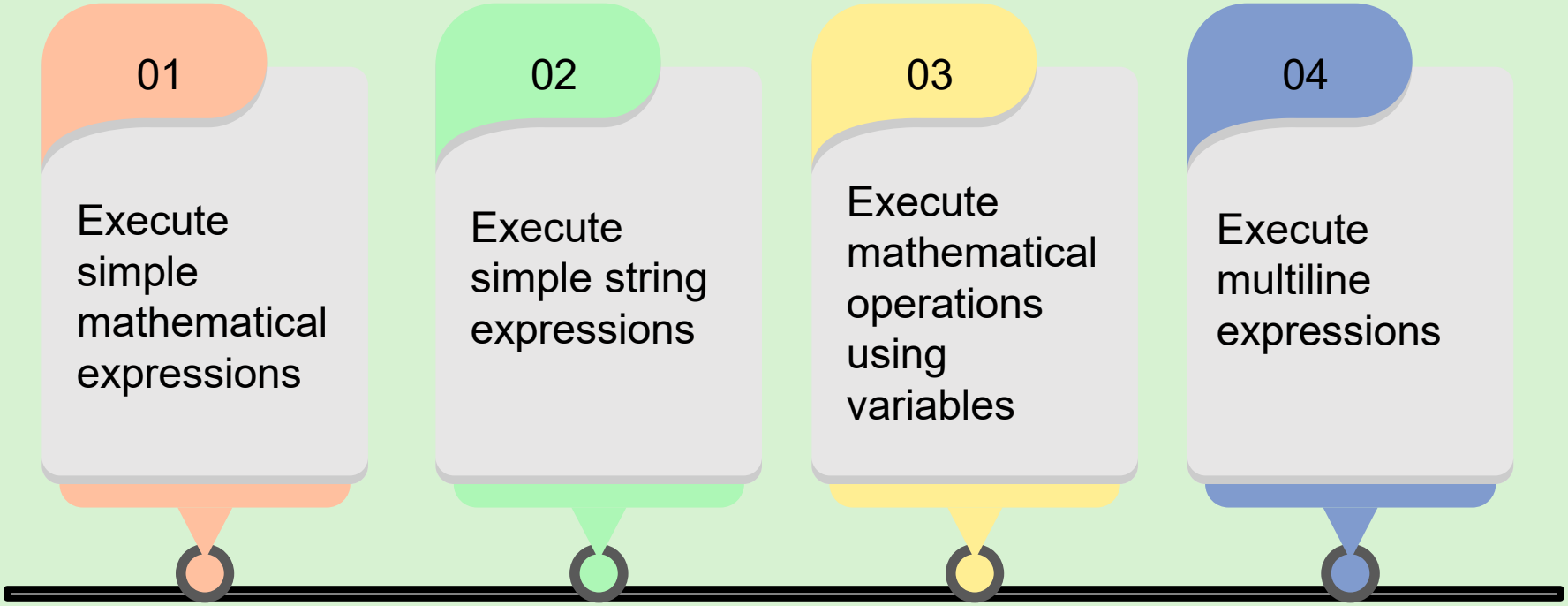


Overview of REPL

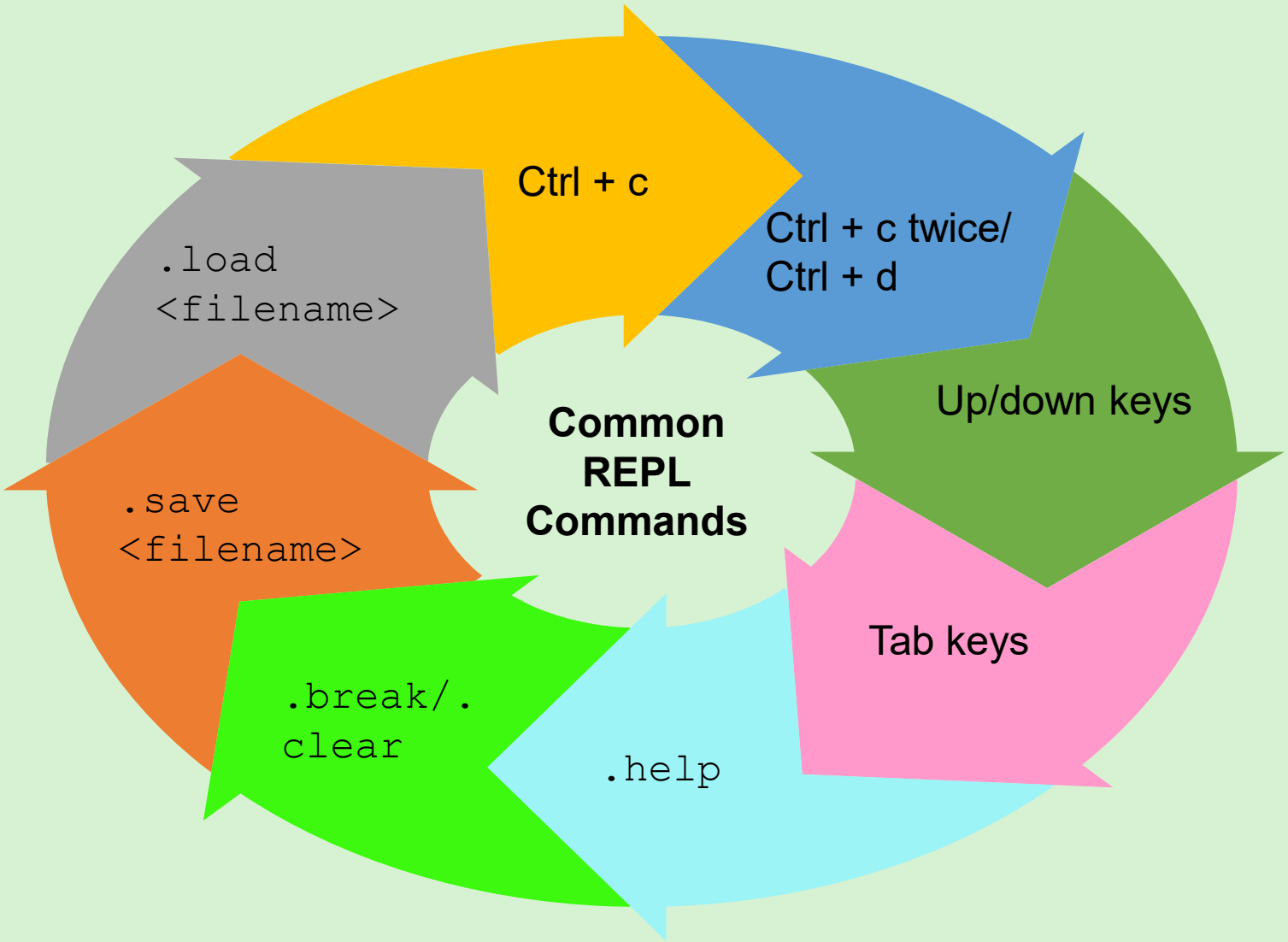


The four parts in REPL perform specific tasks.

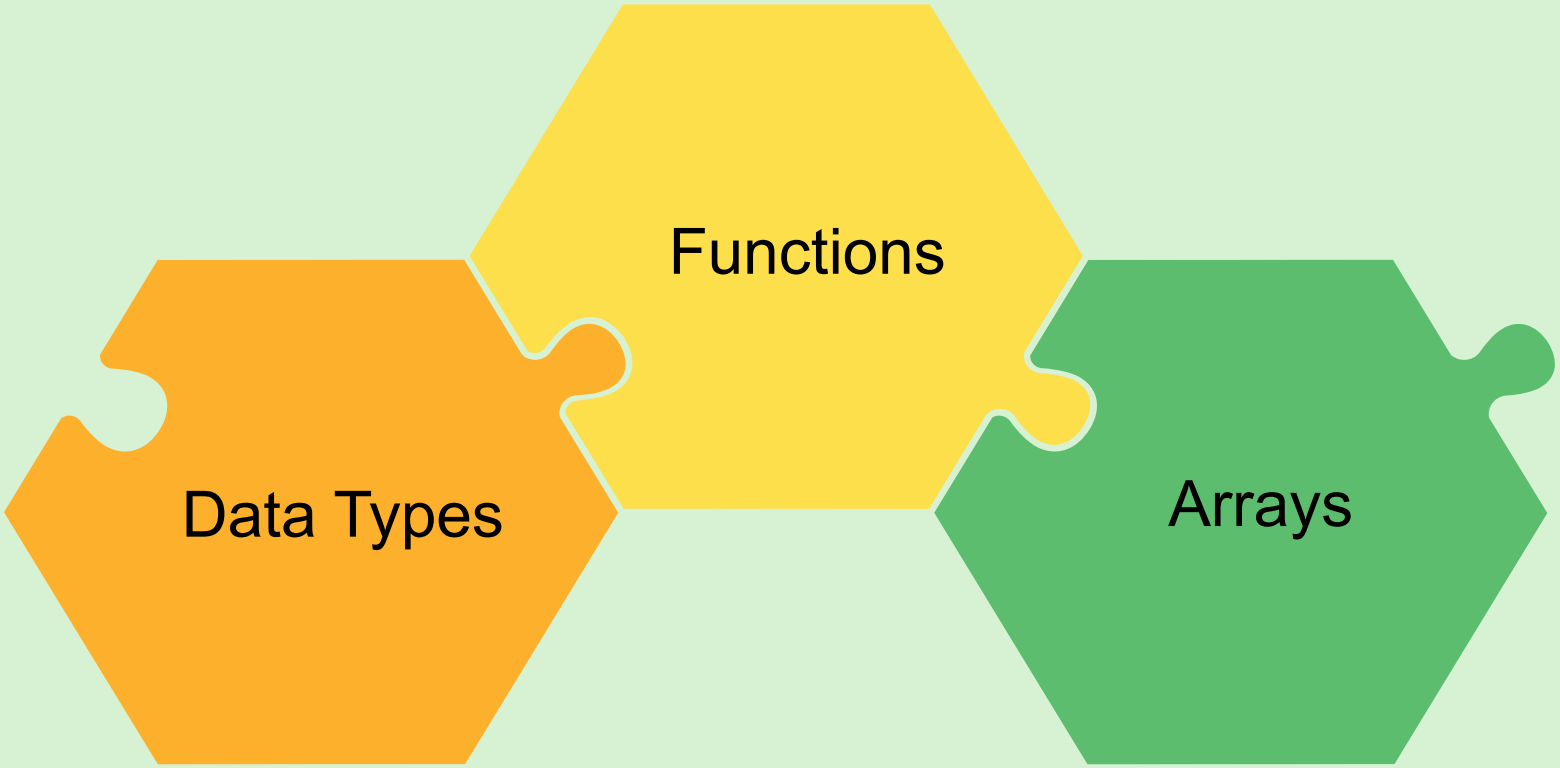
Using the REPL Command Prompt



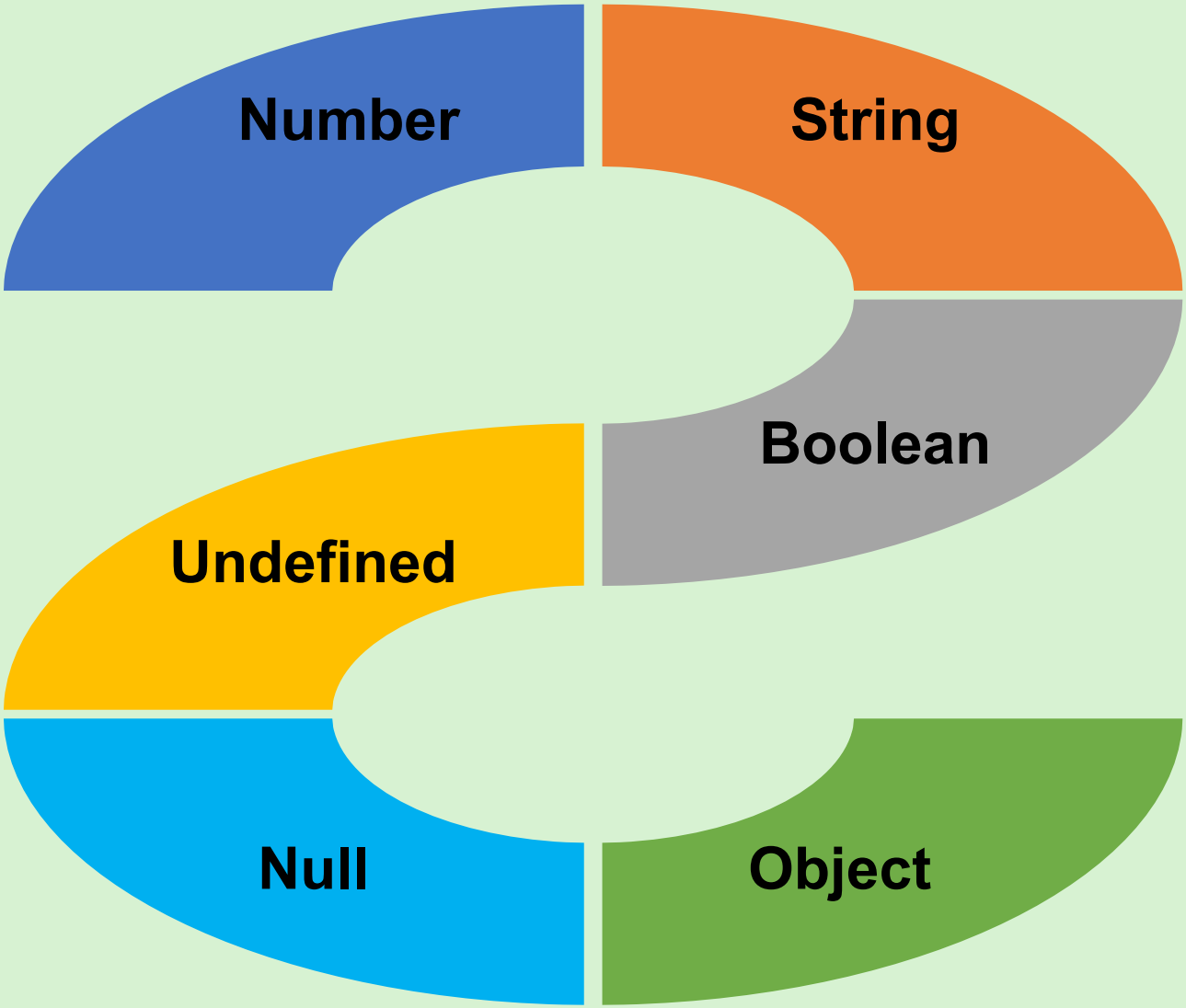
Commonly-used Commands in Node.js REPL



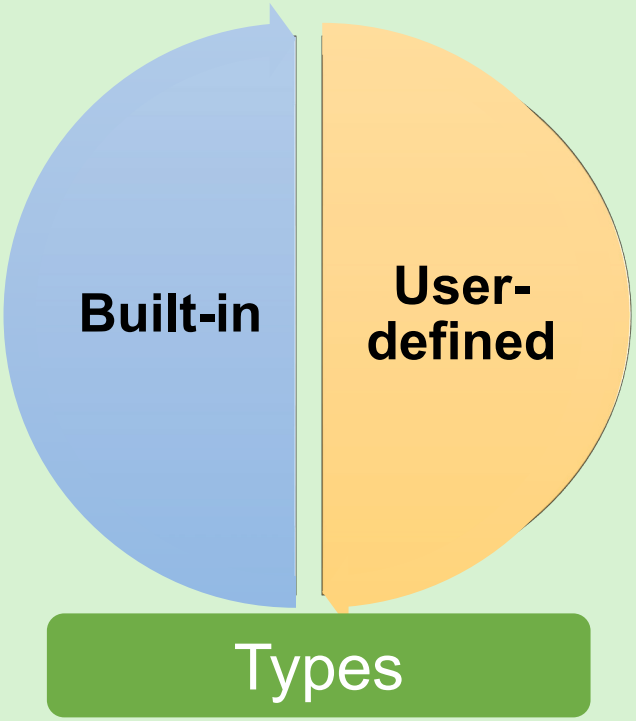
Node.js Programming Constructs



Data Types in Node.js



Functions in Node.js



Features

Functions:

- Use the `function` keyword to define a function
- Define the parameters in parenthesis
- Include set of instructions in the function body
- Can be invoked multiple times in a program
- Allow passing of parameter values (arguments) during execution

Arrays in Node.js

Features

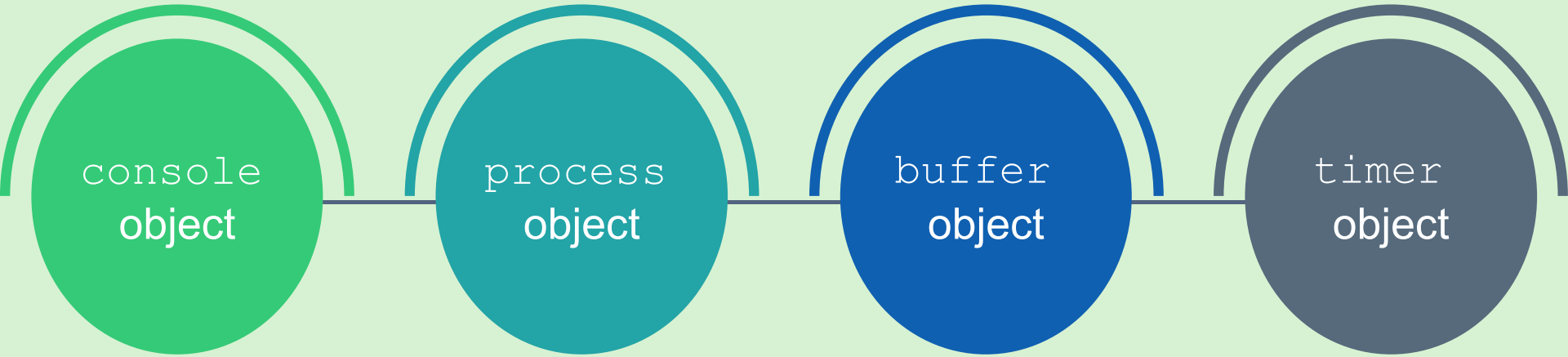
Arrays:

- Can store more than one value of any data type
- Contain elements or items (stored values)
- Use square brackets separated by commas

NPM and NVM

NPM	NVM
Is a default package management tool installed with Node.js	Downloads, installs, manages, and upgrades Node.js application version on the system
Installs JavaScript packages from npm registry	Maintains multiple versions of the Node.js application
Installs, updates, or uninstalls Node.js packages	Updates Node.js application version on the system seamlessly

Global Objects



Summary

- Node.js, a JavaScript-based runtime environment, has various components to handle the client request, complete the execution, and provide the response.
- Node.js manages multiple client requests simultaneously.
- Node.js is single-threaded and follows an asynchronous programming model.
- REPL environment reads the input, evaluates the code, prints the output, and iterates through the tasks.
- Console-based applications perform their tasks in command-line environments.
- NVM manages the versions of Node.js application.
- NPM installs, updates, and uninstalls Node.js packages and modules.
- Global objects in Node.js are available throughout the lifetime of an application.

Session: 3

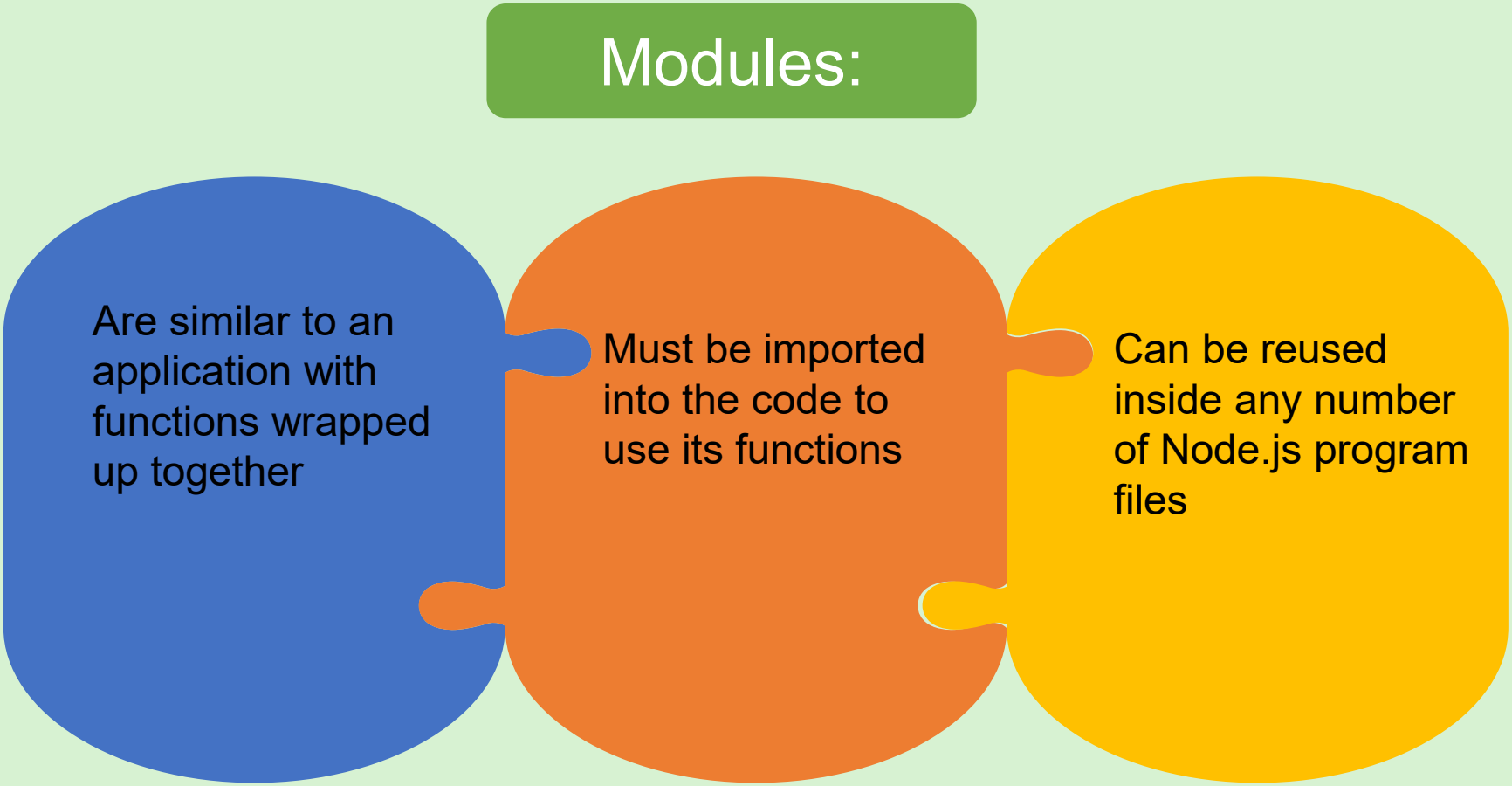
Modules and Packages in Node.js

Server-side Development with Node.js

Objectives

- Explain what modules are and its three types
- Explain the significance of packages
- Describe the purpose of Web framework and utility functions

Modules in Node.js



Types of Modules in Node.js

Core (Built-in) Modules

Get installed along with the Node.js installation

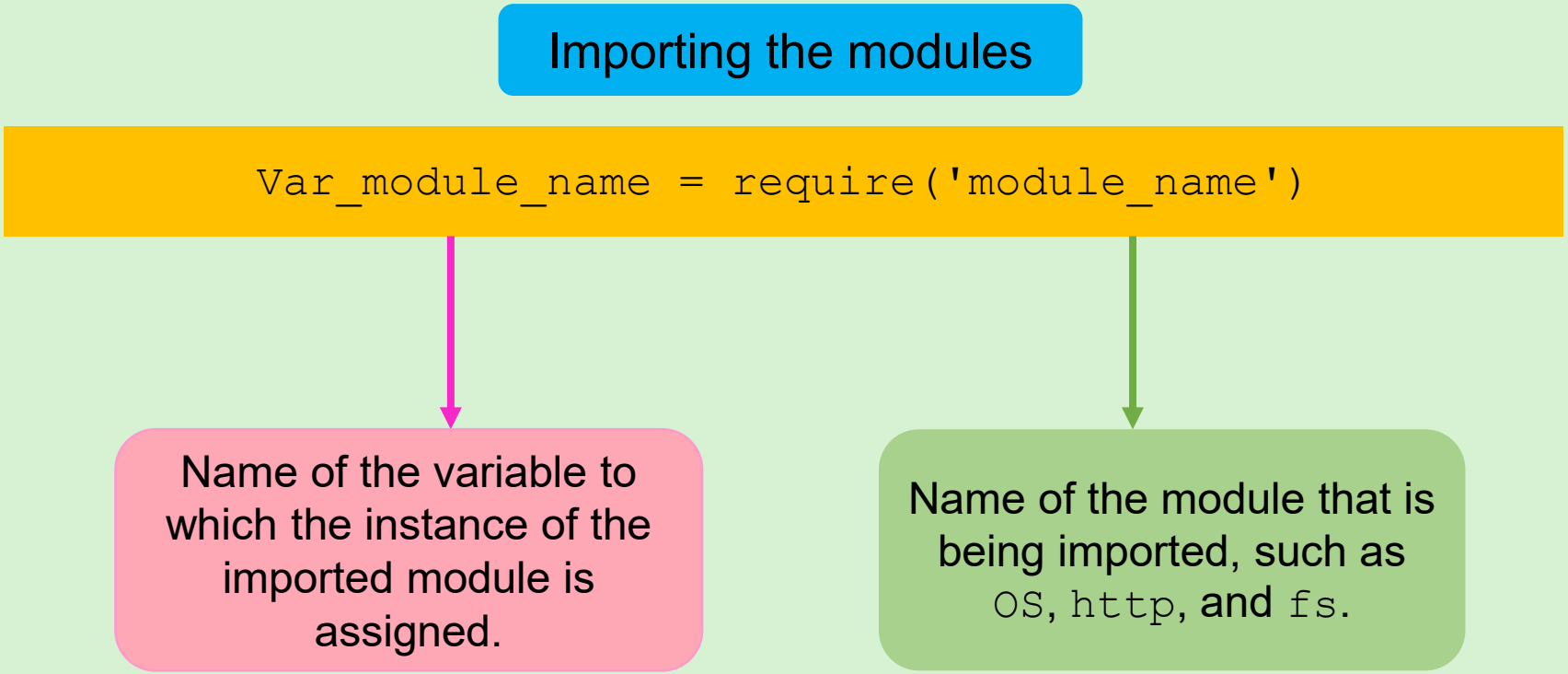
Local Modules

Are created locally within a Node.js application and can be distributed across projects using the Node Package Manager (NPM)

Third Party Modules

Are created by some third party for use by other Web applications, available online, and installed using the NPM

Working with Modules



Using the Methods in the Module

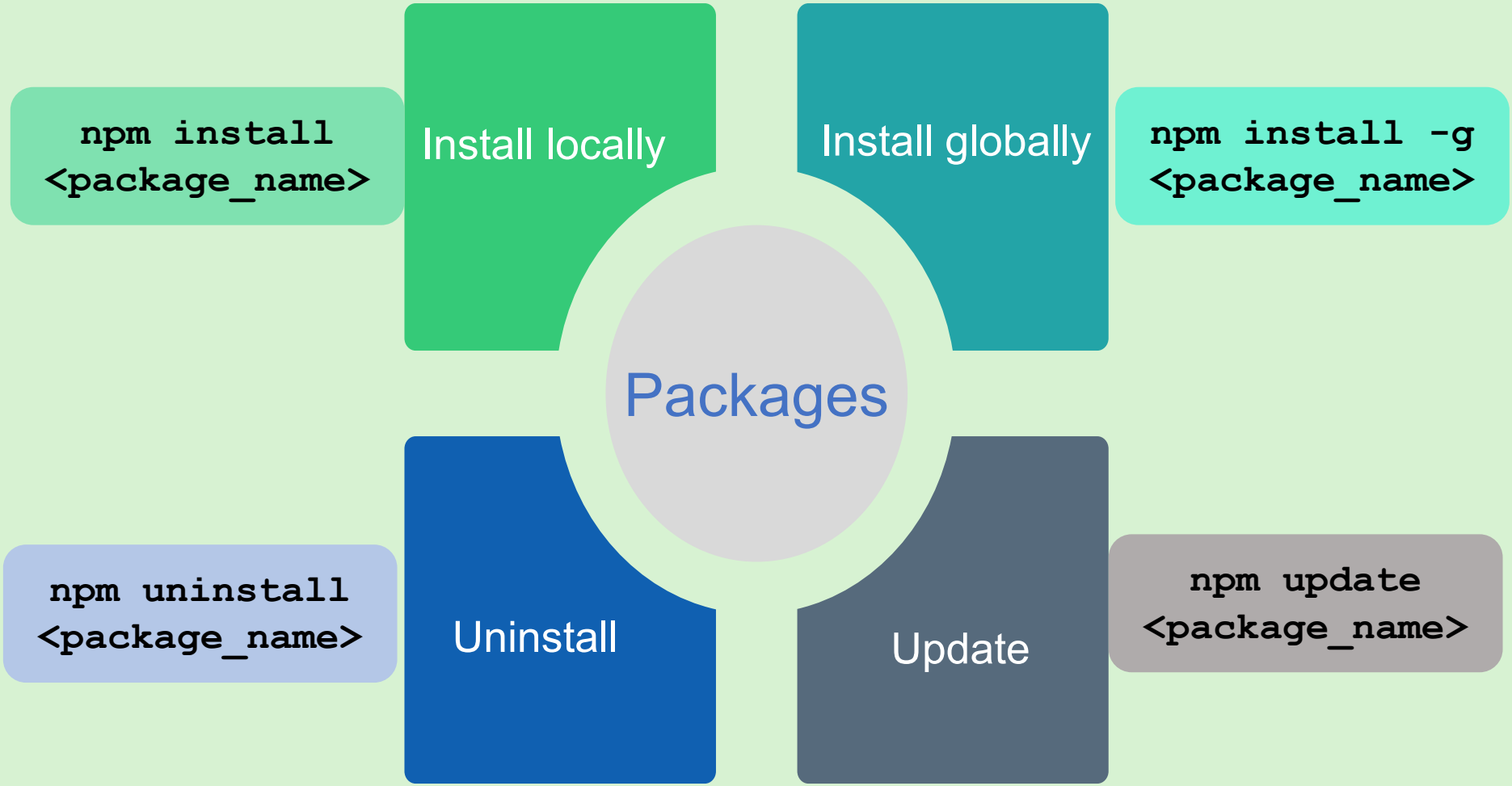
Operation	Method	Syntax
To open a file	`fs.open`	`fs.open(path,flags[,mode],callback)`
To write to a file	`fs.writeFile`	`fs.writeFile(filename, data, [encoding], [callback_function])`
To append to a file	`fs.appendFile`	`fs.appendFile(filename, data, [options], callback)`
To read a file	`fs.readFile`	`fs.readFile(filename, callback)`
To delete a file	`fs.unlink/fs.rmdir`	`fs.unlink(filename, callback)`

Packages in Node.js

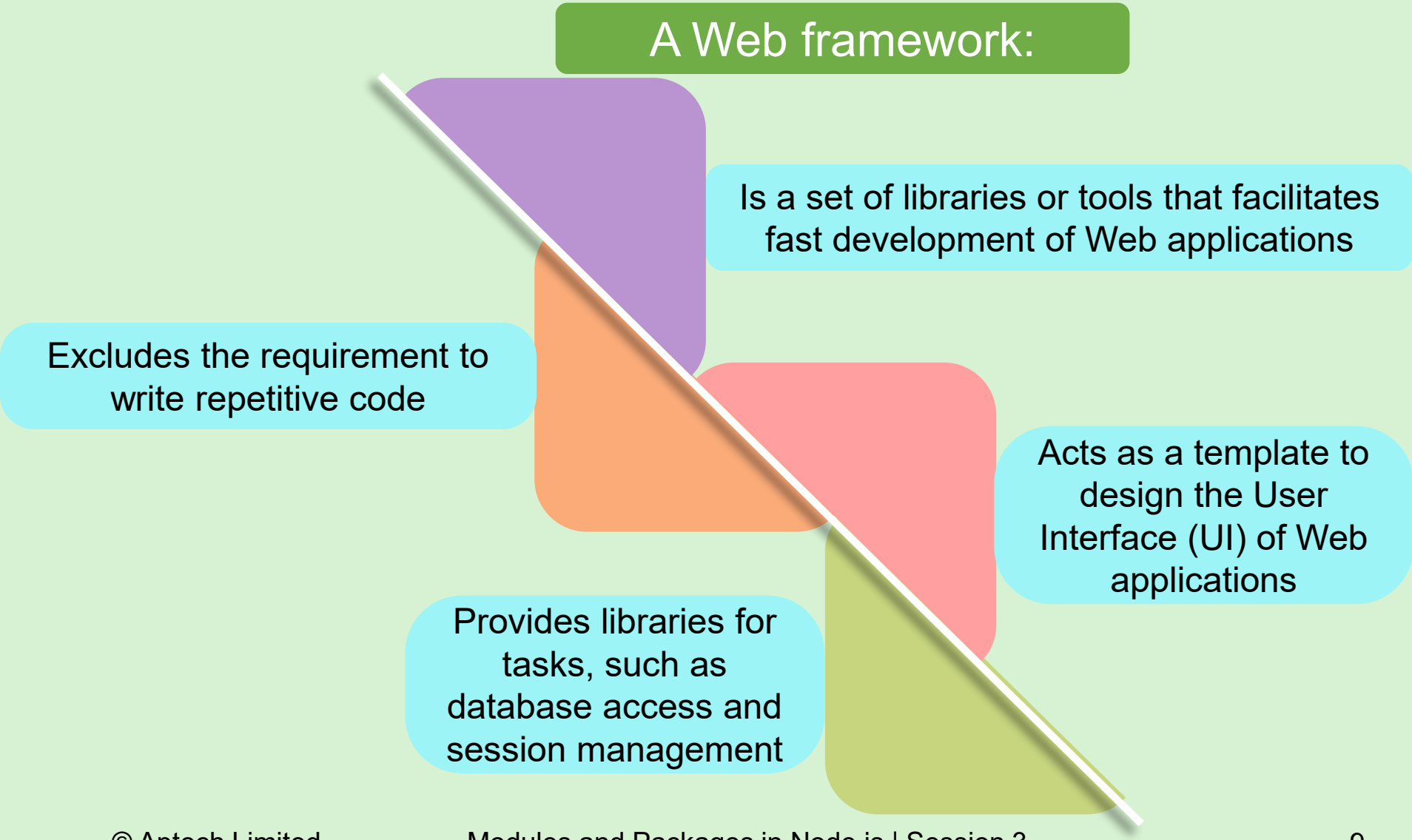
A package:

- Is a container of modules
- Collects all the associated modules together as a single unit
- Uses the `require` function to import into the application

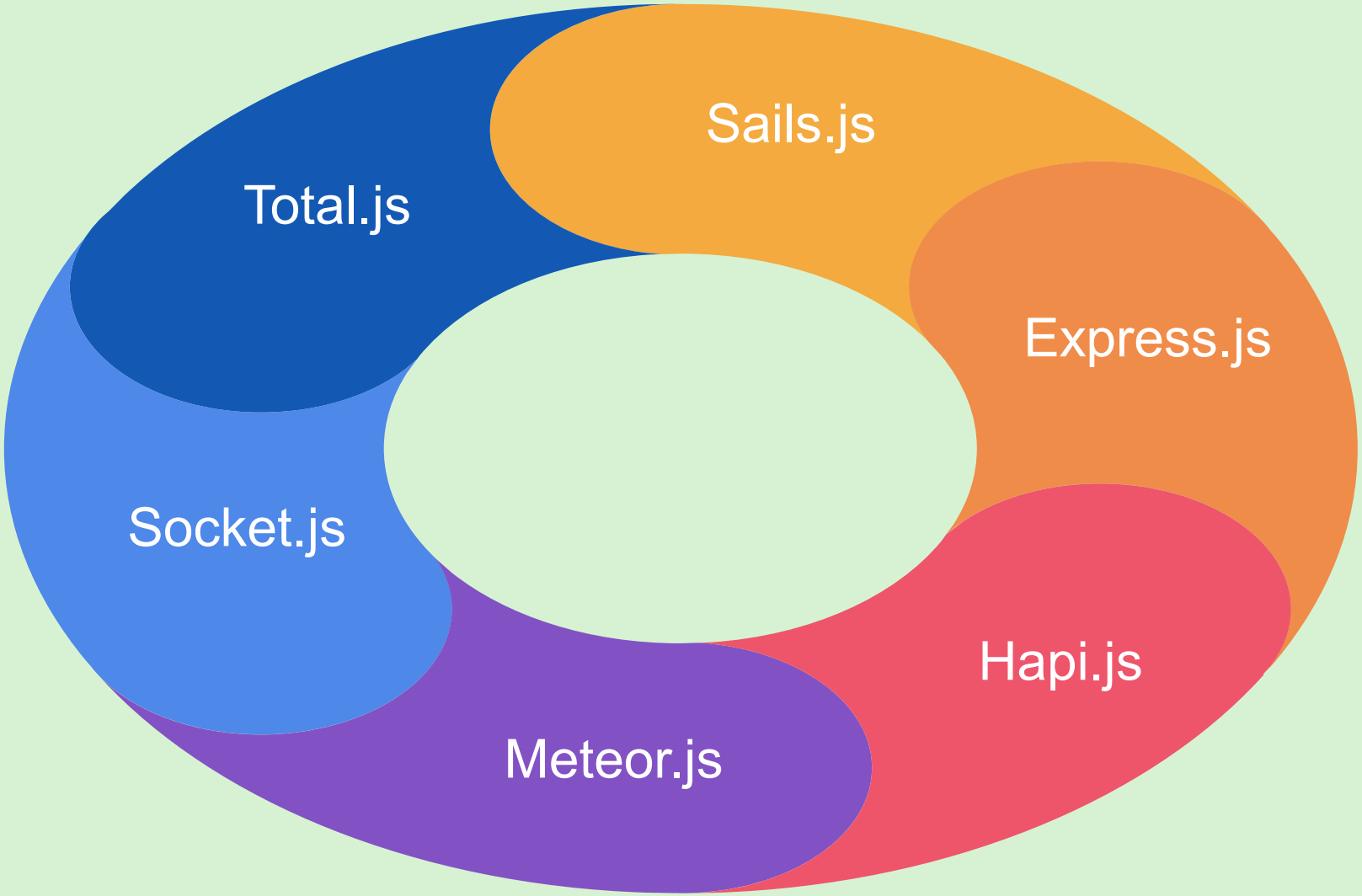
Using Packages



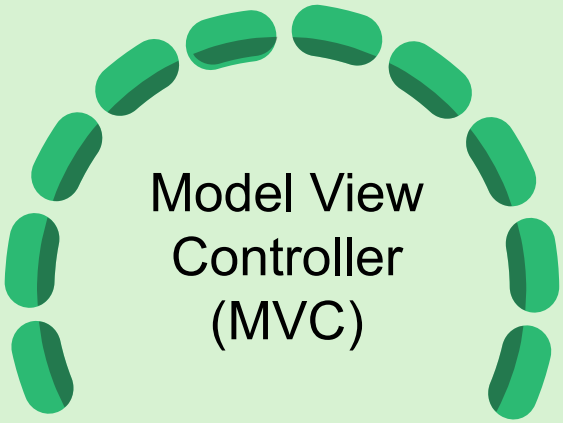
Web Frameworks



Common Node.js Frameworks

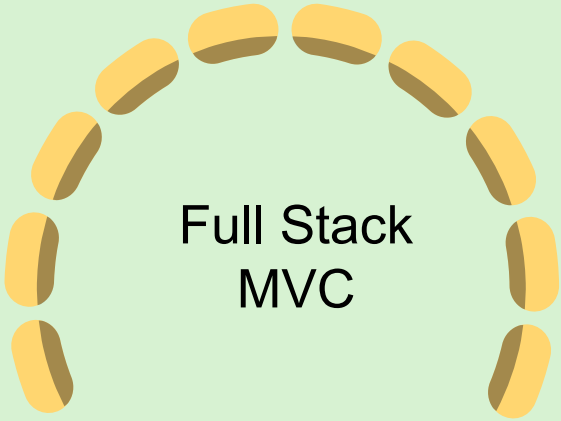


Types of Framework Models in Node.js

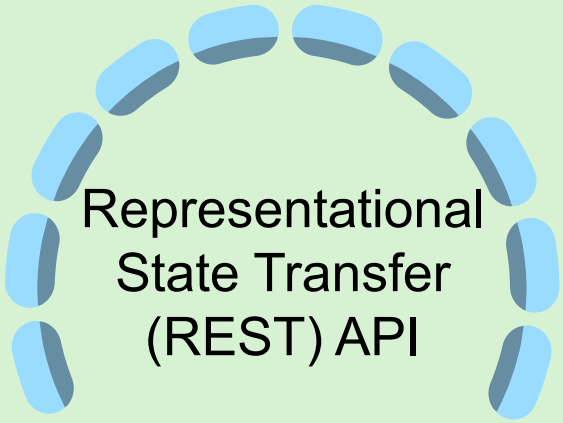


Splits the code logic into Model, View, and Controller to respond to the user request.

Example: Express,js

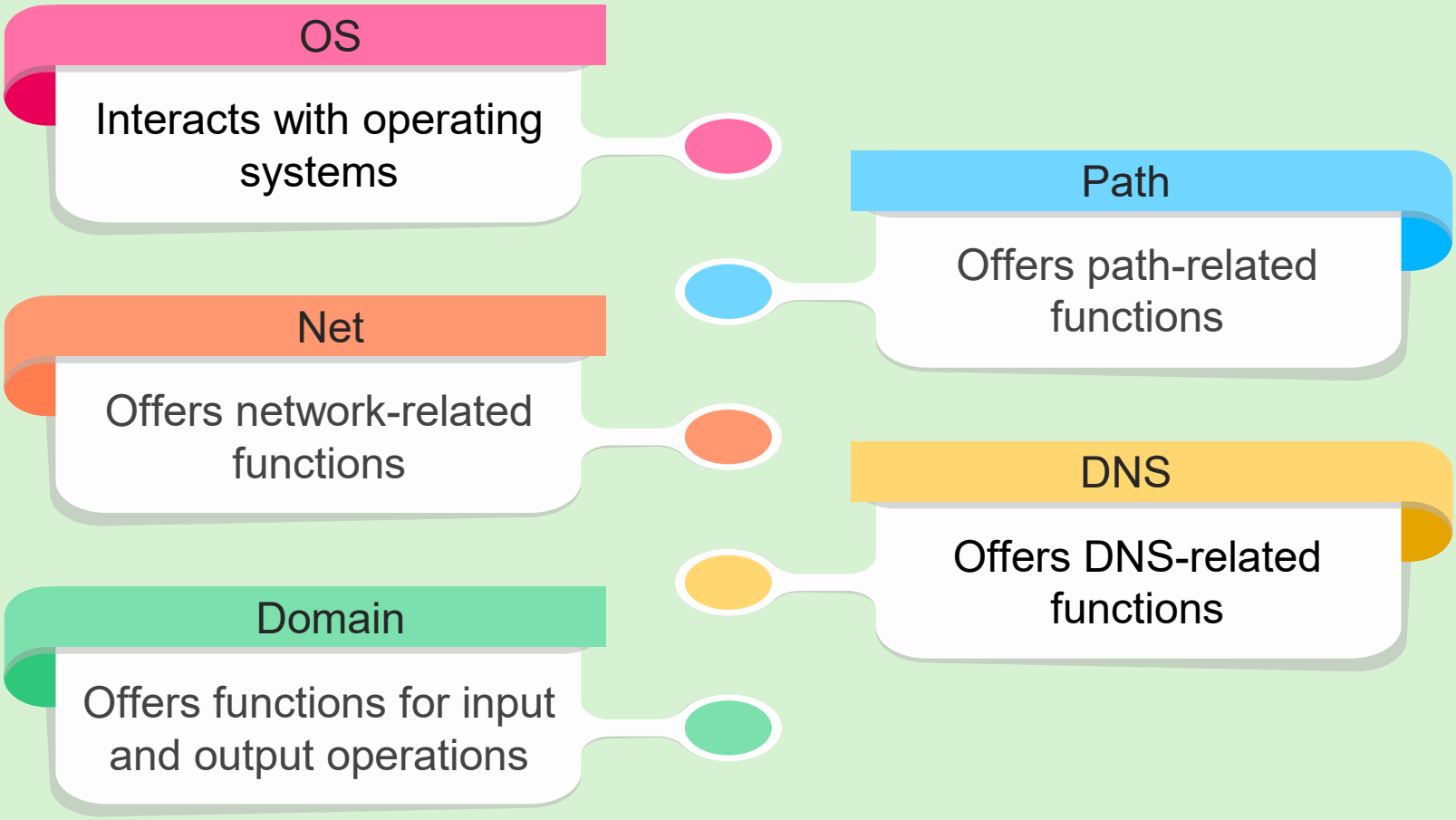


Provides with special libraries and tools for developing and managing front-end and back-end applications

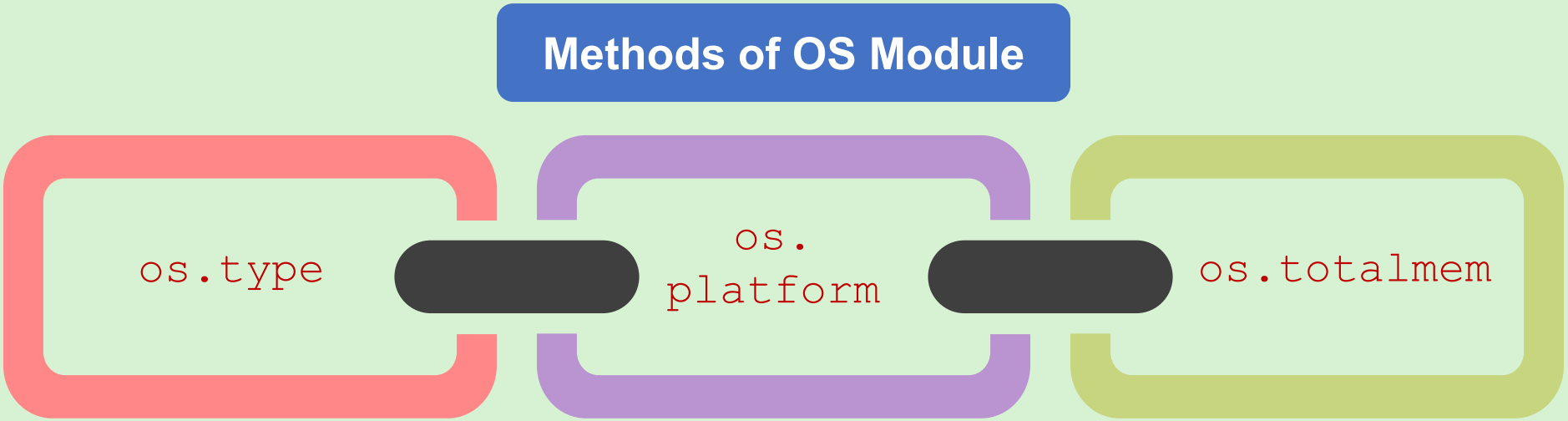


Defines a set of rules to enable communication between a server and a client using the HTTP methods

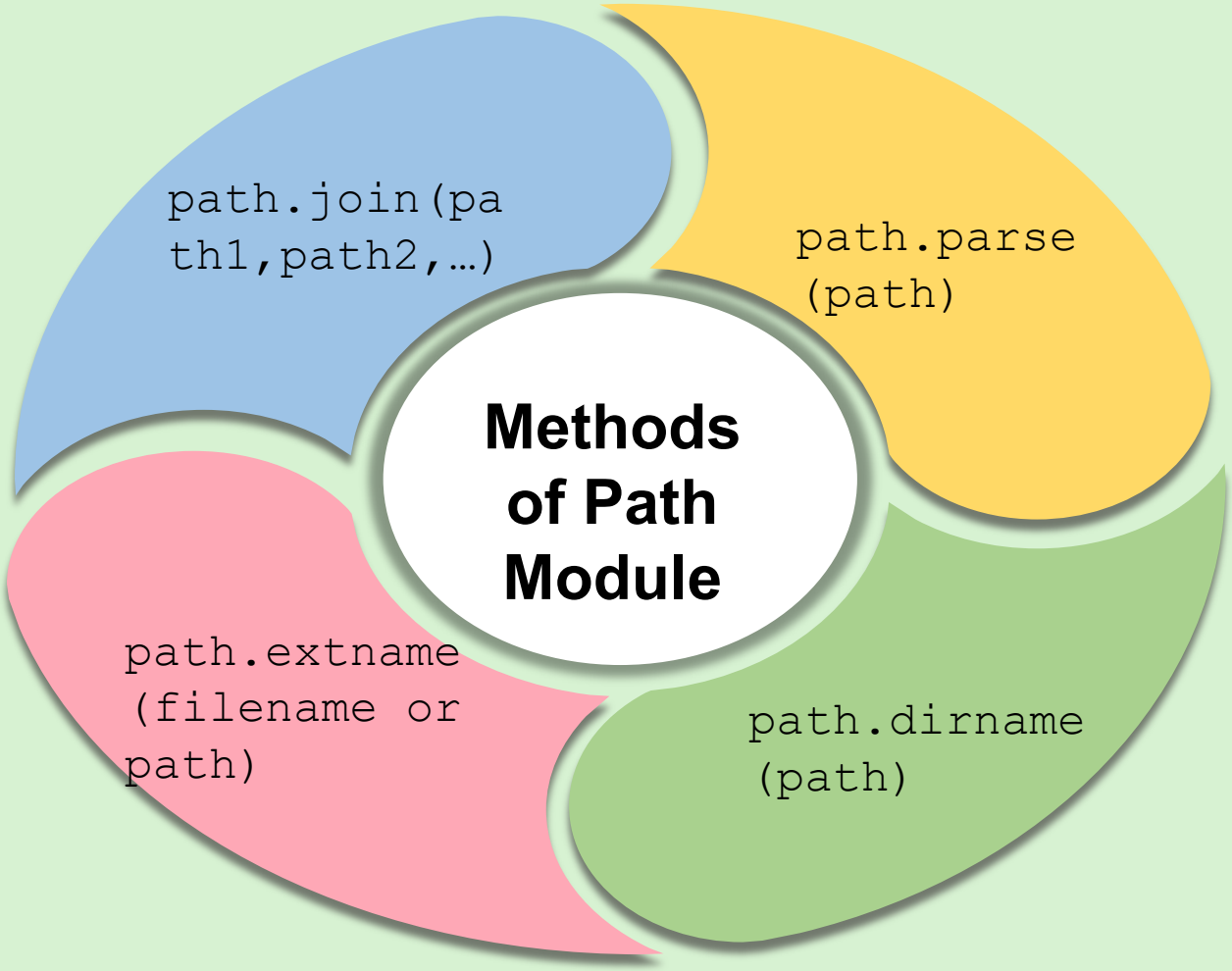
Utility Functions



Working with OS Module



Working with Path Module



Summary

- Modules are ready to use collection of functions.
- The three types of modules are core modules, local modules, and third-party modules.
- Core modules are built-in modules that come with the installation of Node.js.
- Packages are collections of one or more modules, which are installed using the NPM tool.
- Frameworks are a collection of libraries and tools that speed up the application development.
- Utility functions perform general-purpose tasks that can be useful for communicating with operating systems and working with files and directories.

Session: 4

Built-in Modules

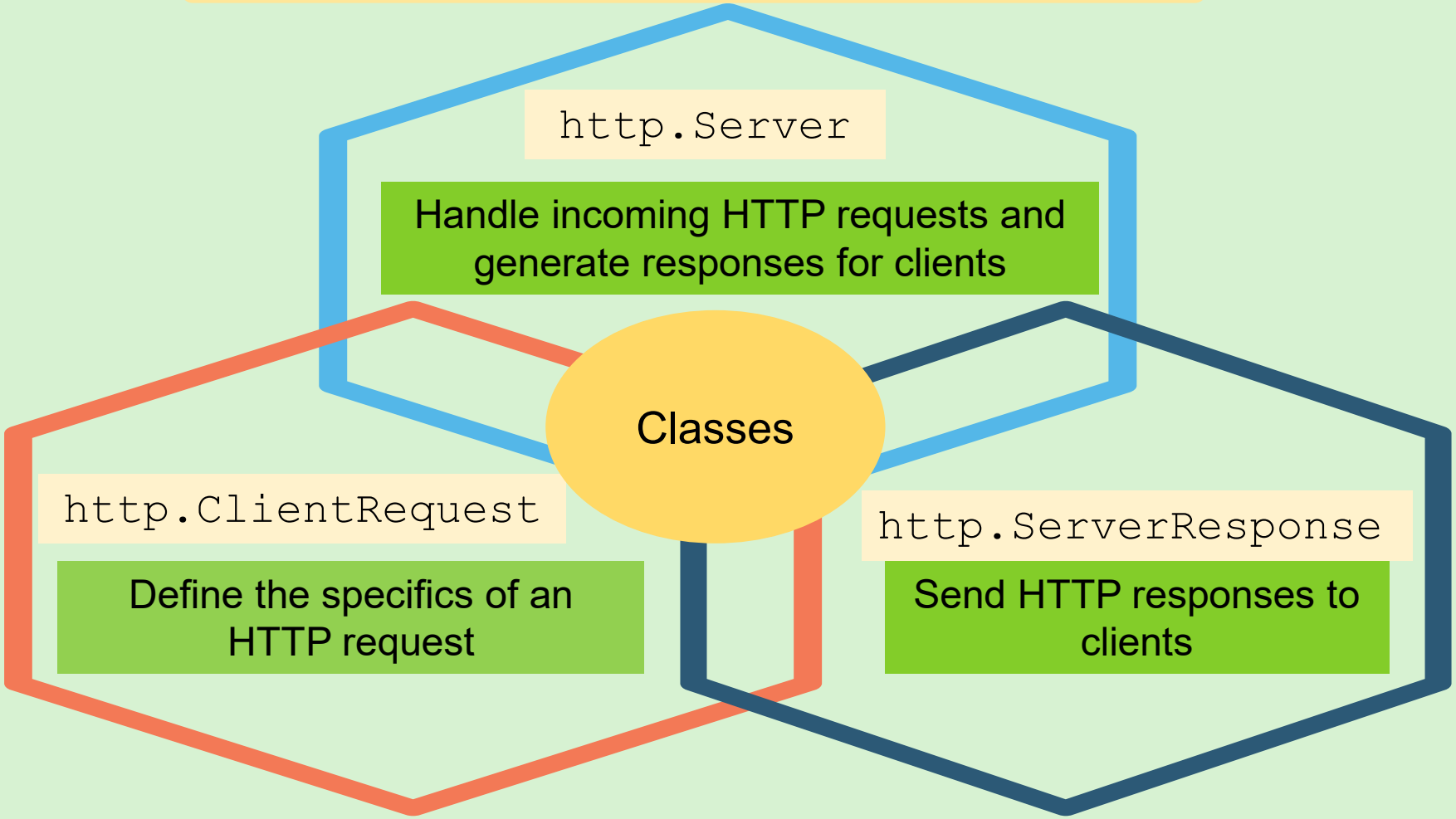
Server-side Development with Node.js

Objectives

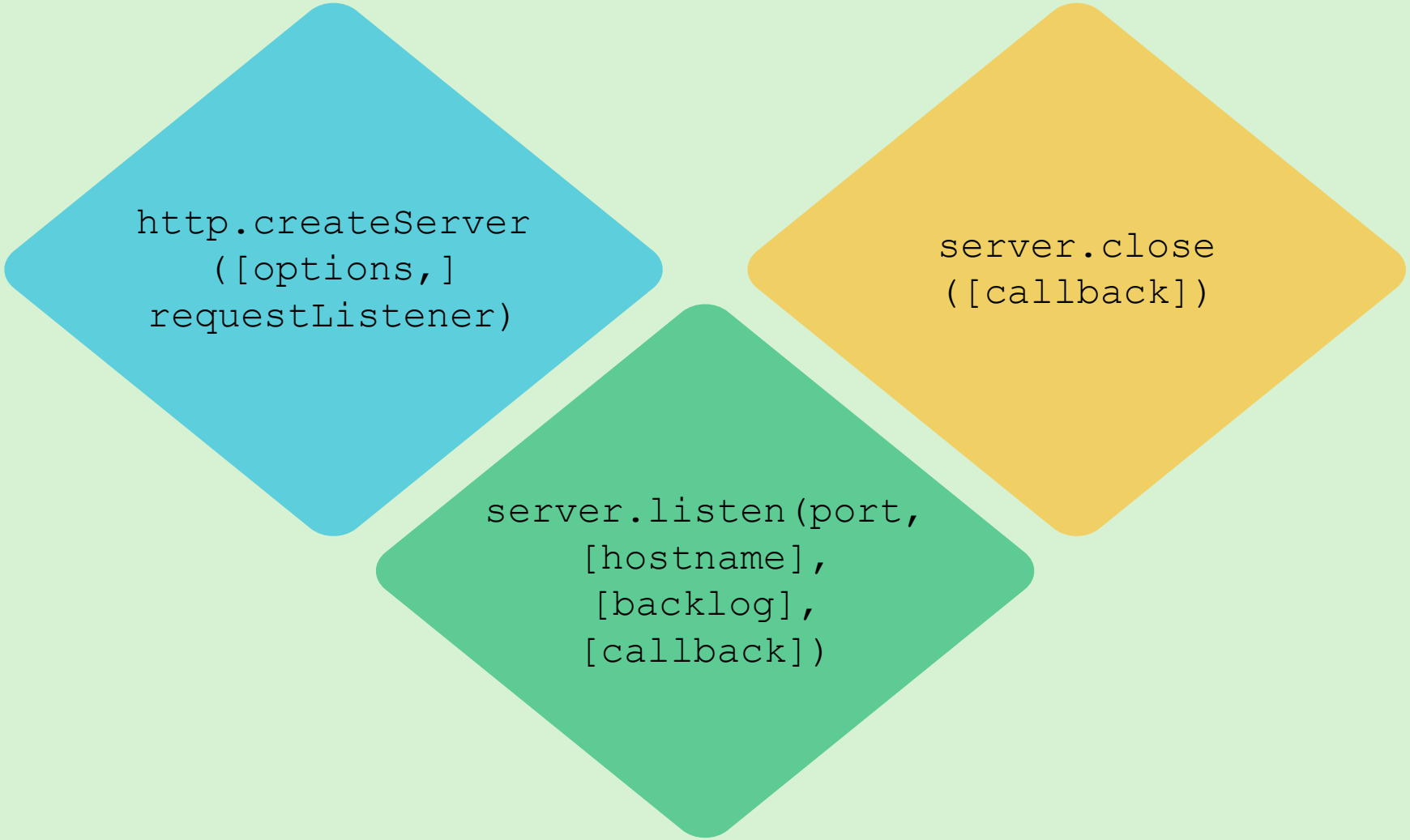
- Describe built-in modules in Node.js
- Explain `http` module to create an HTTP server and how to use it
- Explain how to use the `url` module to parse and work with URLs
- Explain how to use the `events` module to handle asynchronous events

The `http` Module

Can be used to create HTTP servers and make HTTP requests

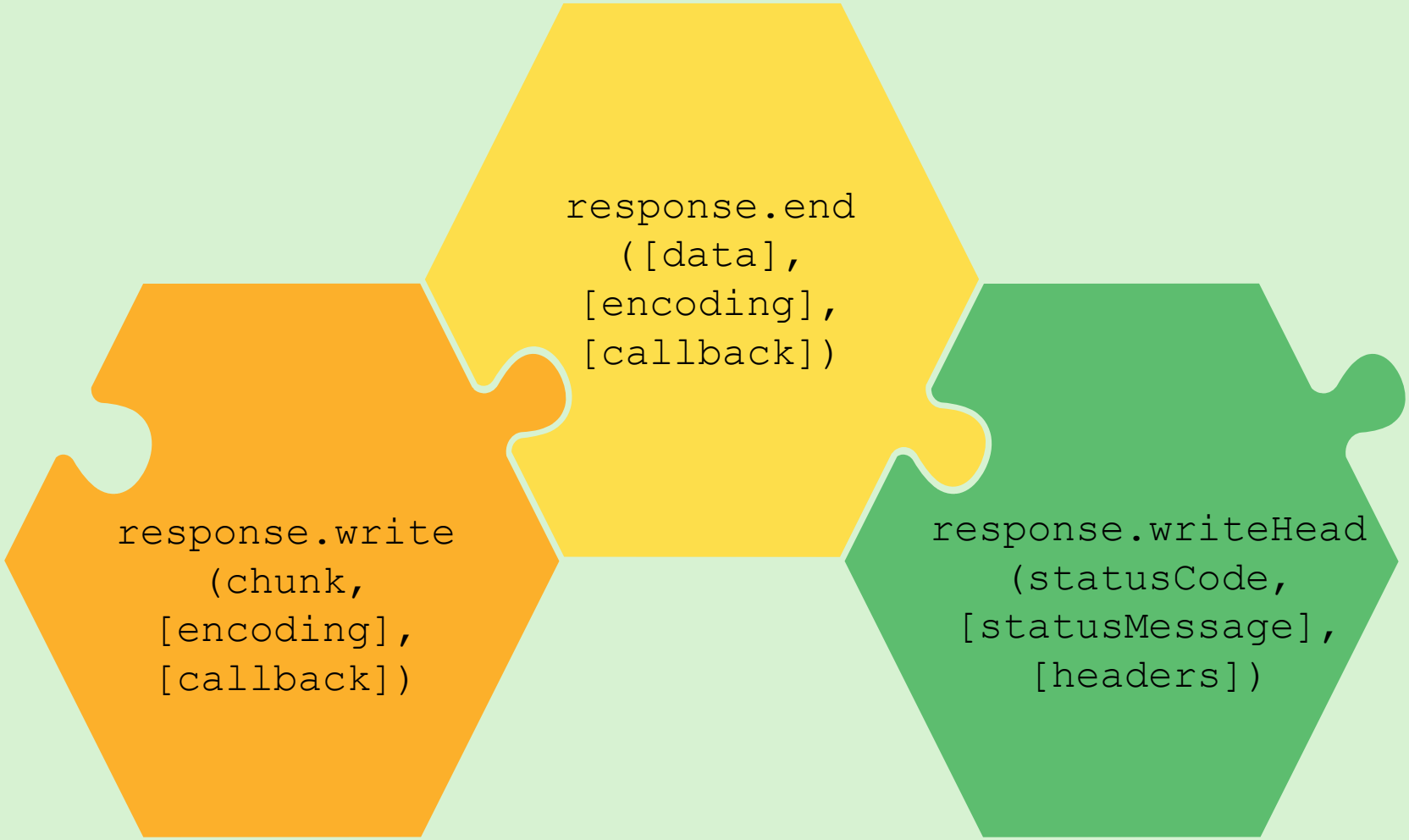


Methods of the `http.Server` Class



Methods of the `http.ClientRequest` Class

Methods of the `http.ServerResponse` Class



Creating a Simple HTTP Server

01 Open a text editor and type the code.

02 Import the HTTP module.

03 Create an HTTP server.

04 Handle the HTTP requests.

05 Set the format of the response to plain text.

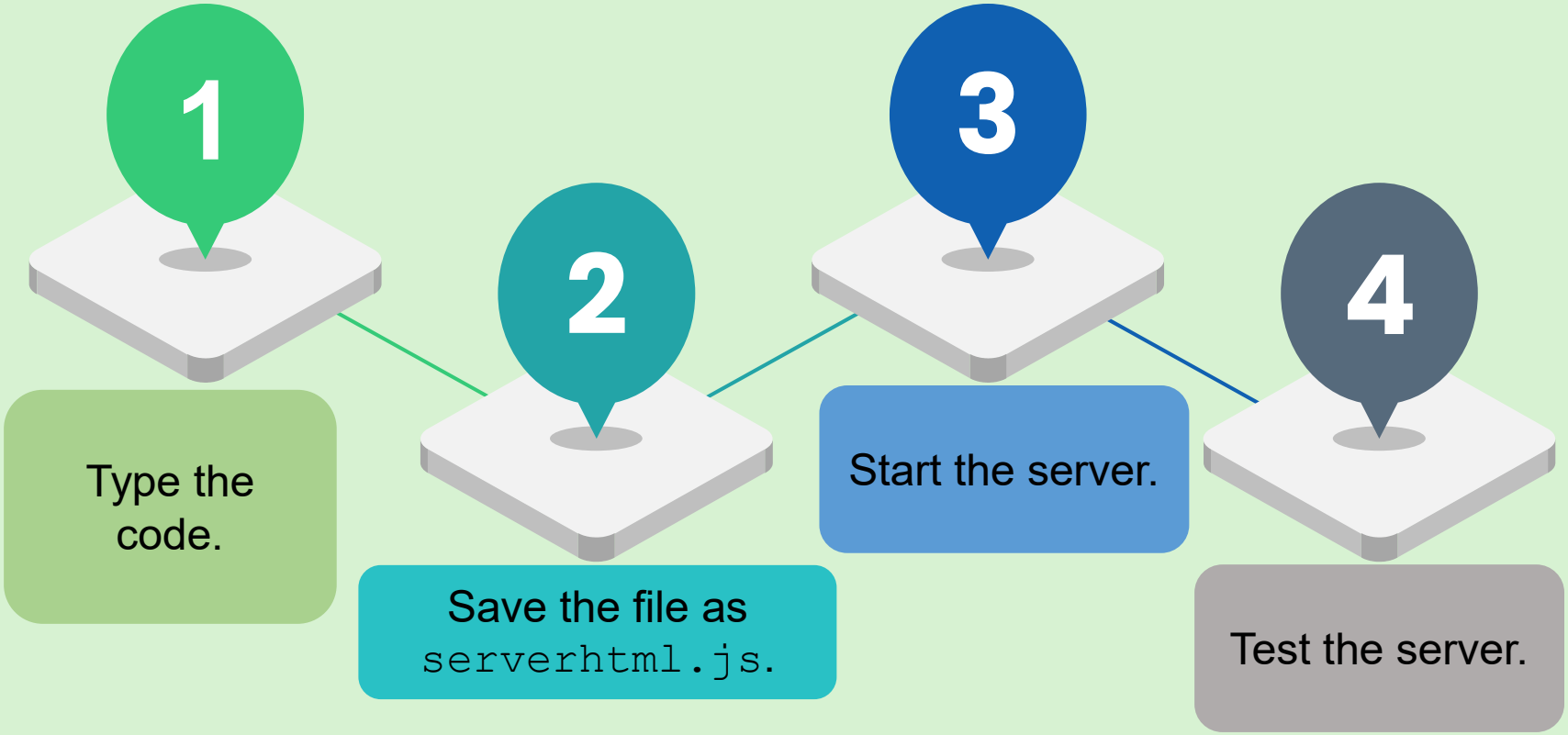
06 Specify the port.

07 Save the file as `servercreate.js`.

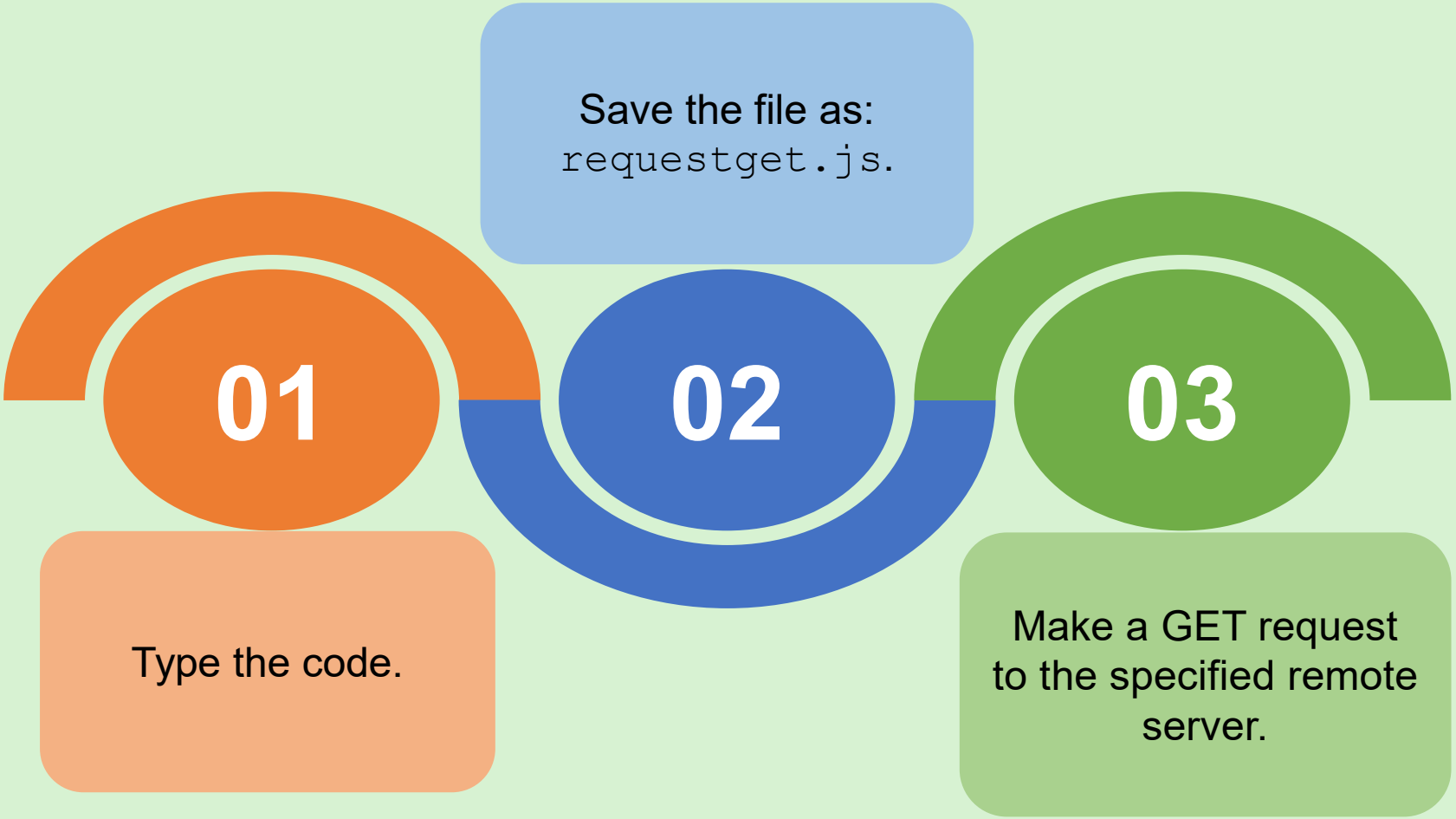
08 Start the server.

09 Test the server.

Creating an HTML Response for the HTTP Server



Creating an HTTP Client Request Using the GET Method



The `url` Module

Application

Parse URLs into their components.

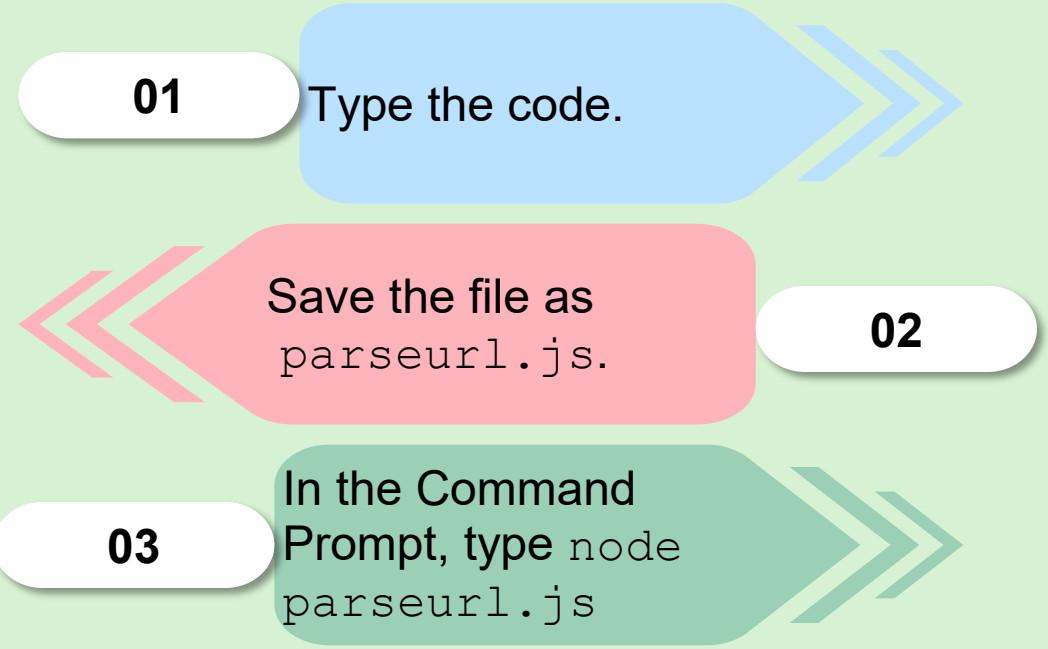
Build URLs from components.

Retrieve specific components of the URL

Parsing the URL

`url.parse()` method: Parses a URL string into an object having different components of the URL

To parse the URL string:
https://www.jsonplaceholder.typicode.com/todos/61

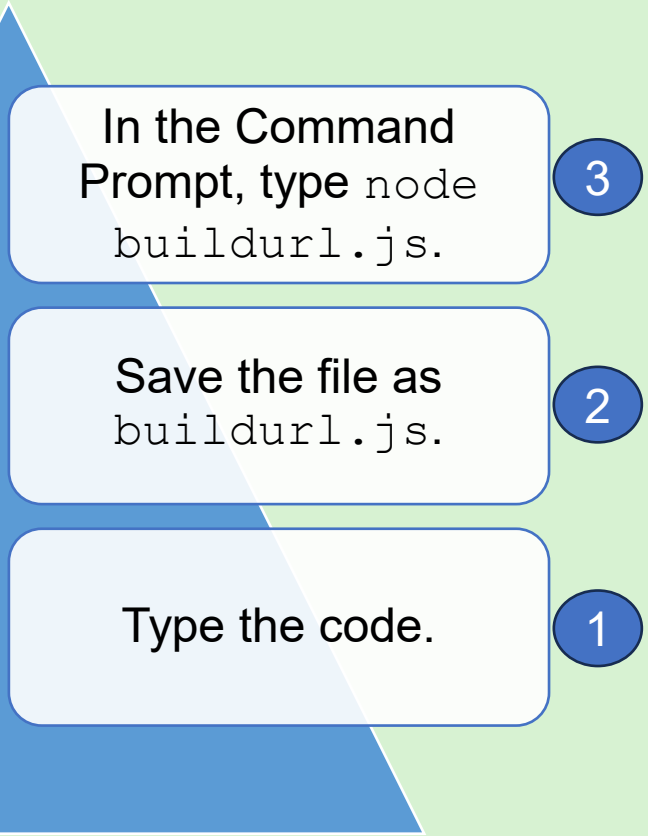


Building the URL

`url.format()` method: Creates a URL string from a URL object or URL components

To build the URL string using:

- `protocol: 'https:',`
- `hostname: 'www.jsonplaceholder.typicode.com',`
- `port: '8080',`
- `pathname: '/todos/61'`

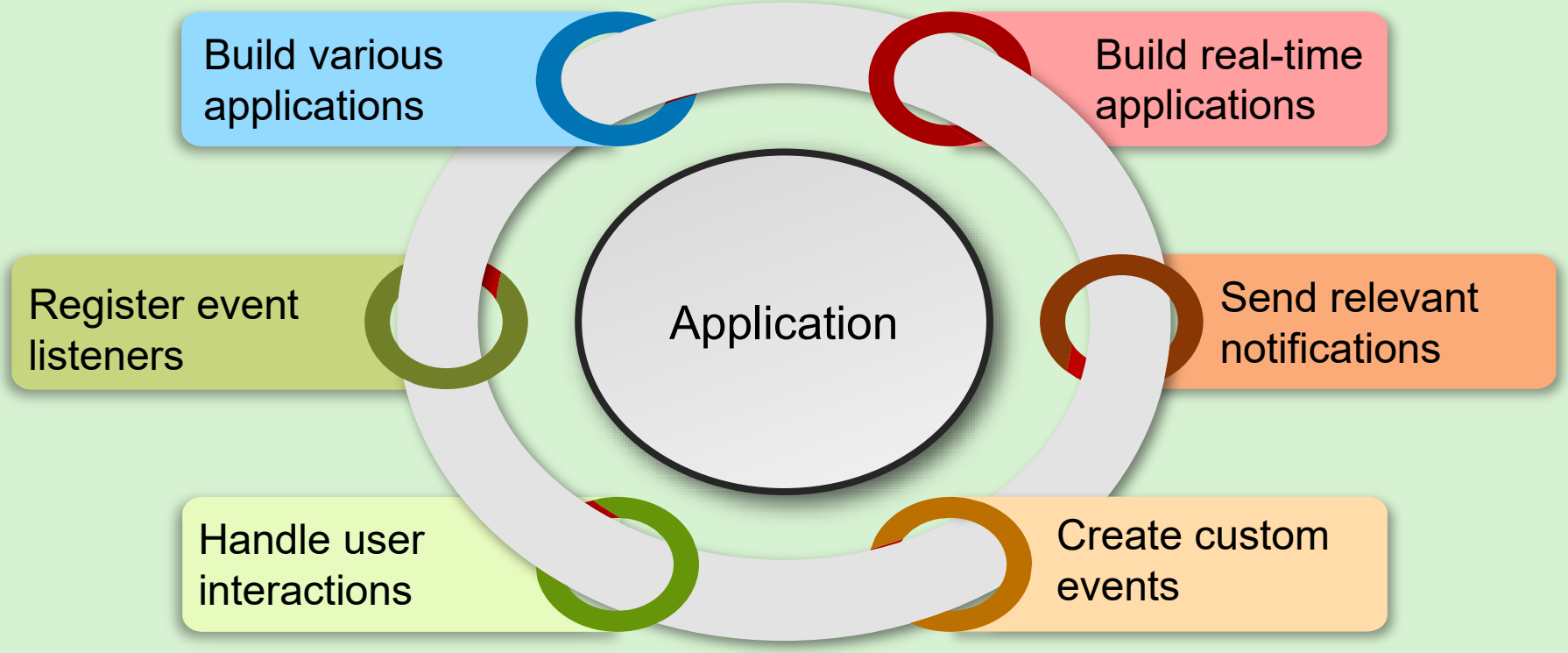


Retrieving Specific Components of the URL

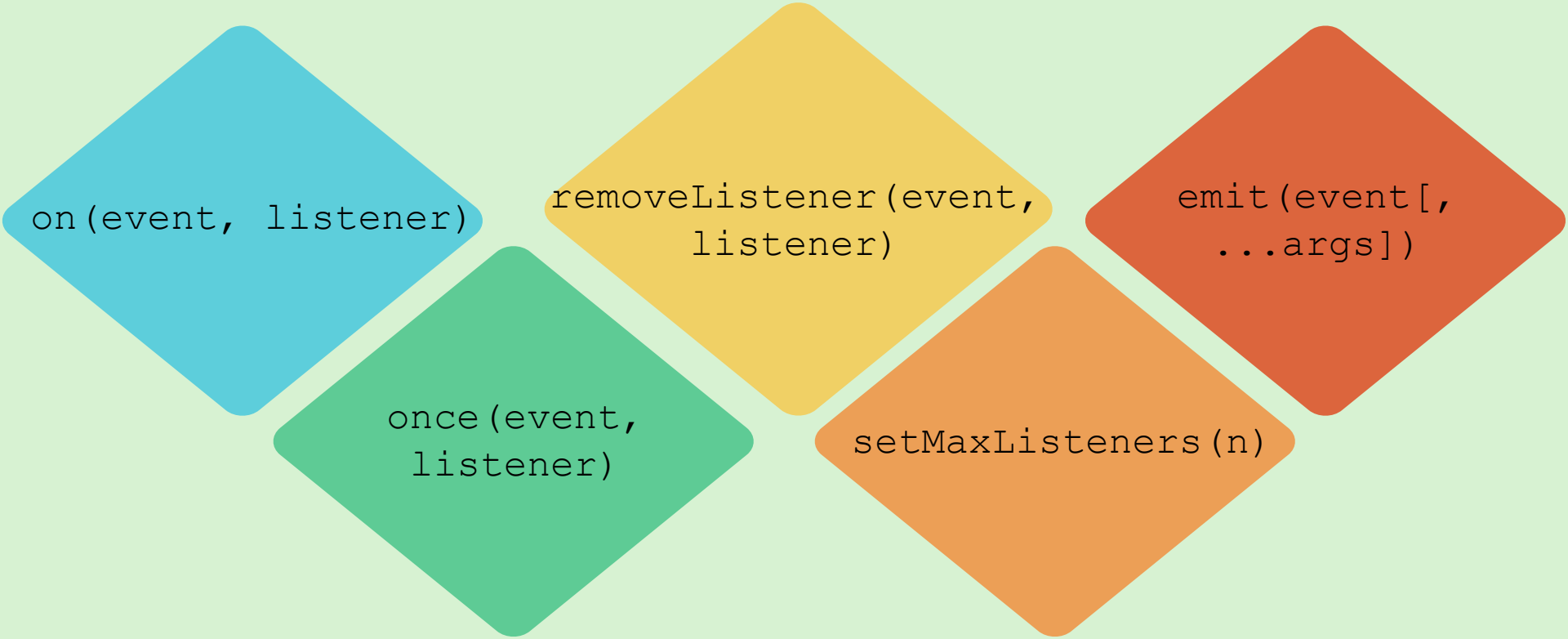
The `events` Module

`events` module:

- Includes the `EventEmitter` class to work with events in Node.js
- Helps to create custom event emitters by extending the `EventEmitter` class



Methods of the `events` Module



Working with the `events` Module

Creating an event driven notification system

1. Type the code.
2. Save the file as `eventemit.js`.
3. In the Command Prompt, type `node eventemit.js`.

Managing event listeners in the '`sendNotification`' event

1. Type the code.
2. Save the file as `eventonce.js`.
3. In the command prompt, type `node eventonce.js`.

Summary

- Node.js built-in modules offer essential functionality for a wide range of tasks, including Web servers, file systems, and more, with benefits including performance, reliability, and interoperability.
- The Node.js `http` module simplifies the creation of HTTP servers and requests, providing various methods and classes for interacting with HTTP protocols.
- Common classes in the `http` module in Node.js include `http.Server`, `http.ClientRequest`, and `http.ServerResponse`.
- The Node.js `url` module offers utilities for parsing and manipulating URLs.
- The Node.js `events` module uses the `EventEmitter` class to facilitate event-driven programming for building applications and handling I/O, user interactions, real-time systems, and custom events.

Session: 5

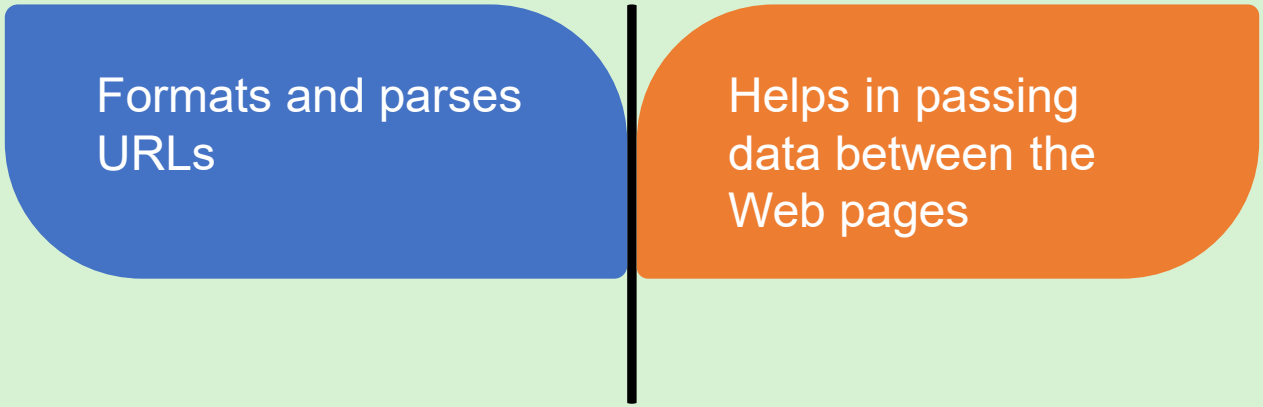
More Built-in and Local Modules

Server-side Development with Node.js

Objectives

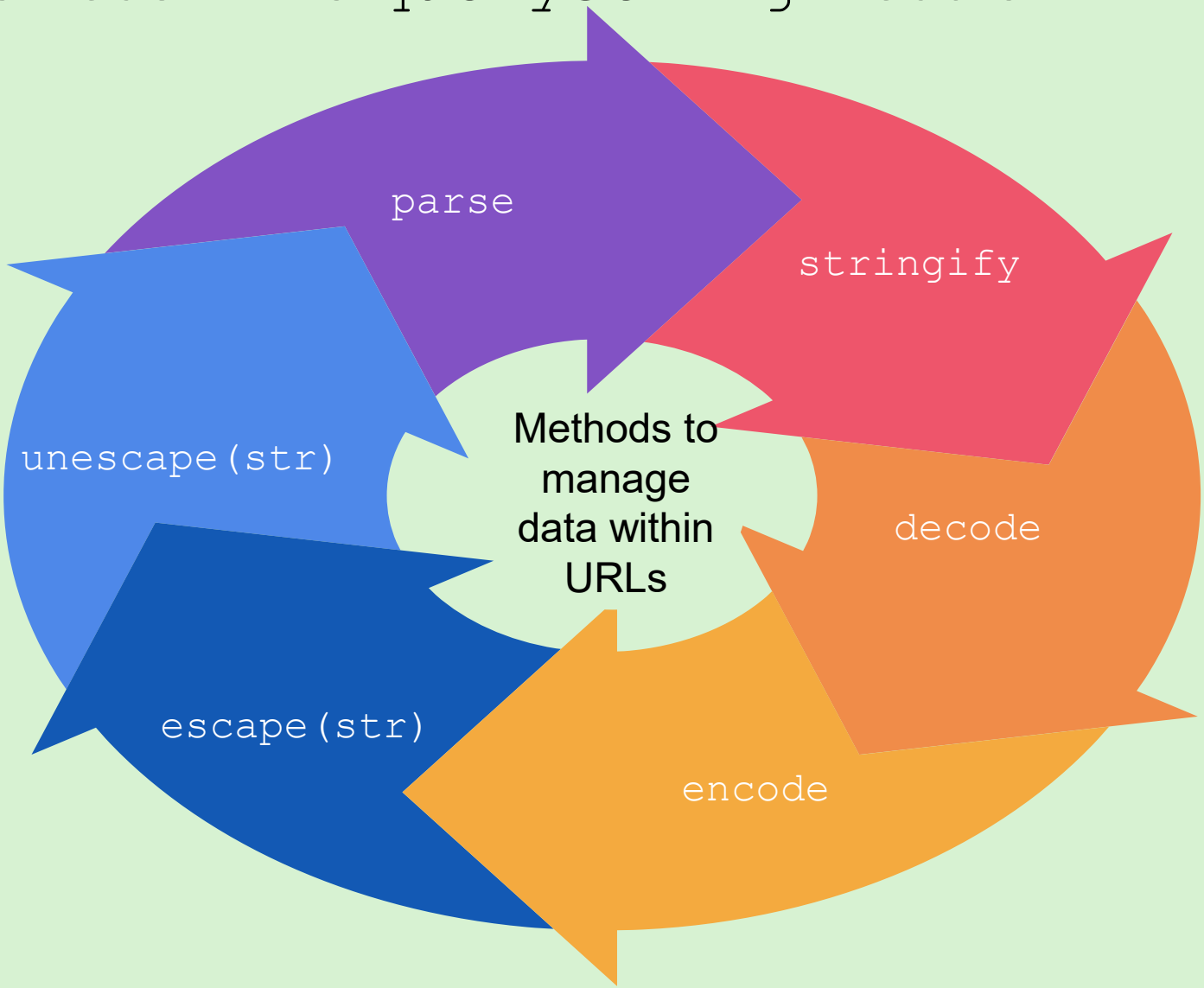
- Explain the purpose and usage of `querystring` in Node.js
- Describe how to send emails using Node.js
- Describe the process of uploading files using Node.js
- Explain the concept of local modules and their application in Node.js

`querystring` Module

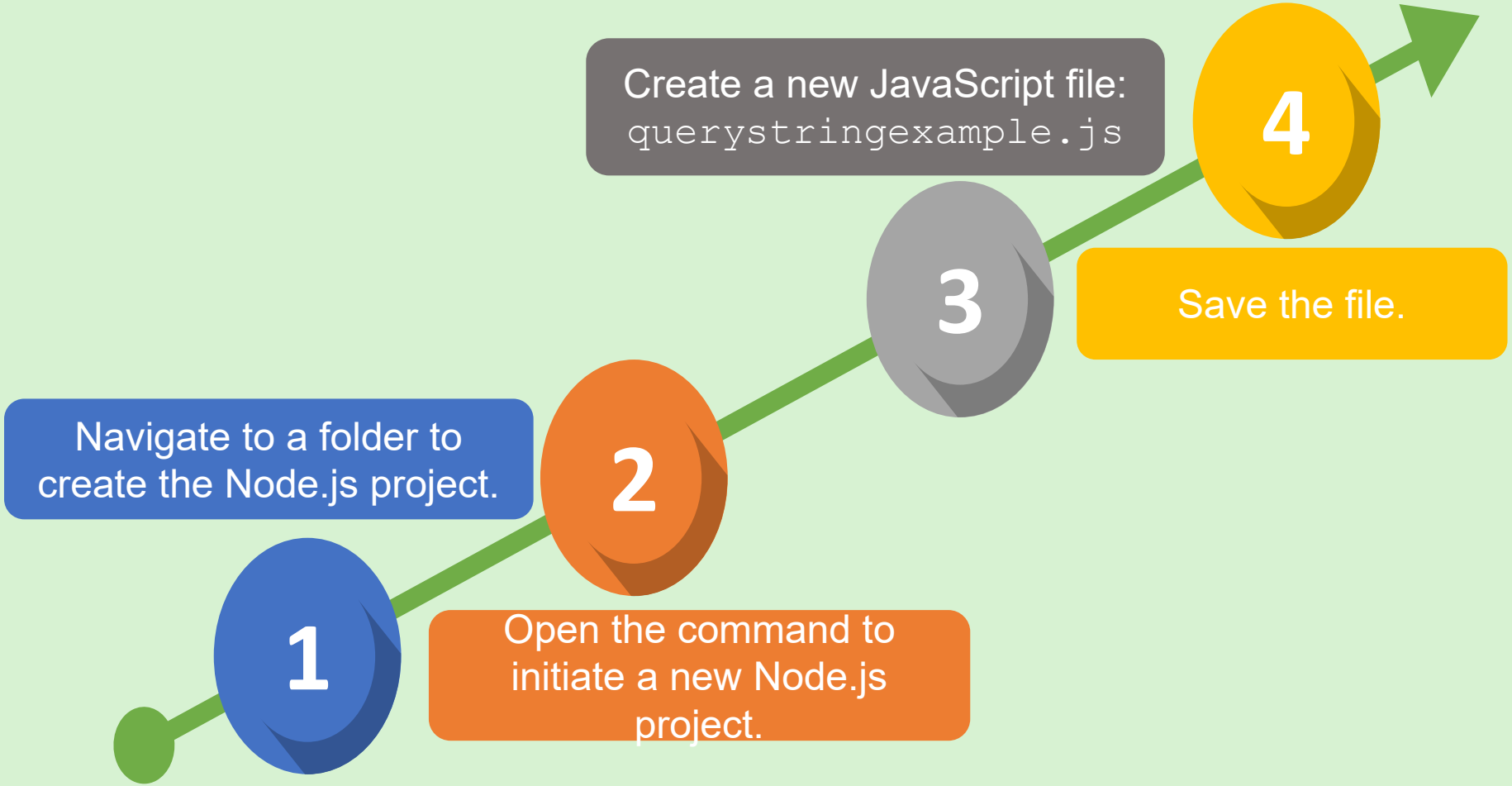


The user can add extra data in the form of key-value pairs to a URL, which is transferred to Web server as an HTTP request.

Methods in the `querystring` Module



Setting up a Basic Node.js Application



Using the `querystring.parse` Method

Syntax

```
querystring.parse( str[, sep[, eq[, options]]]) )
```

1. Parsing a URL query string.
2. Stringifying an object into a URL query string.
3. Parsing a query string with custom delimiters.
4. Parsing a query string with `maxKeys` set to 1.
5. Parsing a query string with `maxKeys` set to 2.
6. Parsing a query string with `maxKeys` set to 0.

querystring.stringify Method

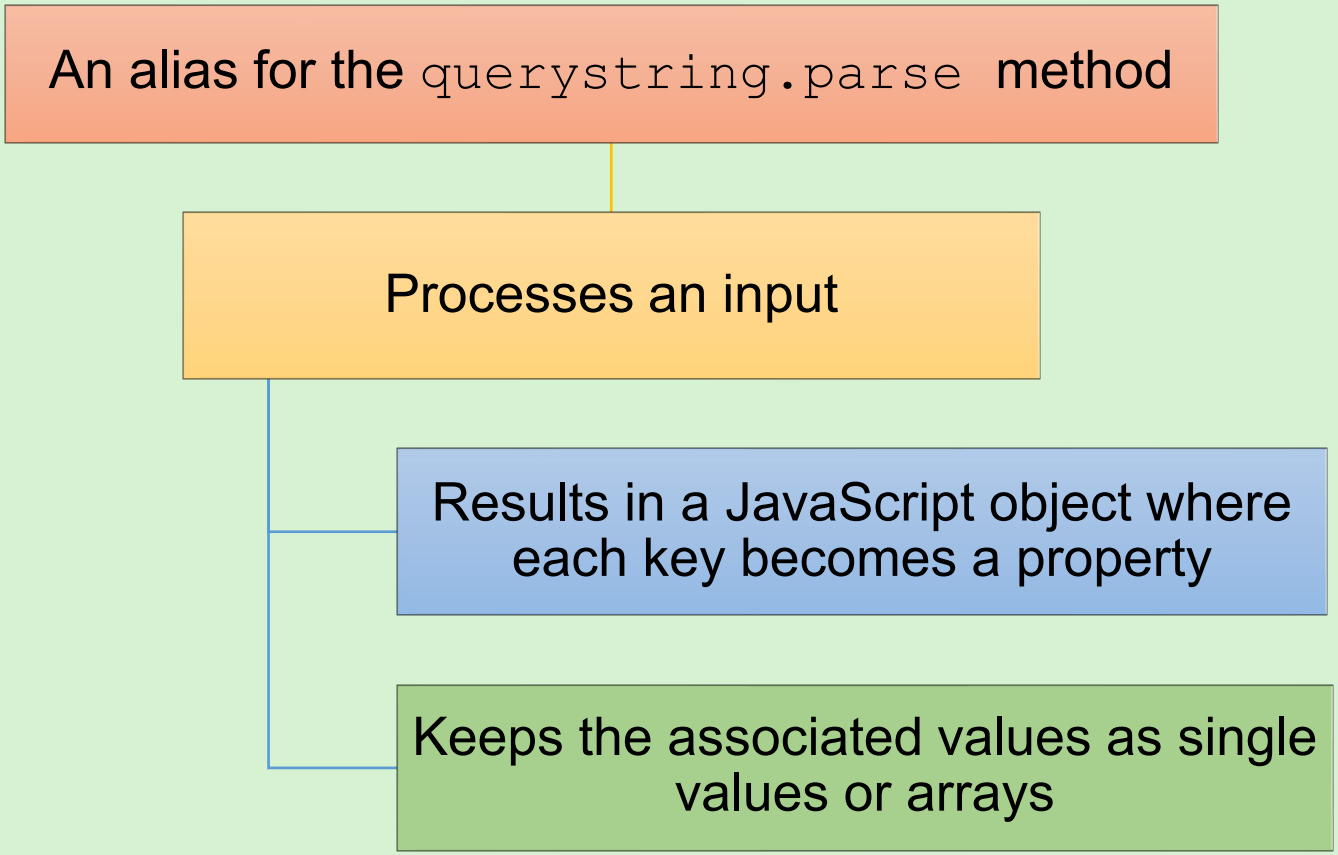
Syntax

```
querystring. stringify( obj[, sep[, eq[, options]]]) )
```

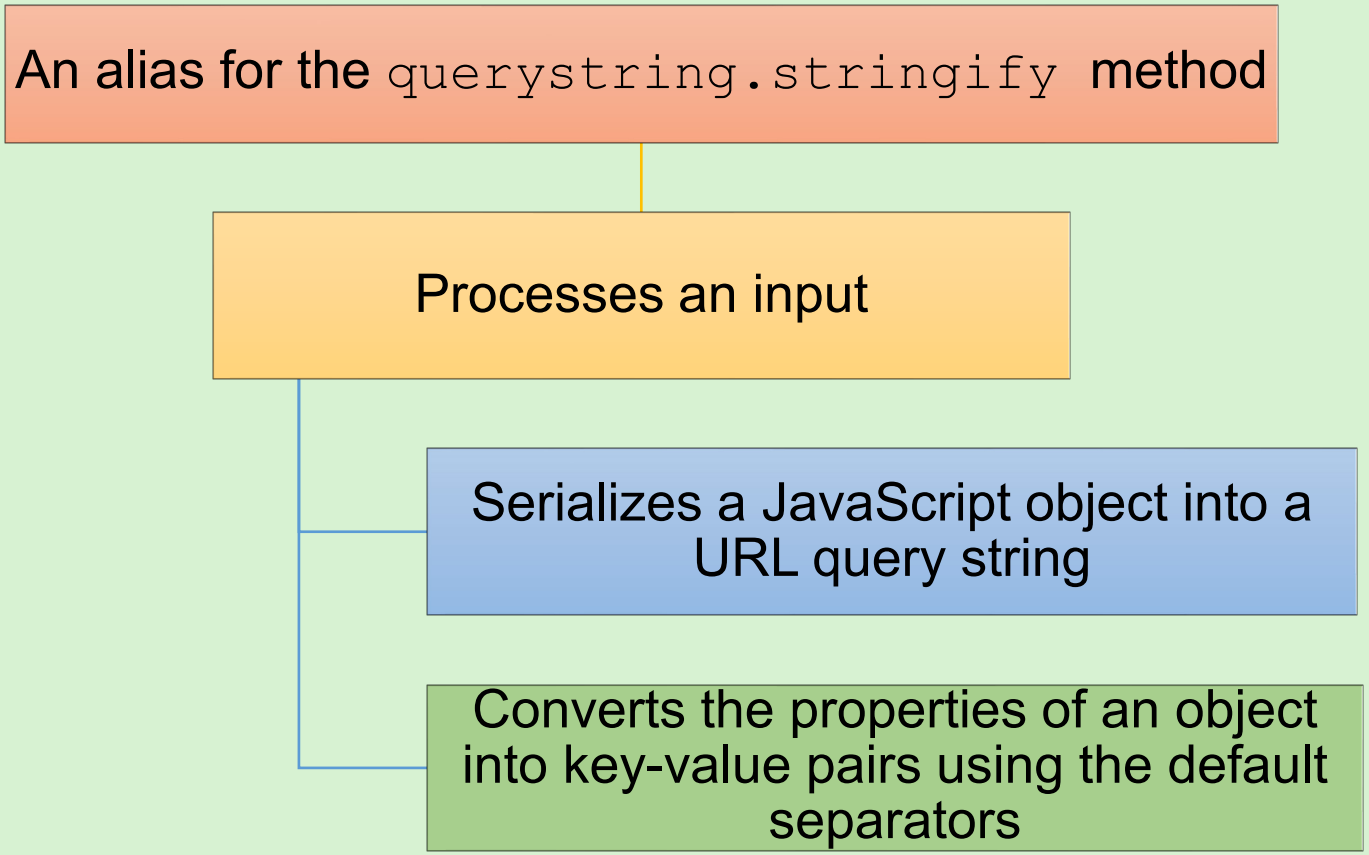
Working with the default behavior of the `stringify` method when converting objects into URL strings.

Using custom separators and delimiters to achieve object serialization.

`querystring.decode` Method



`querystring.encode` Method



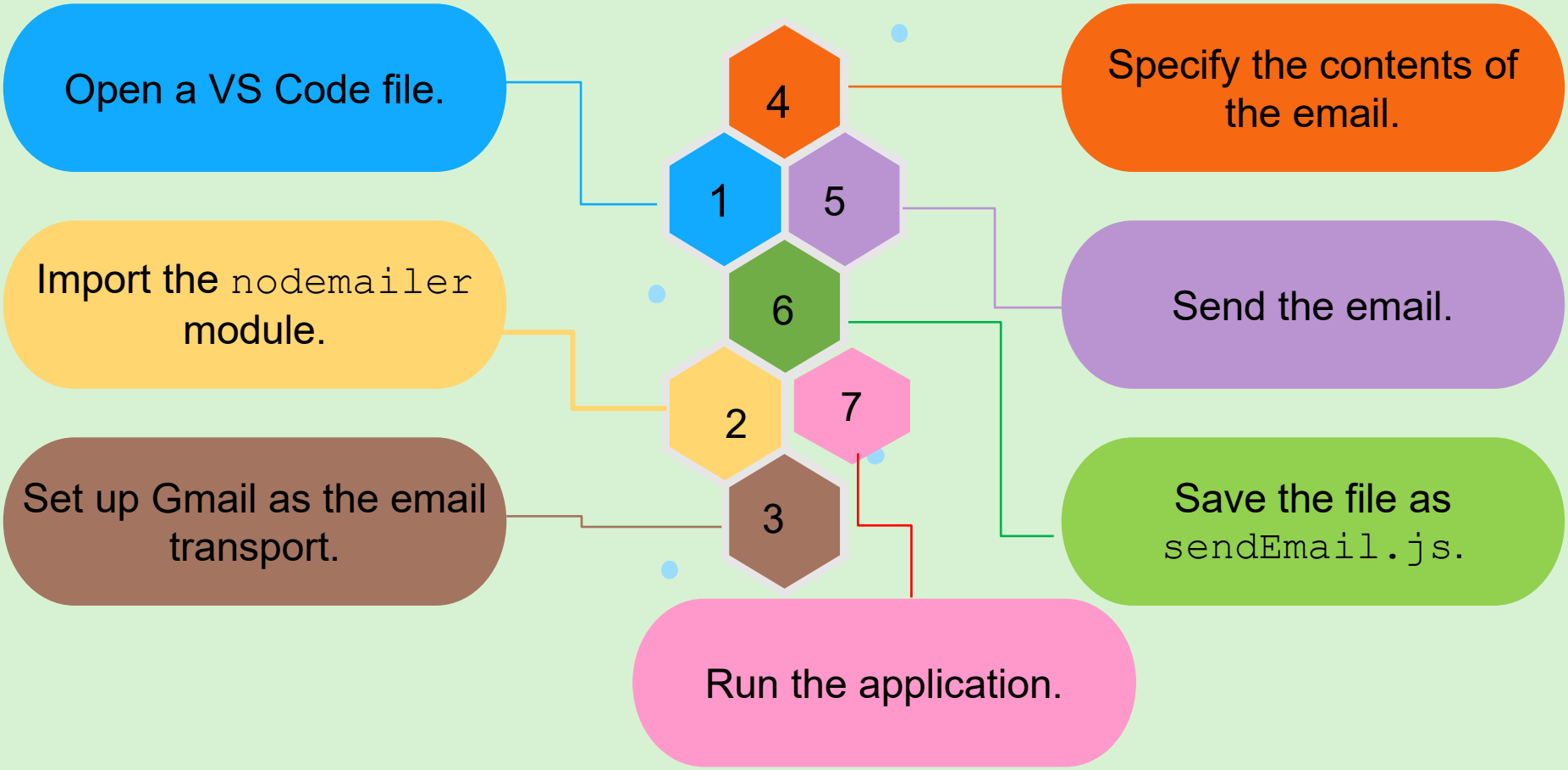
`nodemailer` Module

This module is part of a third-party library that helps users to work with emails in Node.js applications.

To install the `nodemailer` module on the local system:

Run: `npm install nodemailer`

Sending an Email from a Node.js Application



Uploading Files

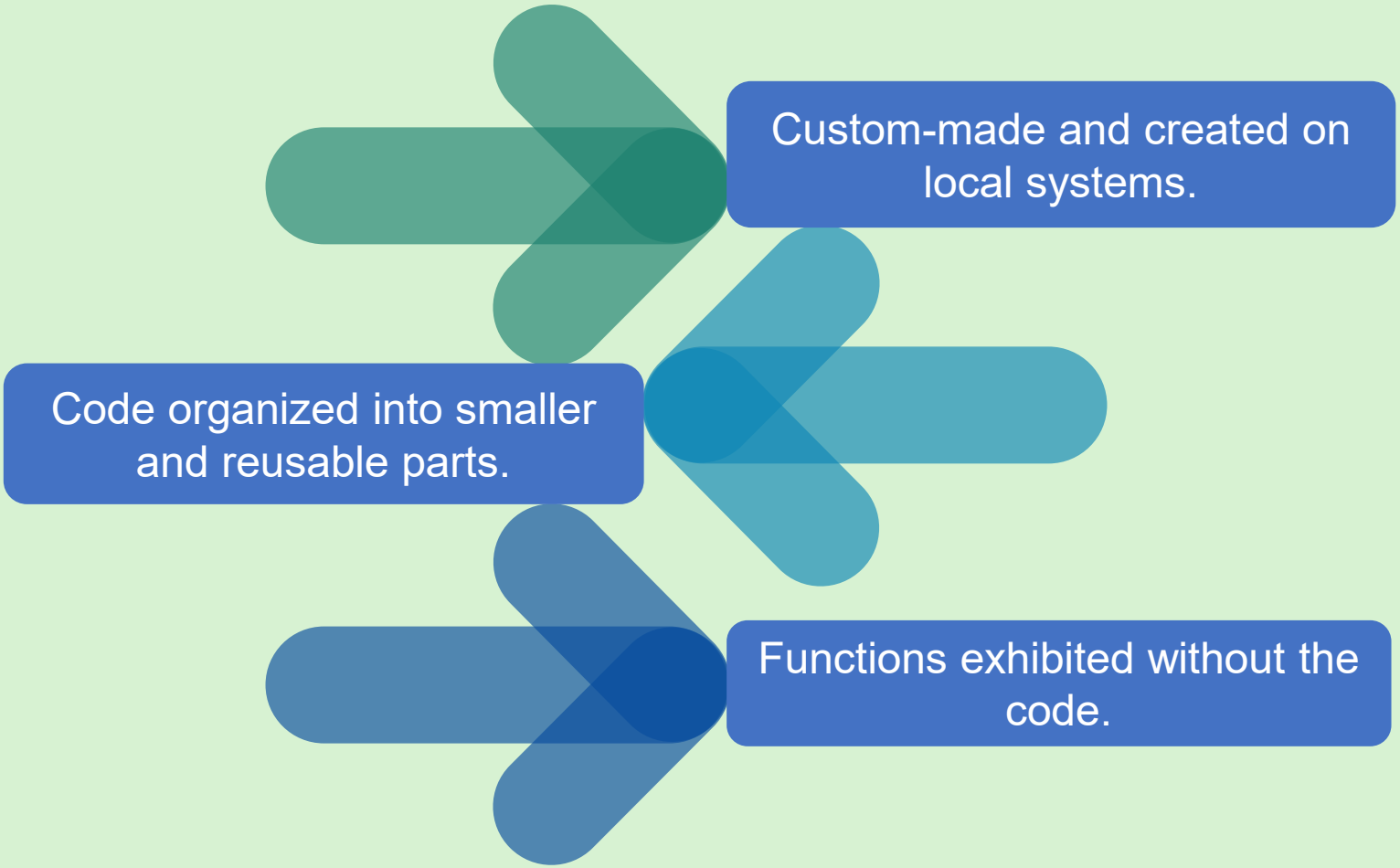
Use third-party modules (`formidable`) for uploading media files and documents.

Install the `formidable` module by running the command: `npm install formidable`.

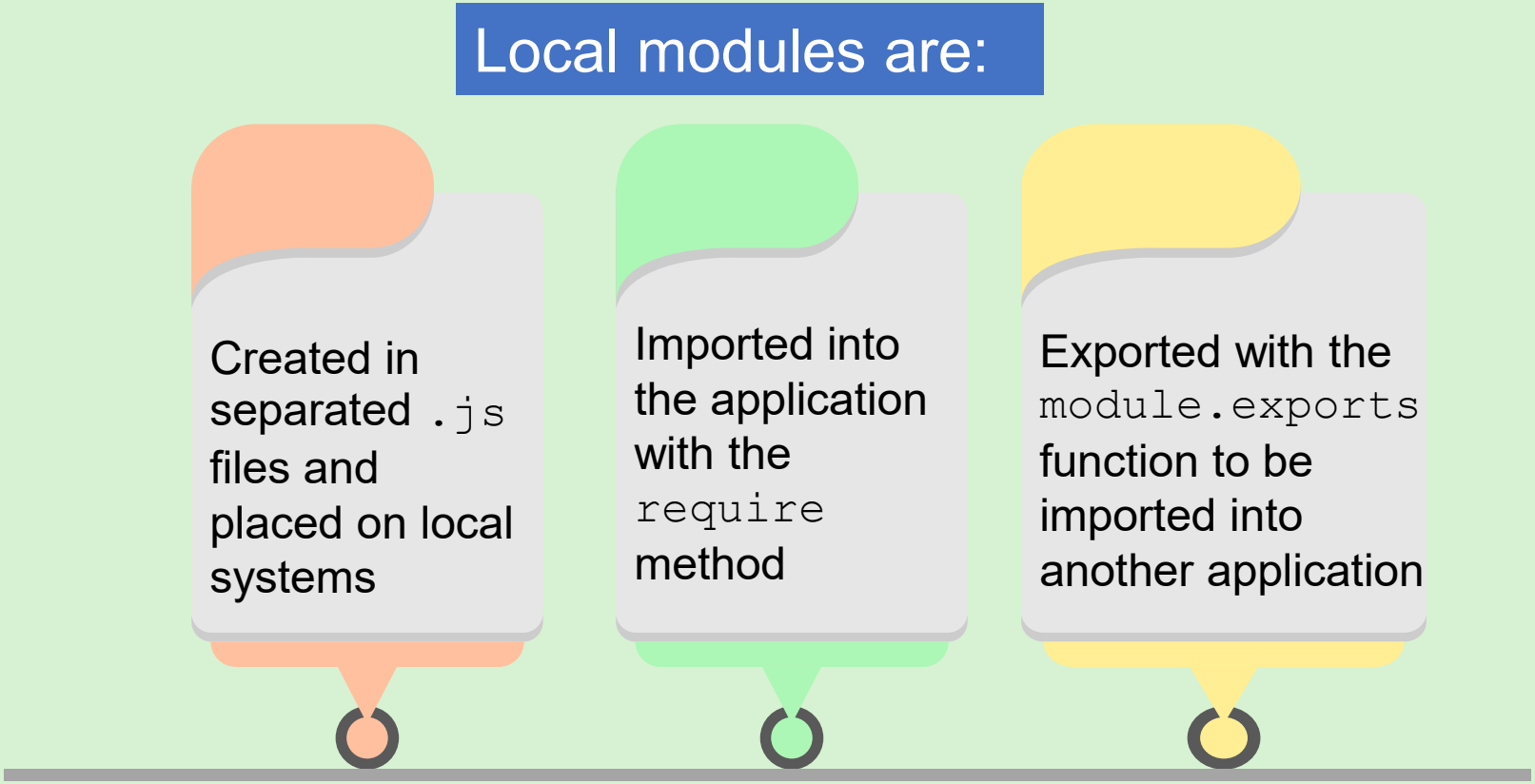
Creating an Application to Upload Files

- Create an HTML form to upload a file.
- Configure the HTTP server to parse the uploaded file.
- Display the uploaded file form into the `else` block.
- Save the file as `uploadFile.js`.
- Run the application.
- Navigate to the URL http://localhost:3000.
- Select a file from the local system to upload.
- Upload the file.

Local Modules



Working with Local Modules



Summary

- The `querystring` module simplifies the parsing and manipulation of query strings within URLs.
- The essential methods of the `querystring` module are `parse`, `stringify`, `decode`, `encode`, `escape(str)`, and `unescape(str)`.
- Node.js relies on third-party libraries such as `nodemailer` for email-related tasks, allowing users to send emails programmatically and improve communication within applications.
- Node.js lacks a built-in module for file uploads but supports the usage of third-party modules such as `formidable`, which simplifies the process of receiving and handling uploaded files in Node.js applications.
- Local modules enable users to create custom code modules for better organization and reusability in applications.
- The `module.exports` object is a fundamental part of Node.js, included in every JavaScript file by default.

Session: 6

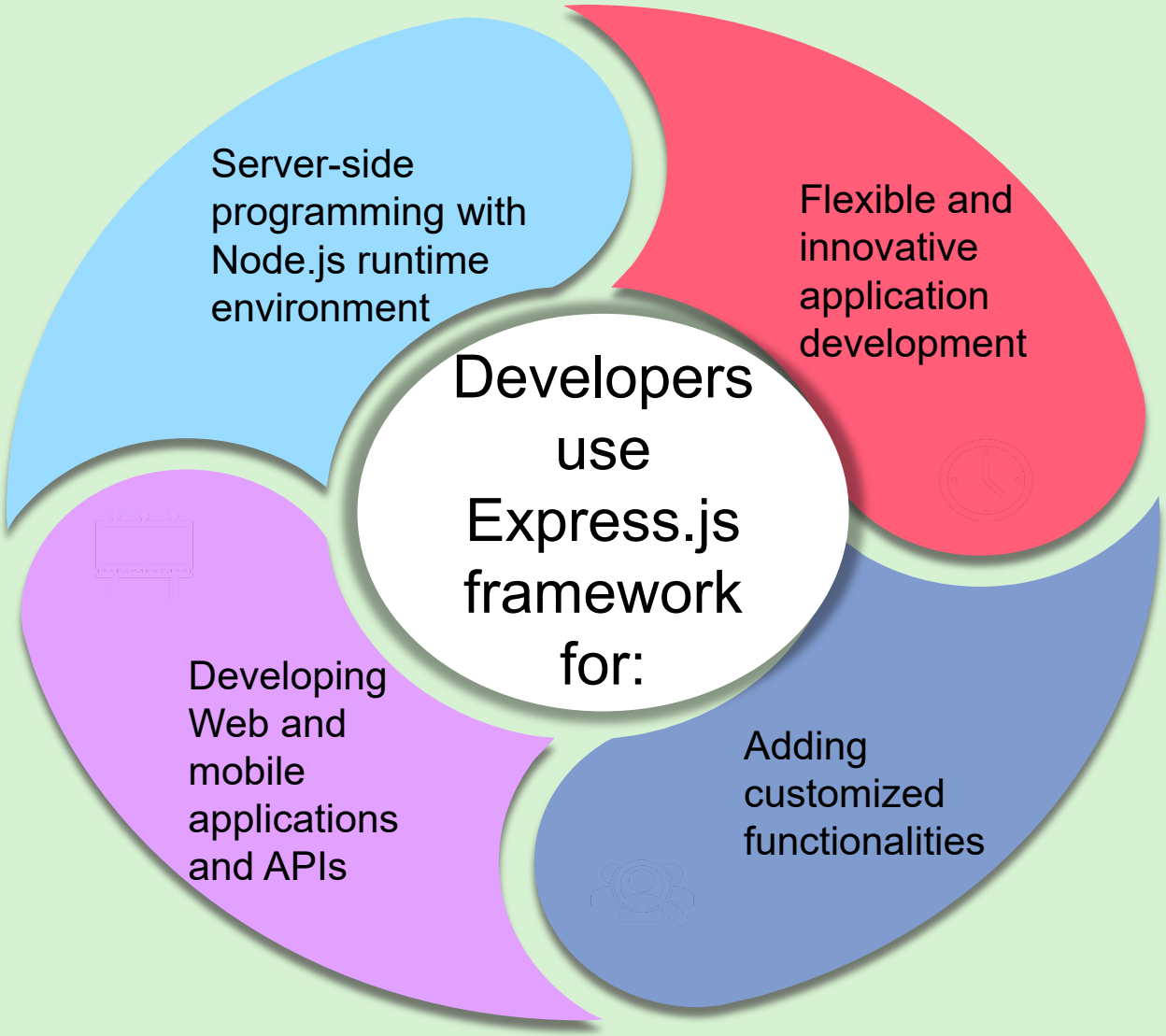
Introduction to Express.js Framework

Server-side Development with Node.js

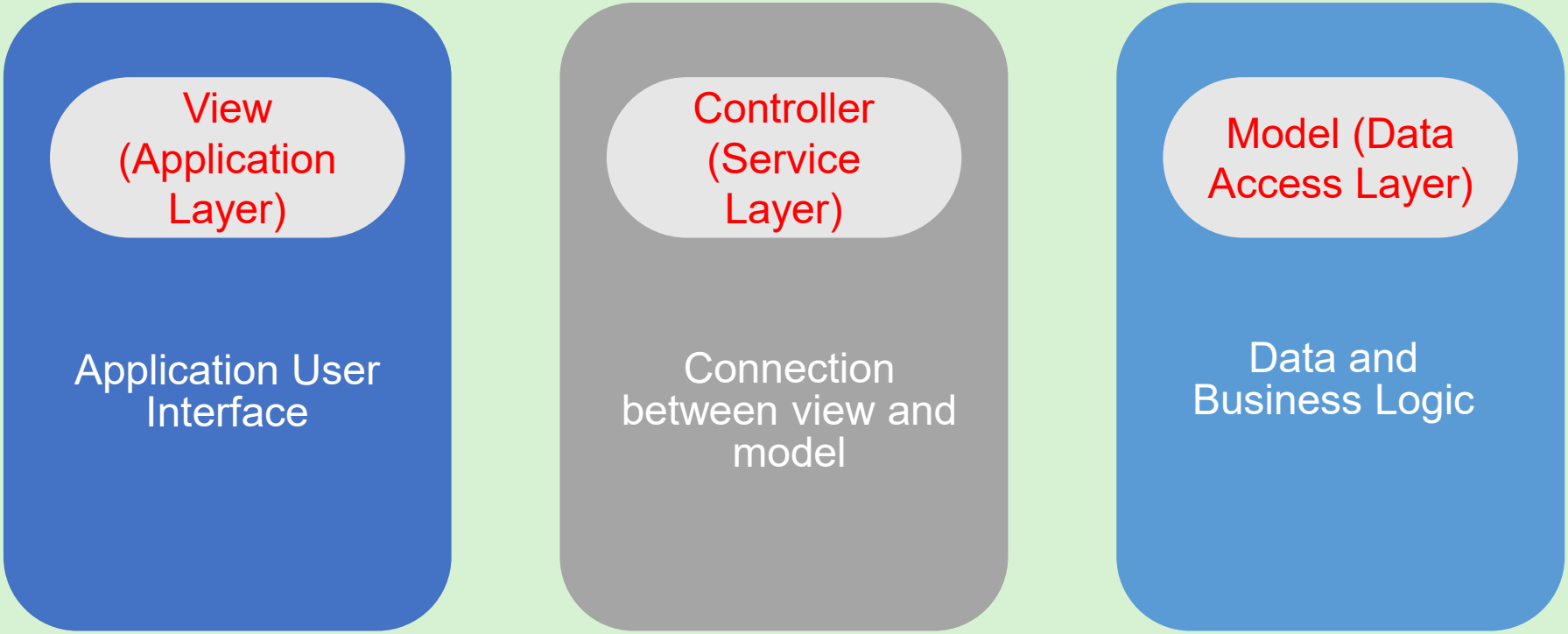
Objectives

- Describe the Express.js framework
- Explain the architecture of Express.js
- List the advantages of using Express.js
- Explain the working of Express.js

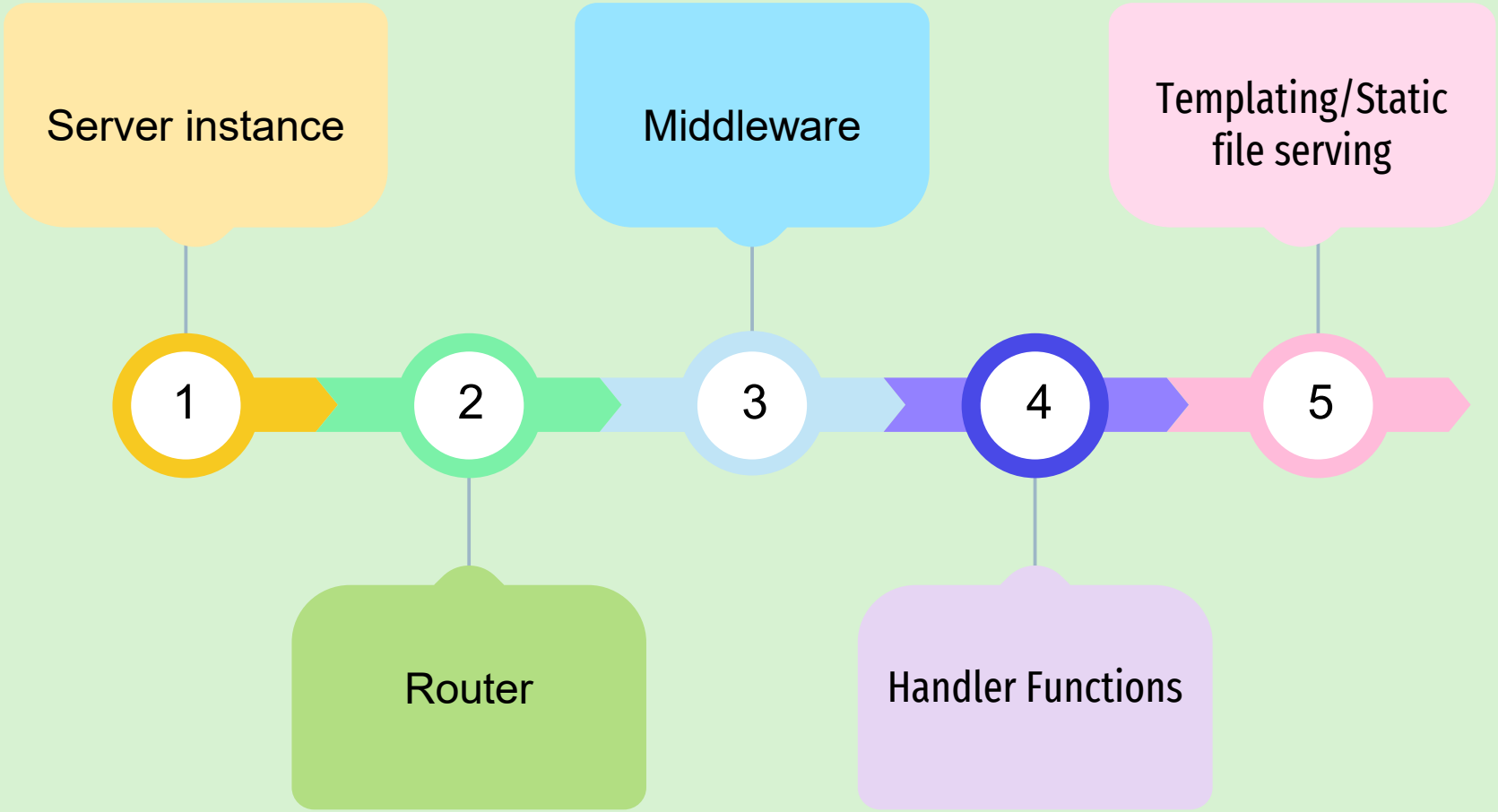
What is Express.js?



Express.js Architecture



Working of Express.js



Why use Express.js?

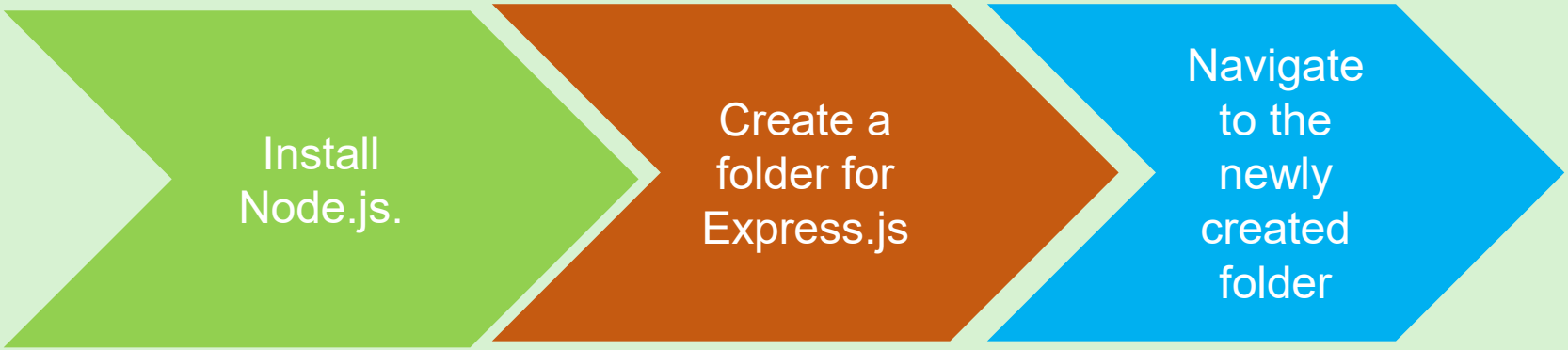
Express.js is:

- A minimalist framework
- Extensible and easy to use
- Compatible with several databases and templating engines

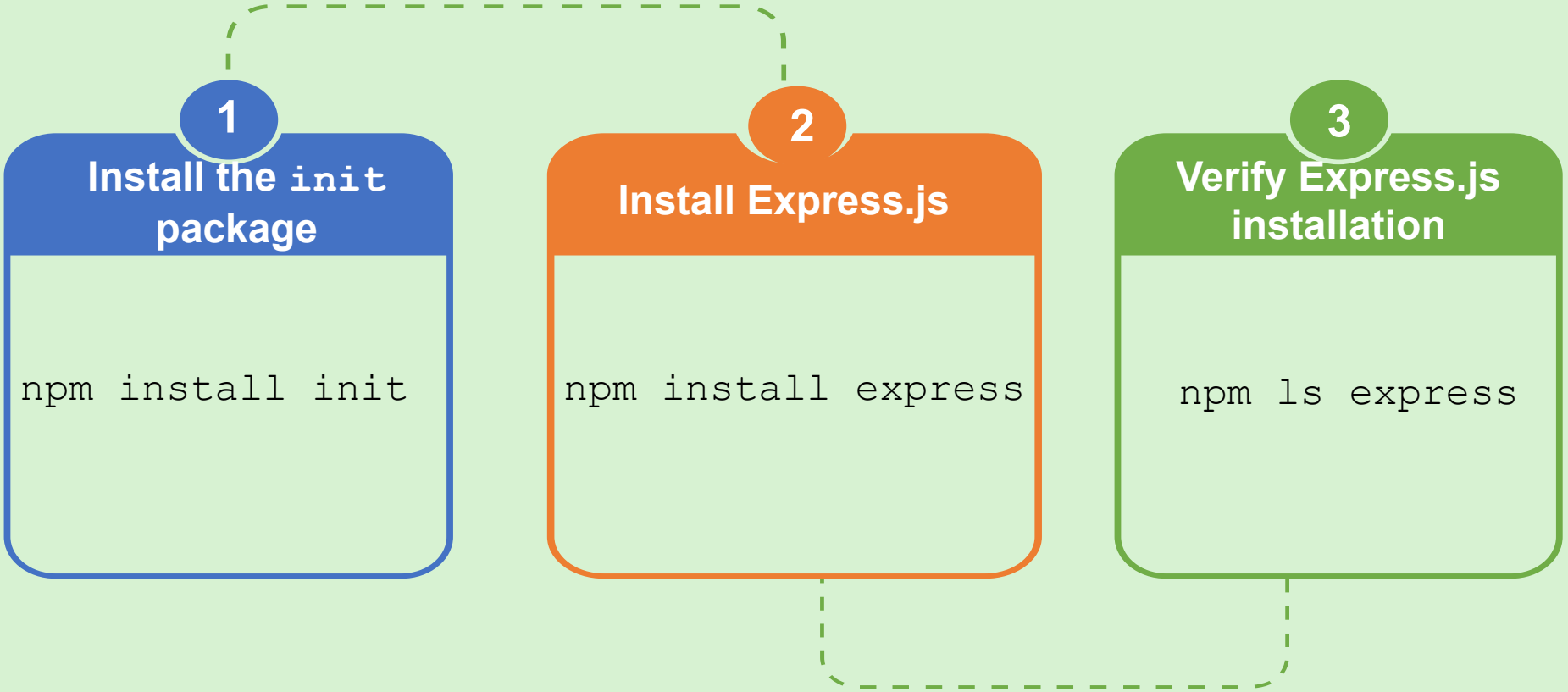
Express.js facilitates:

- Enhanced performance
- Router and middleware support
- Serving of static files

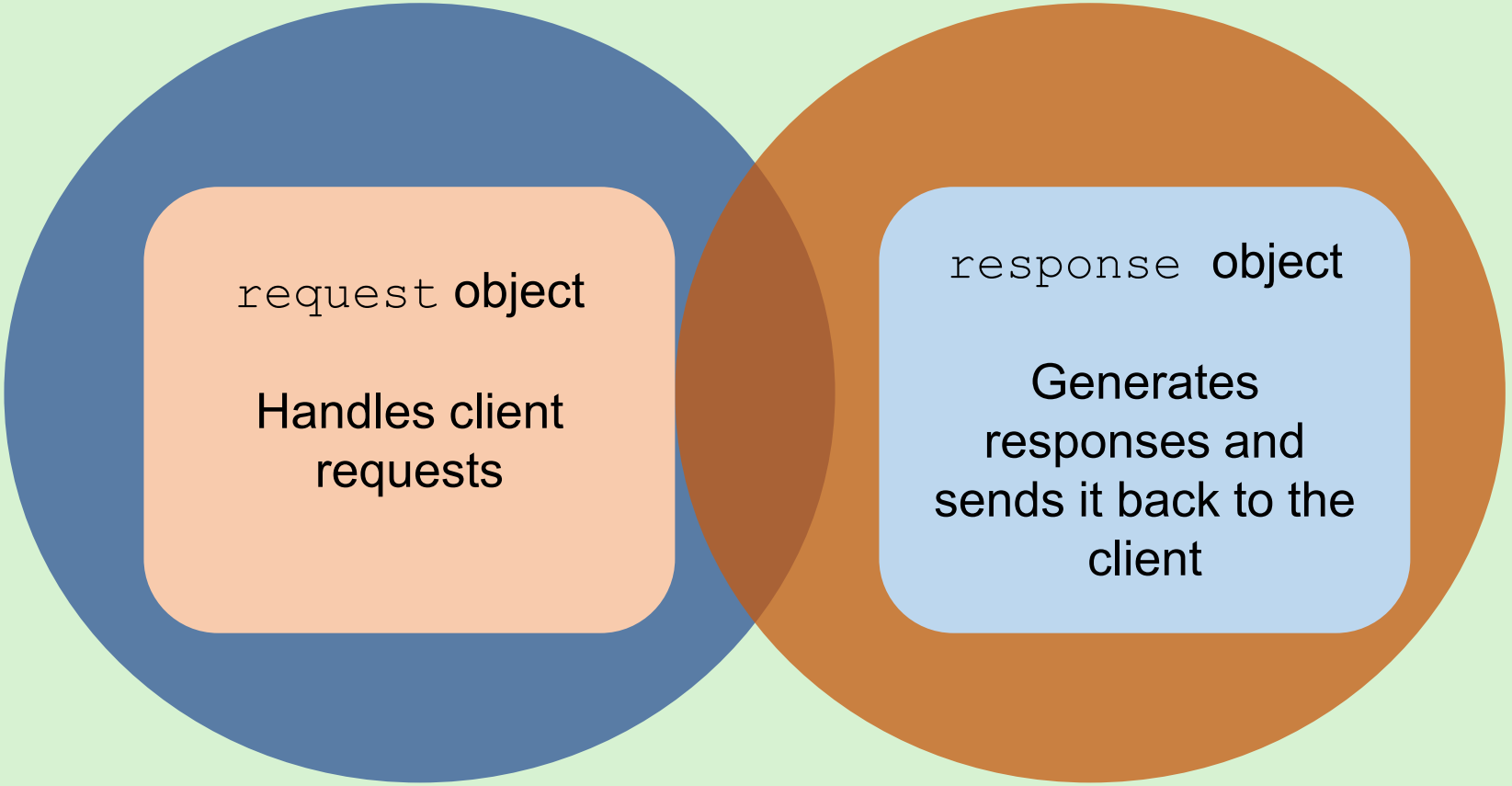
Requirements to Install Express.js



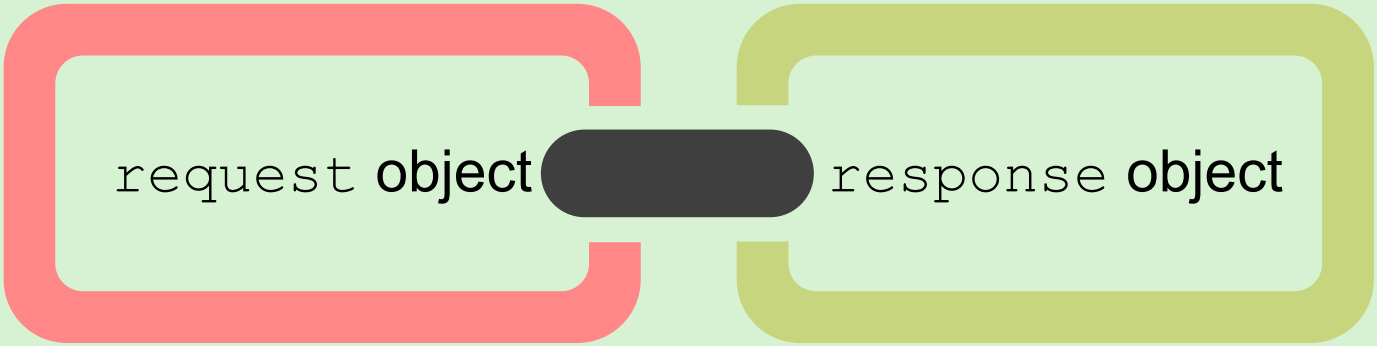
Installing Express.js



`request` and `response` Objects



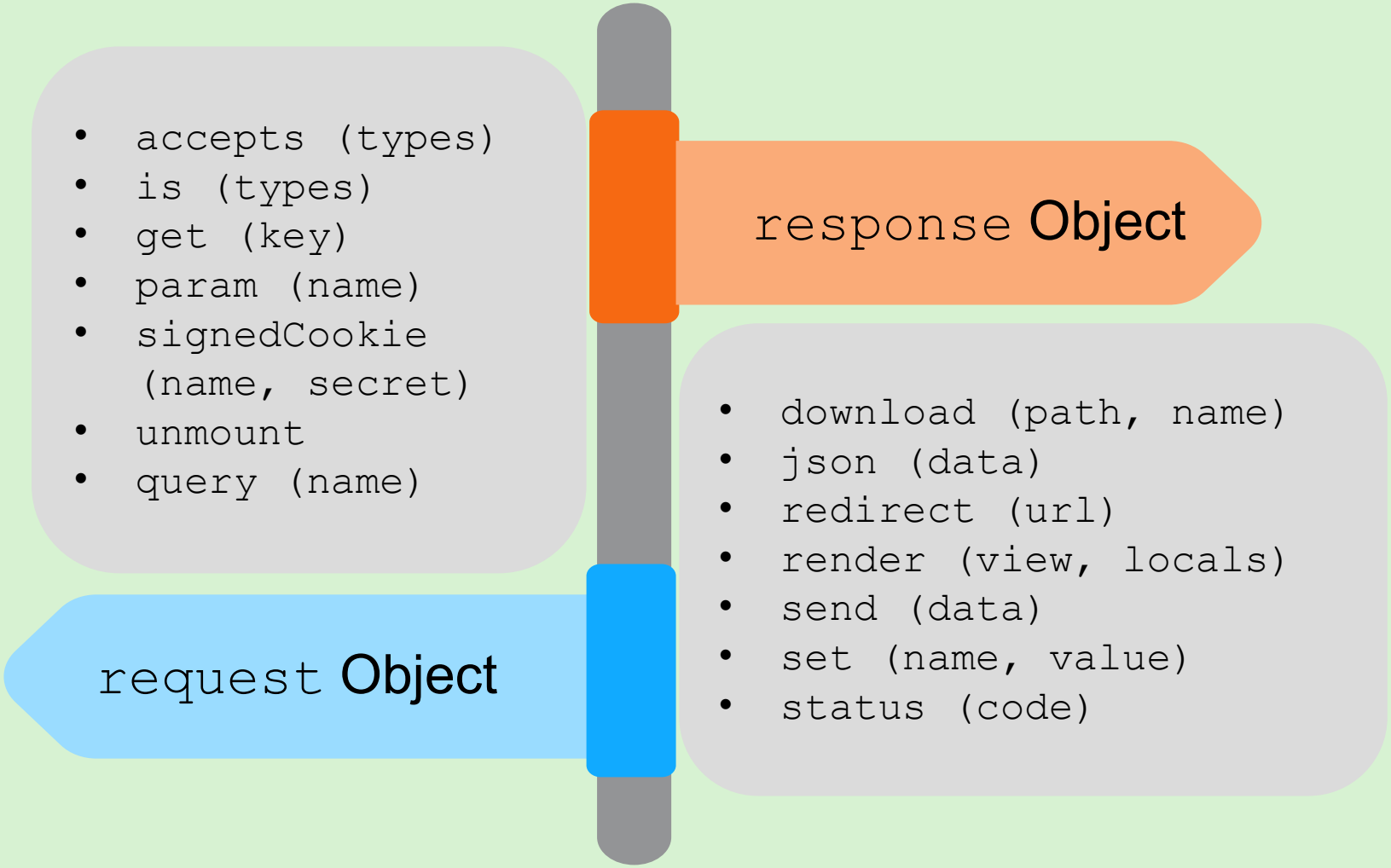
Properties of the `request` and `response` Objects



- `body`
- `headers`
- `ip`
- `method`
- `params`
- `query`
- `url`

- `app`
- `charset`
- `headers`
- `statusCode`
- `type`

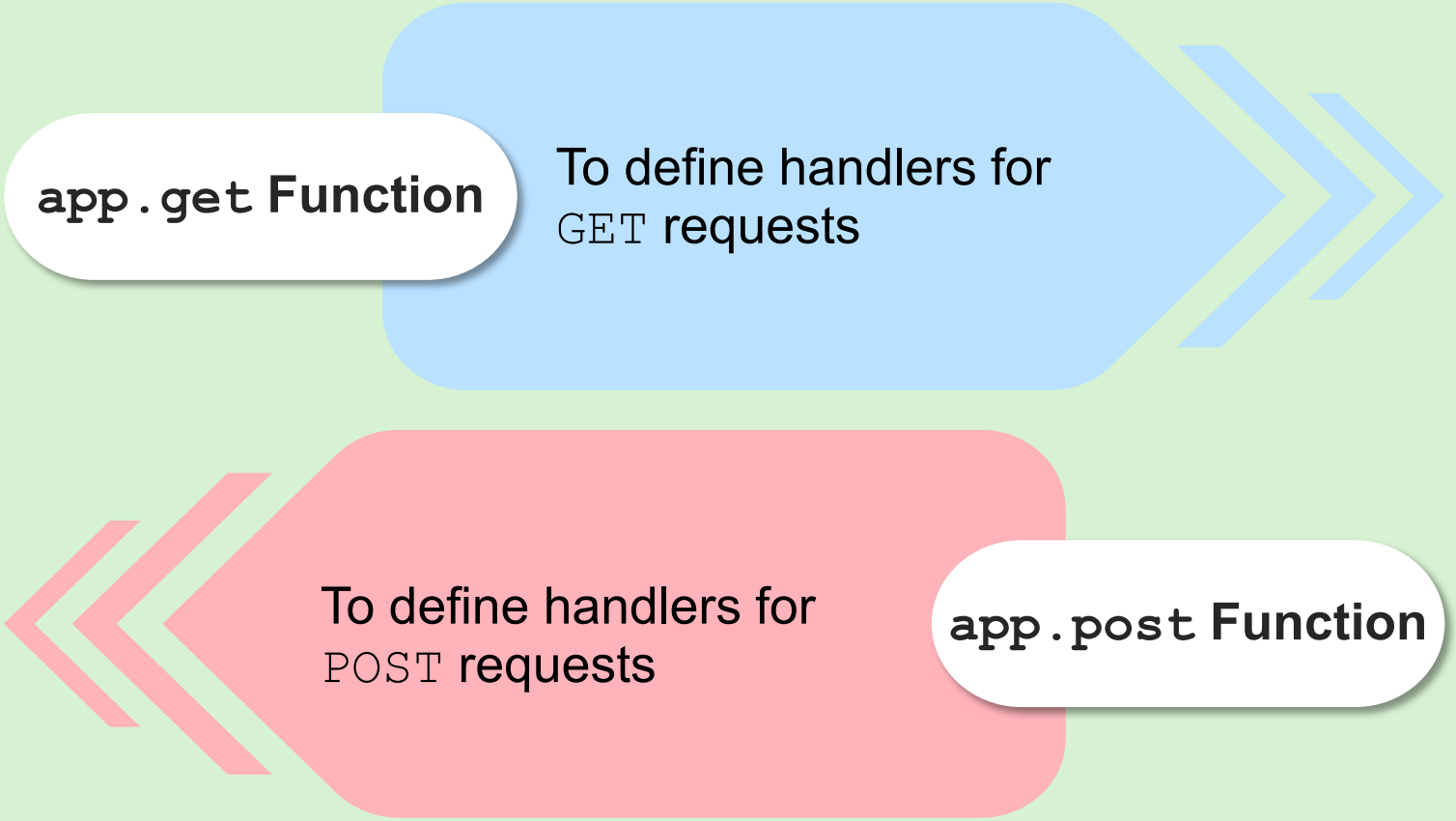
Methods of the `request` and `response` Objects



GET vs. POST Methods

GET Method	POST Method
Can send small amount of data to the server	Can send large amount of data to the server
Appends the data to the URL when sending the request	Encloses the data in the request body when sending the request
Stores the requests in the browser history and log and can be bookmarked	Does not store the requests in the browser history and log and cannot be bookmarked
Can include only ASCII characters in the requests	Can include all types of data in the requests
Can cache the requests	Cannot cache the requests
Offers low security to the request data	Offers high security to the request data
Can be used to obtain data from a particular resource	Can be used to submit data from a particular resource

Handling Requests



Handling GET and POST Requests in a Web Application

01 Open the VS Code.

02 Create an Express Web server.

03 Send the StudentResult.html page.

04 Send the student's result as a response.

05 Save the Notepad file.

06 Execute the application.

Understanding the middleware Feature

```
C:\MyApps>node middlewareSample.js
My Express App is running at http://localhost:3000
```

Server Started

Message Logged for First Request

```
C:\MyApps>node middlewareSample.js
My Express App is running at http://localhost:3000
This is a simple middleware that logs a message for request:1
```

```
C:\MyApps>node middlewareSample.js
My Express App is running at http://localhost:3000
This is a simple middleware that logs a message for request:1
This is a simple middleware that logs a message for request:2
```

Message Logged for Second Request

Message Logged for Subsequent Requests

```
C:\MyApps>node middlewareSample.js
My Express App is running at http://localhost:3000
This is a simple middleware that logs a message for request:1
This is a simple middleware that logs a message for request:2
This is a simple middleware that logs a message for request:3
This is a simple middleware that logs a message for request:4
This is a simple middleware that logs a message for request:5
```

Summary [1-2]

- Express.js framework is used for server-side programming with Node.js runtime environment.
- Express.js can handle different types of HTTP requests, generate dynamic HTML, provide middleware functions, and serve static files.
- The Express.js framework is made up of three interconnected components: view, controller, and model, spread across three layers: application, service, and data access, respectively.
- The view component is responsible for generating the HTML content that is sent to the client.
- The model component handles the data and the business logic and interacts with the underlying data sources.
- The controller component provides the logic that acts as a bridge between the view and model components.

Summary [2-2]

- The order in which components of Express.js are involved is server instance → router → middleware → handler functions → templating/static file serving.
- Express.js is a minimalist, extensible framework that is easy to use and adopt.
- It provides enhanced performance and supports several databases and templating engines.
- The `request` object handles the HTTP requests coming from the clients and provides information, such as the URL, HTTP method, HTTP header, request body, and client IP.
- The `response` object is responsible for generating the responses for the requests and sending them back to the client.
- Express.js provides the `app.get` function to define handlers for `GET` requests and the `app.post` function to define handlers for `POST` requests.

Session: 7

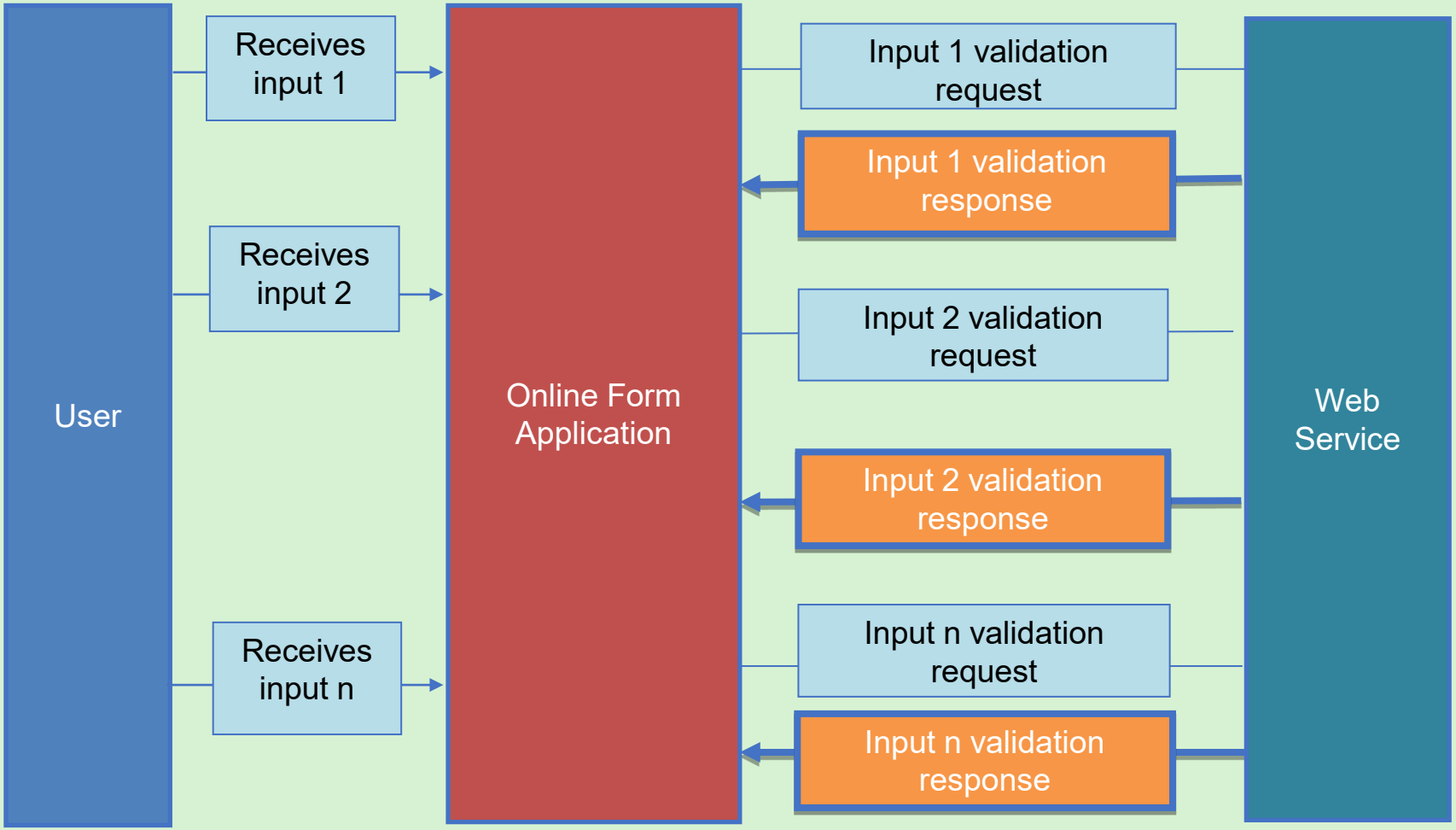
Asynchronous and Synchronous Programming

Server-side Development with Node.js

Objectives

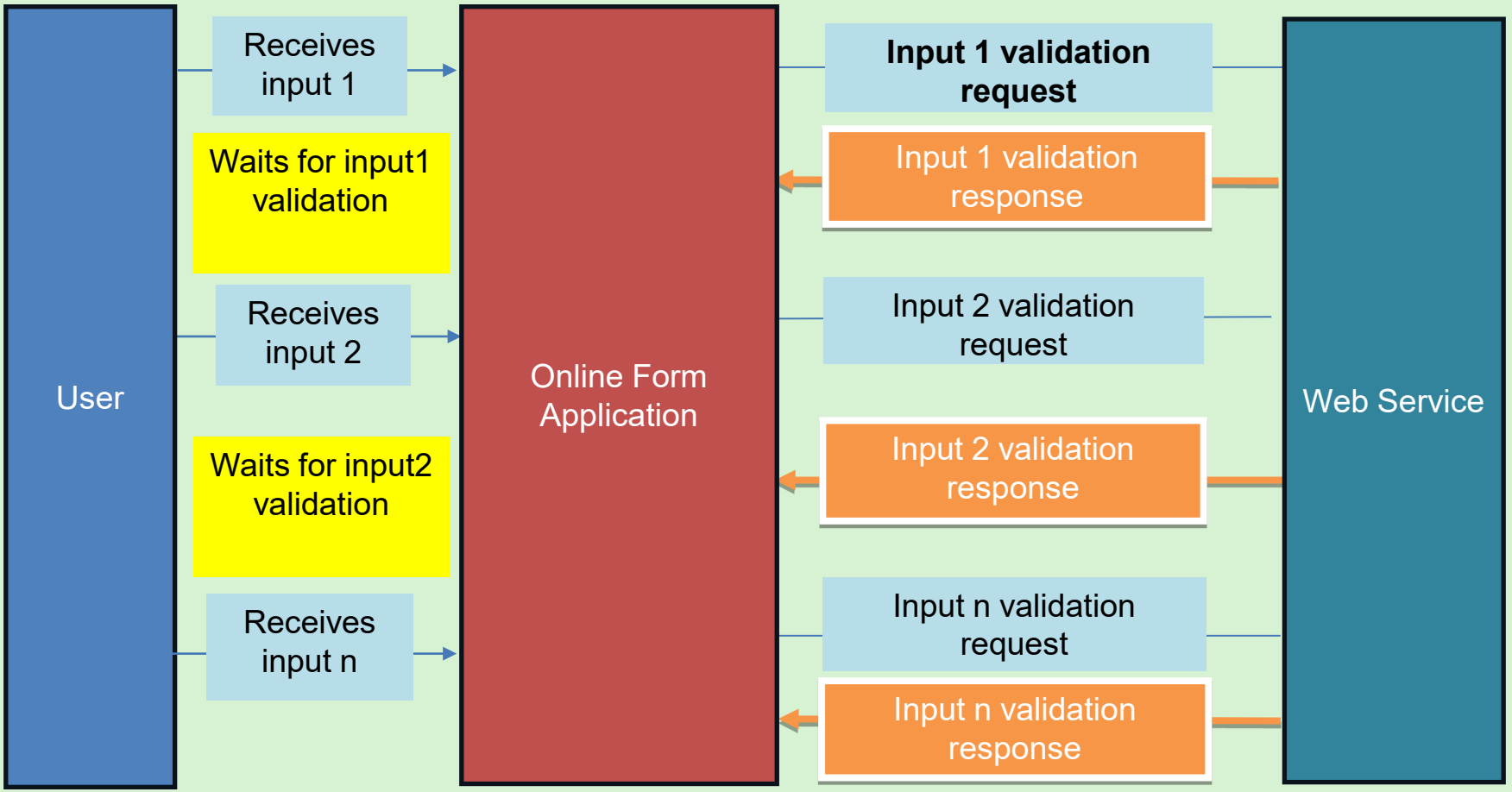
- Compare synchronous and asynchronous programming models
- Explain asynchronous programming model using callback, `Promise`, and `async/await` in Node.js
- Describe synchronous programming model in Node.js
- Explain the event loop architecture in Node.js

Introduction to Asynchronous Programming



Asynchronous Task Execution

Introduction to Synchronous Programming

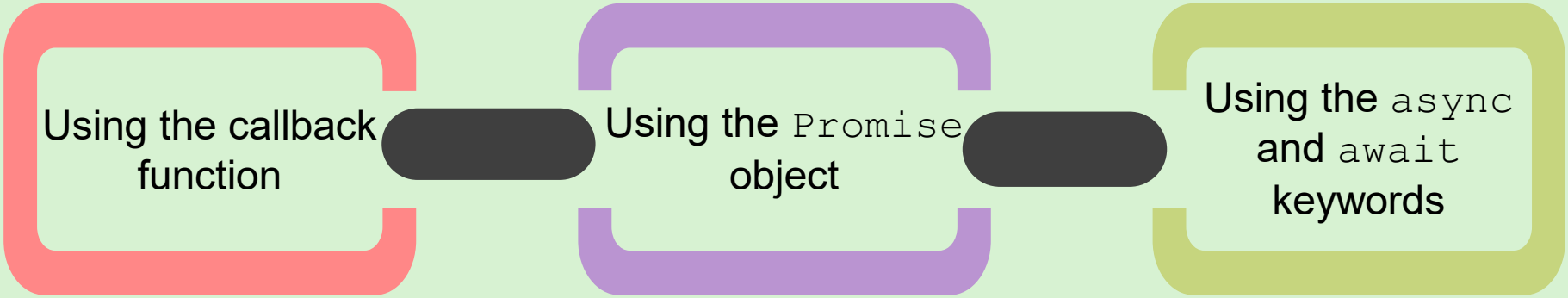


Synchronous Task Execution

Asynchronous vs. Synchronous Programming

Asynchronous Programming	Synchronous Programming
Executes tasks simultaneously	Executes tasks sequentially
Follows non-blocking architecture	Follows blocking architecture
Can make the application responsive throughout	Can make the application unresponsive for time-consuming tasks
Is suitable for tasks related to input/output or network operations	Is suitable for CPU dependent simple tasks
Enhances user experience	Enables hassle free programming

Asynchronous Programming in Node.js



Using the callback Function

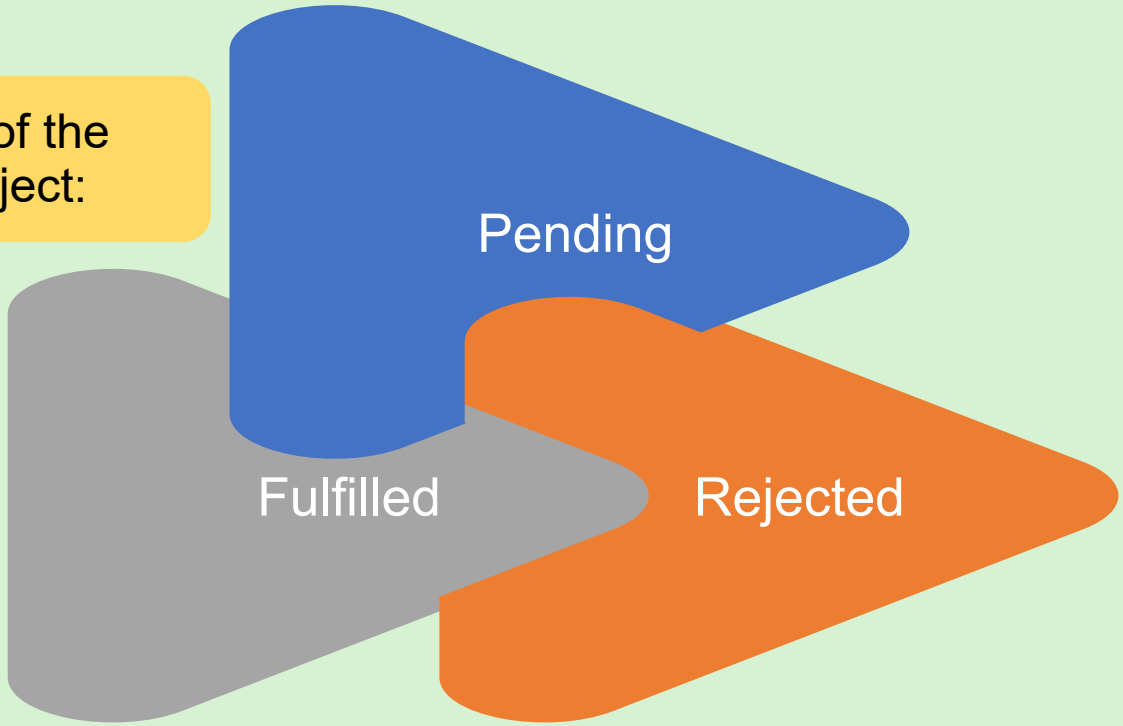
A callback function is:

- Passed as an argument to another function.
- Called when the outer function completes its task.
- Used to handle the results of asynchronous operations without blocking the execution of the rest of the code.

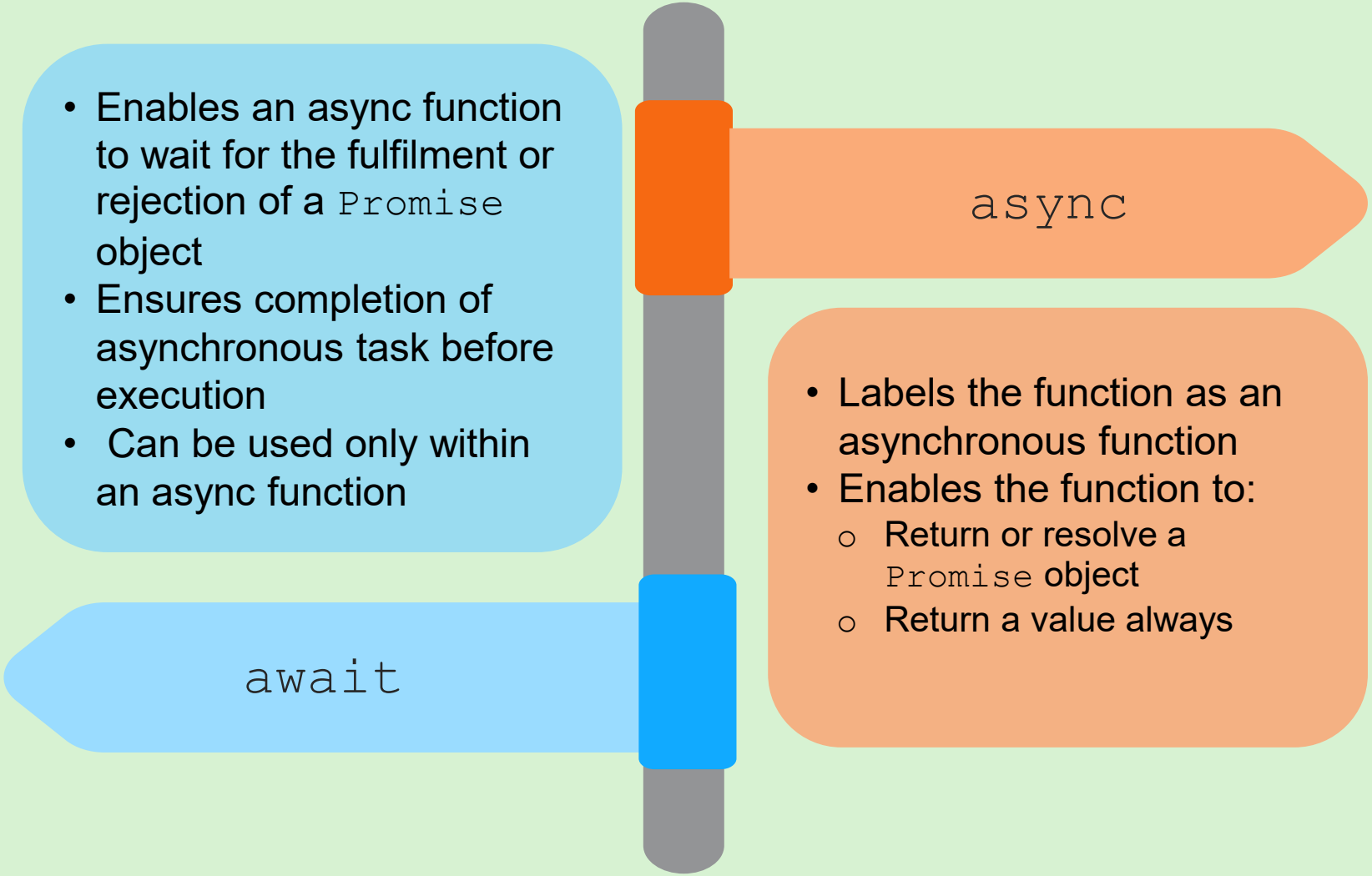
Using the `Promise` Object

The `Promise` object helps in executing multiple tasks in a program in an organized manner.

Three states of the `Promise` object:

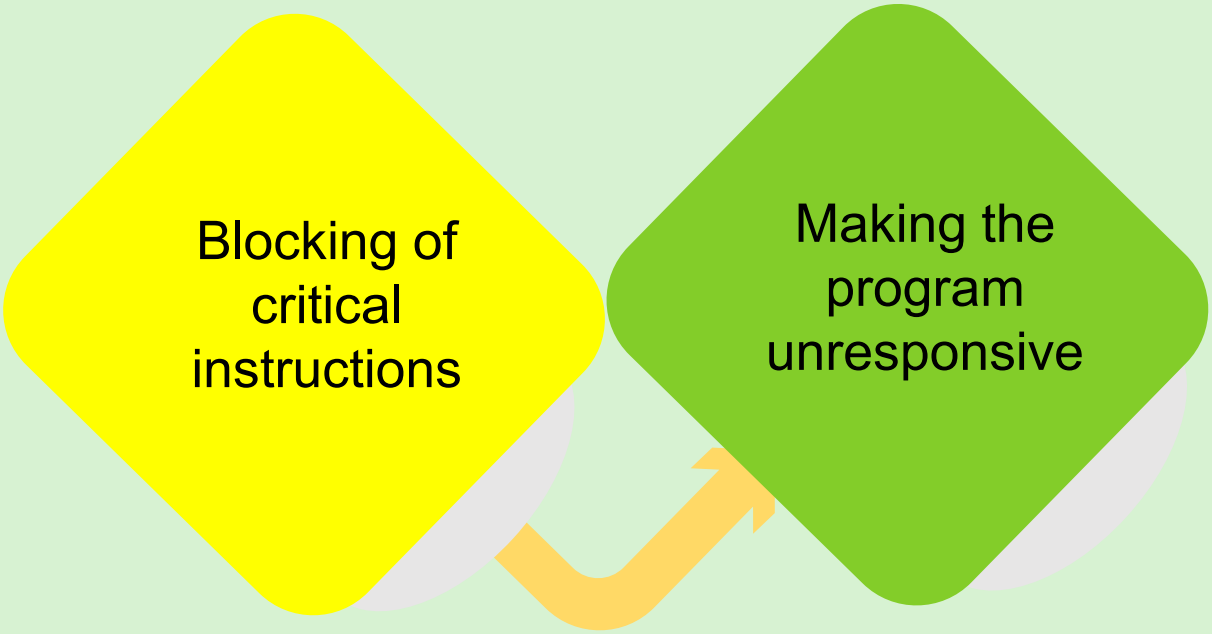


Using the `async` and `await` Keywords



Synchronous Programming

Execution of instructions sequentially



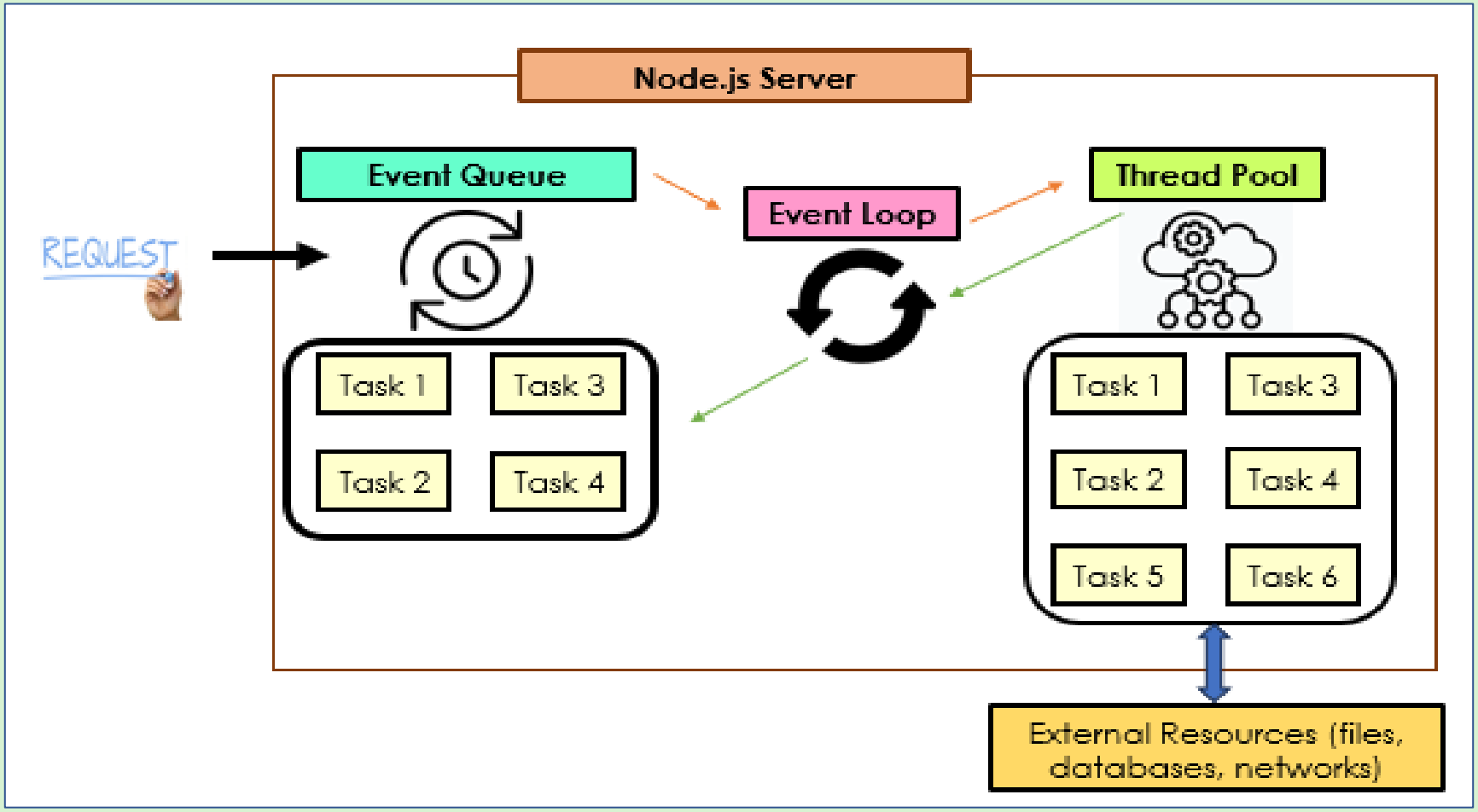
Introduction to Event Loop in Node.js

An event loop:

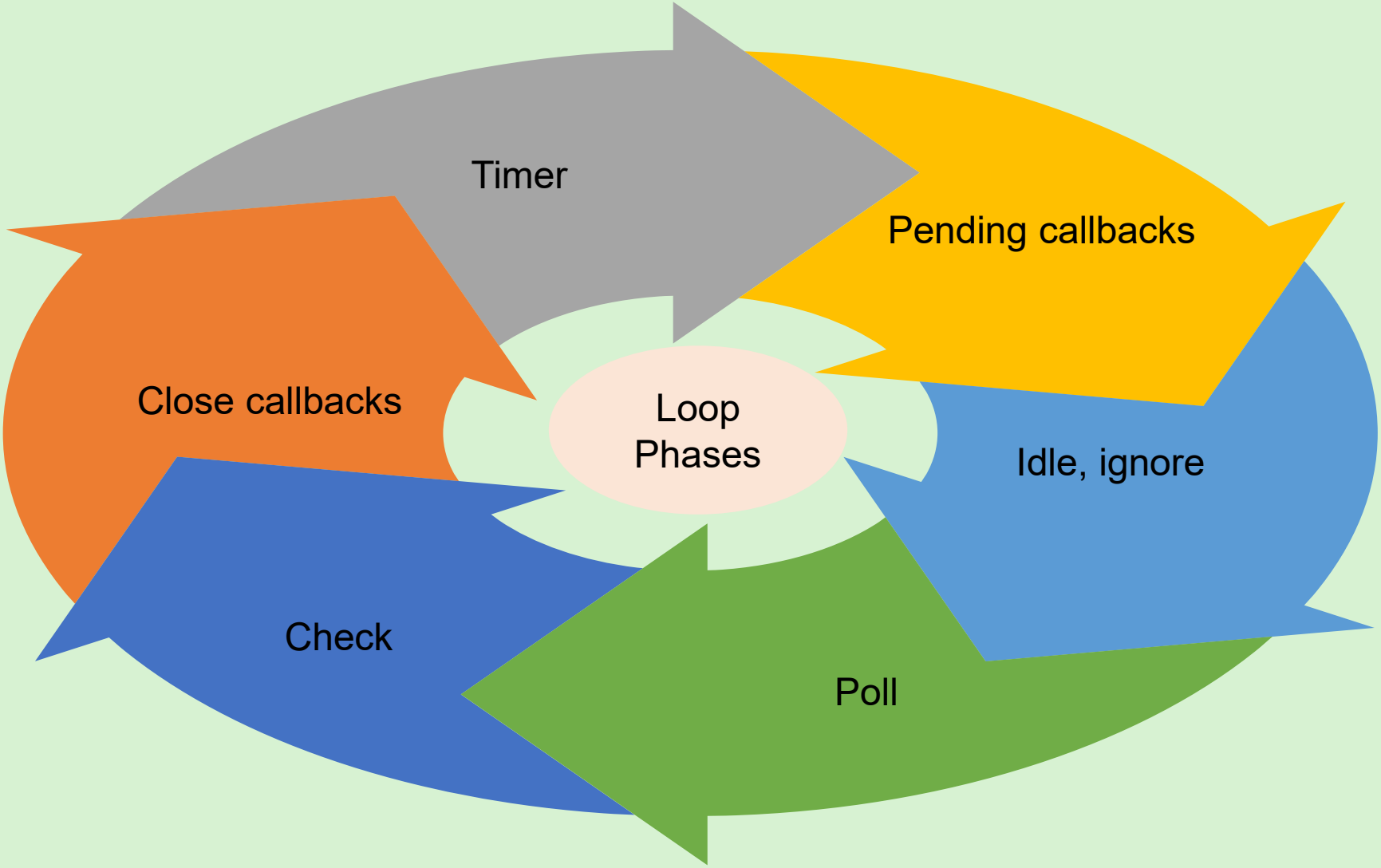
Manages the asynchronous and non-blocking operations

Runs continuously until all the asynchronous tasks in the program

Working of an Event Loop



Phases of the Event Loop



Summary [1-2]

- In an asynchronous programming model, tasks are executed simultaneously.
- The synchronous programming model follows the sequential execution of tasks.
- Node.js relatively follows the asynchronous programming model with the help of a callback function, `Promise` object, and `async`/`await` keywords.
- The event loop architecture in Node.js consists of six phases such as timer, pending callbacks, idle, ignore, poll, check, and close callbacks.
- The callback function executes the asynchronous tasks in the background while other tasks are executed without any blockade.

Summary [2-2]

- The `Promise` object acts as a complementary addition to the callback function.
- The `async` keyword ensures that the async function resolves a Promise object.
- The `await` keyword should be used inside an async function only.
- The event loop architecture in Node.js consists of six phases such as timer, pending callbacks, idle, ignore, poll, check, and close callbacks.
- An event loop maintains the responsiveness of the applications ensuring efficient execution of asynchronous tasks.

Session: 8

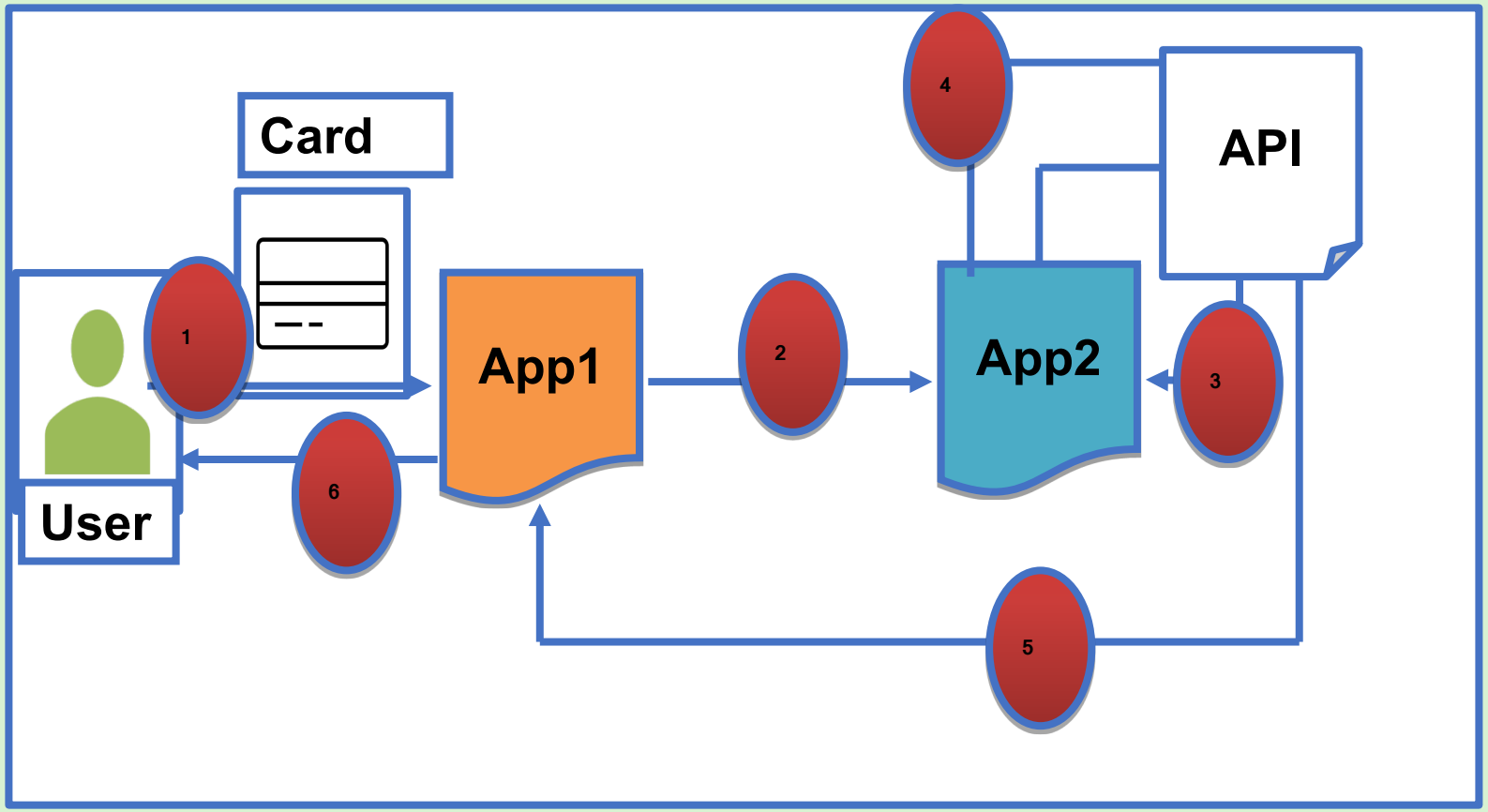
RESTful and HTTP APIs

Server-side Development with Node.js

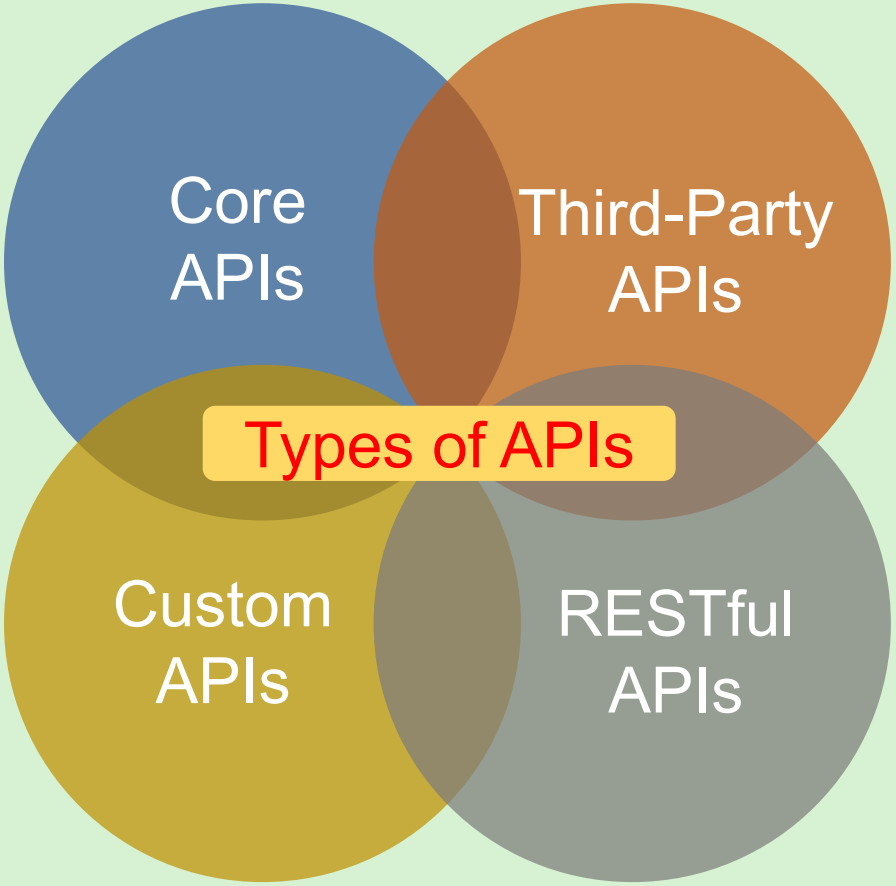
Objectives

- Describe an API and its purpose
- List the common types of APIs in Node.js and their functions
- Explain RESTful and HTTP APIs and their features
- Explain how to use RESTful API and HTTP API to manage data
- Identify the advantages and disadvantages of RESTful and HTTP APIs
- Distinguish between RESTful APIs and HTTP APIs

Overview



Introduction to APIs



HTTP APIs

Client - server Model

Layered architecture

Caching

Features of HTTP APIs

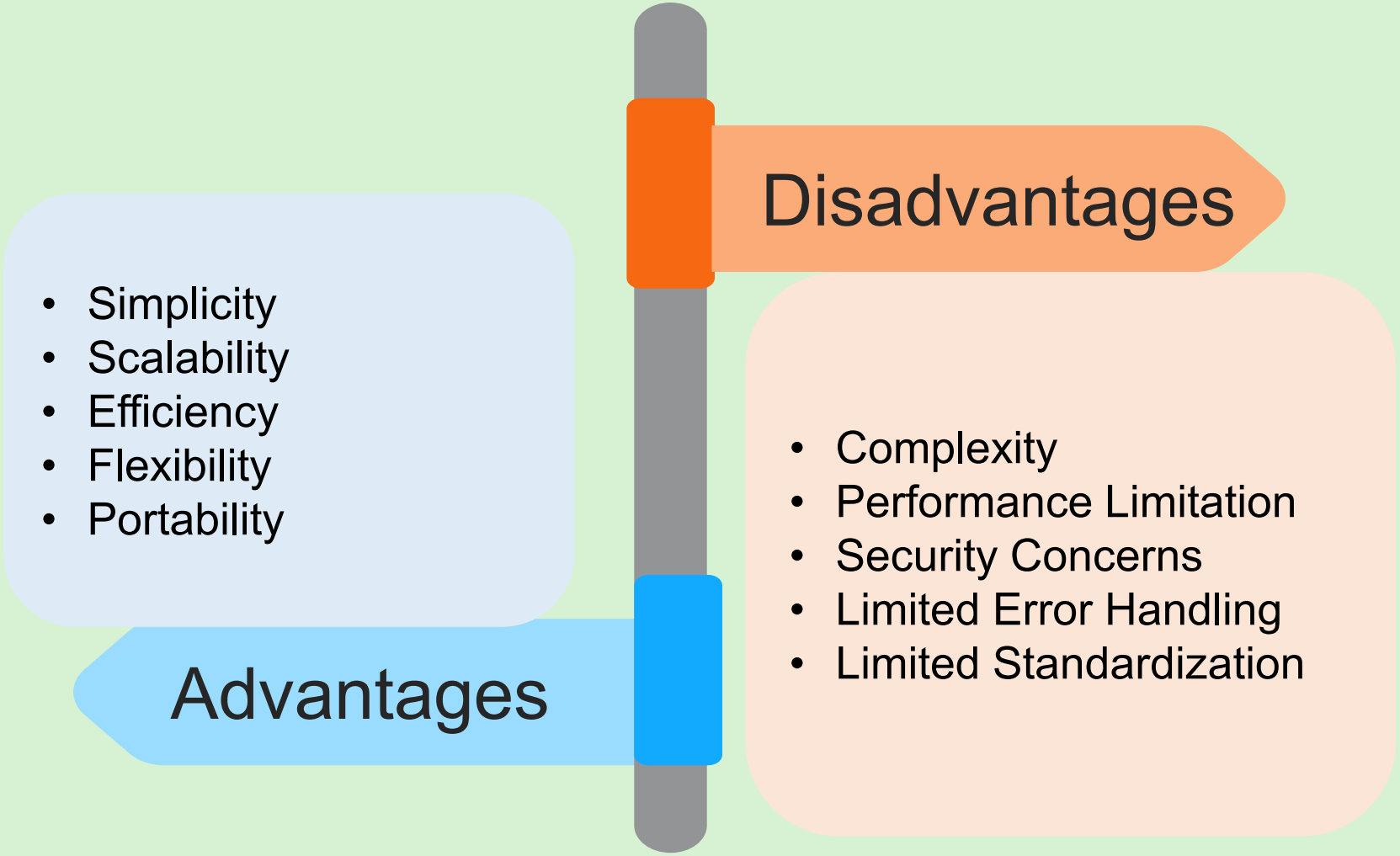
Uniform interface

Standard media types

Resource-centric approach

Stateless interactions

Advantages and Disadvantages of HTTP APIs



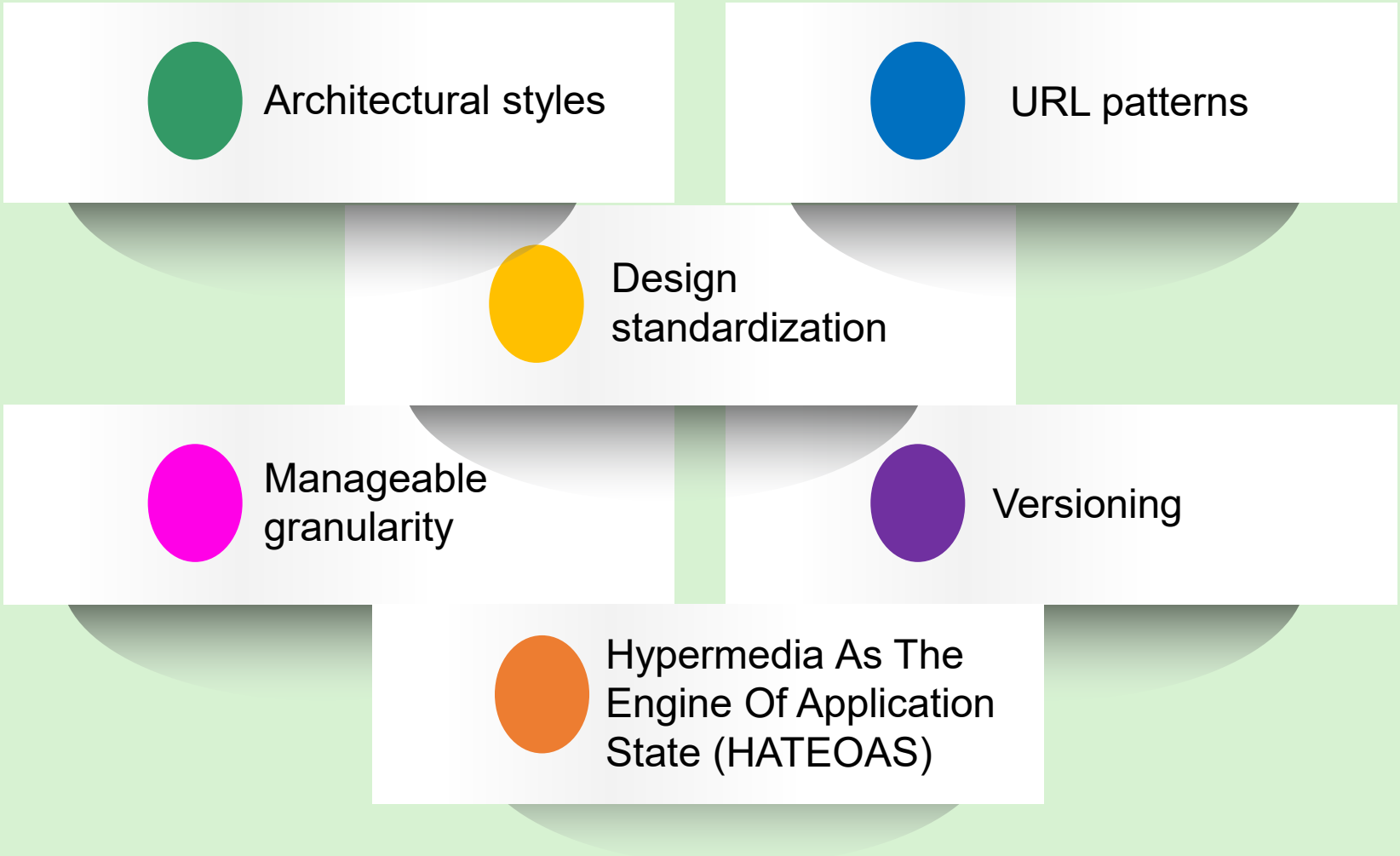
RESTful APIs

A Web API that uses HTTP protocol to exchange data.

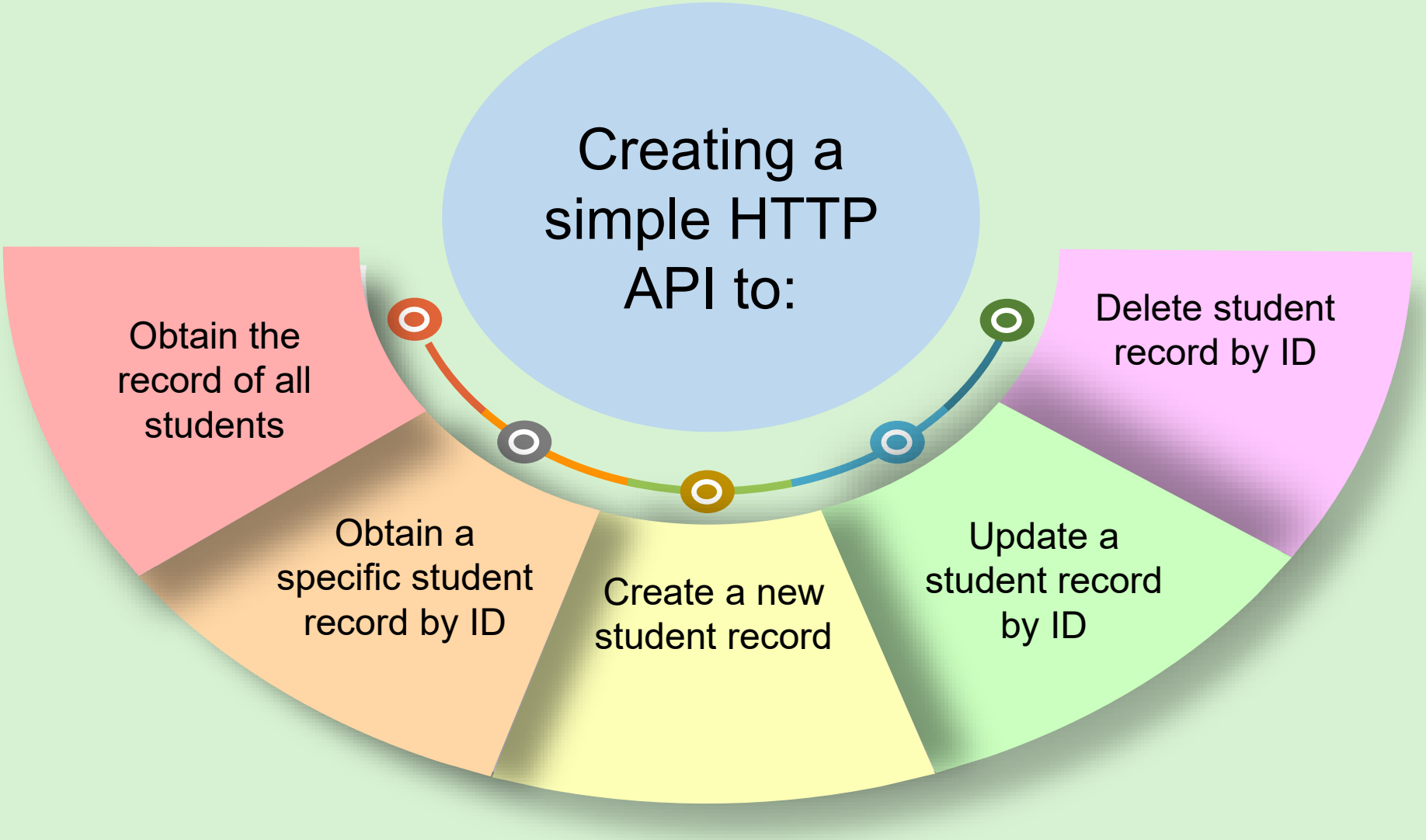
These APIs are a subset of HTTP API that follows the principles and drawbacks of REST architecture.

These APIs have the same features, advantages, and disadvantages of HTTP APIs.

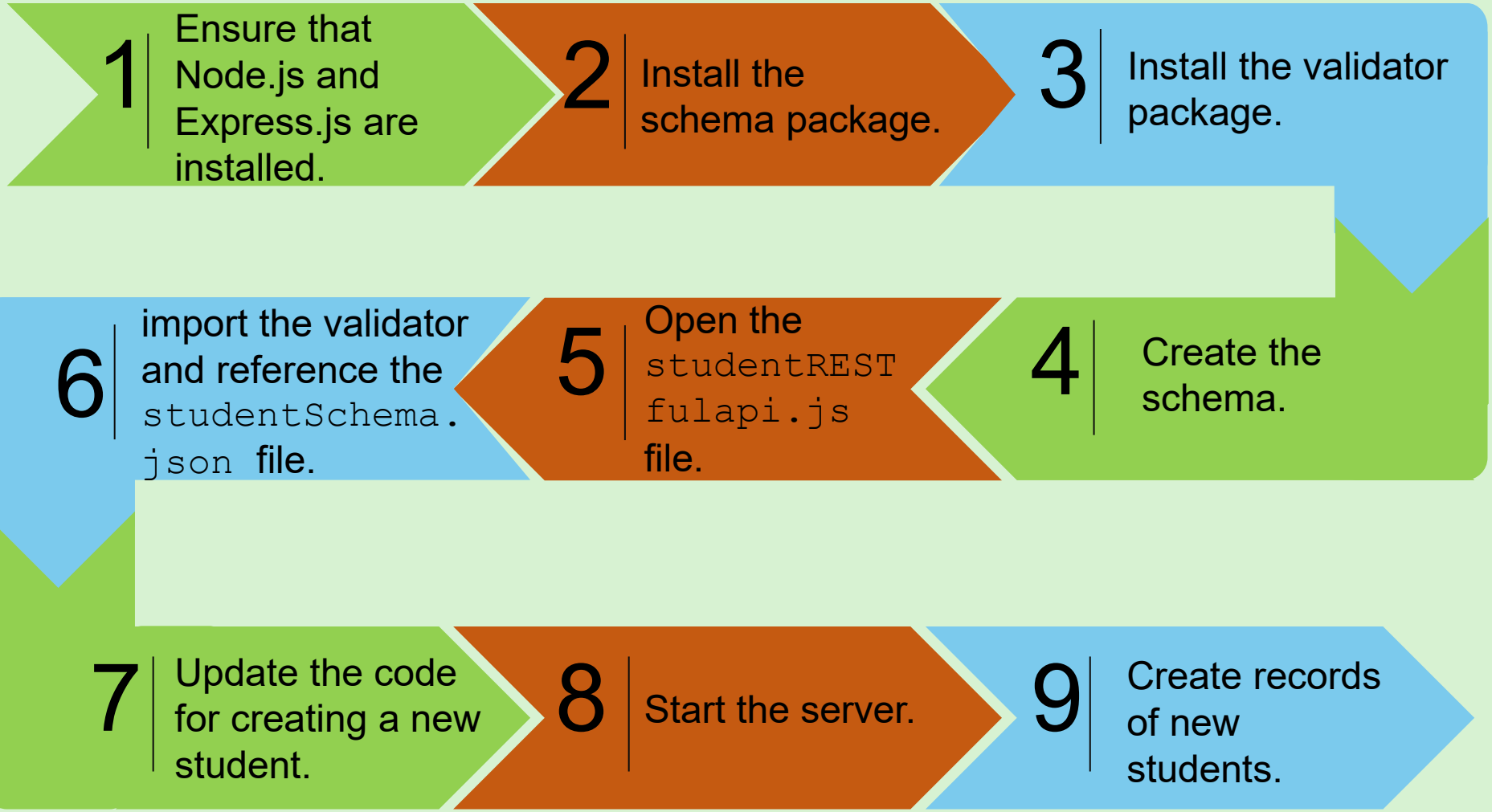
RESTful APIs vs. HTTP APIs



Using HTTP APIs with Express.js



Using RESTful APIs with Express.js



Summary

- APIs translate a user's instructions into a format, which can be interpreted by the system to process the request and send back the response.
- An API facilitates communication and interaction between software applications through a set of rules and protocols.
- Depending upon the scope and usage, Node.js supports several APIs, such as core, third-party, custom, and RESTful APIs.
- HTTP API uses the HTTP protocol to facilitate communication and exchange of data between the systems.
- A RESTful API is an HTTP API that follows the principles and limitations of the REST architectural style to design networked applications.

Session: 9

Integration of Node.js with MongoDB

Server-side Development with Node.js

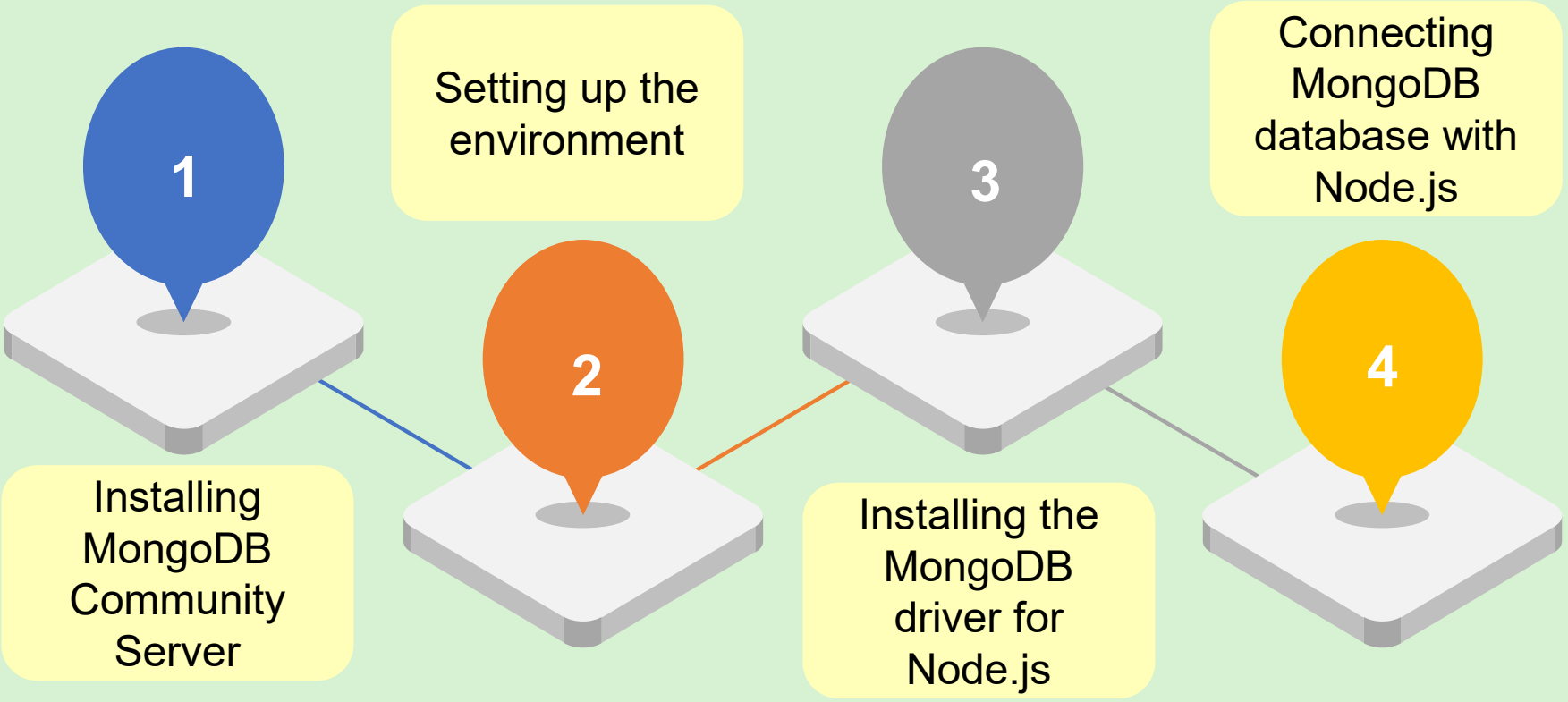
Objectives

- Describe the process to connect Node.js with MongoDB
- Demonstrate the CRUD operations on a MongoDB database using Node.js
- Explain the building of CRUD APIs in Node.js

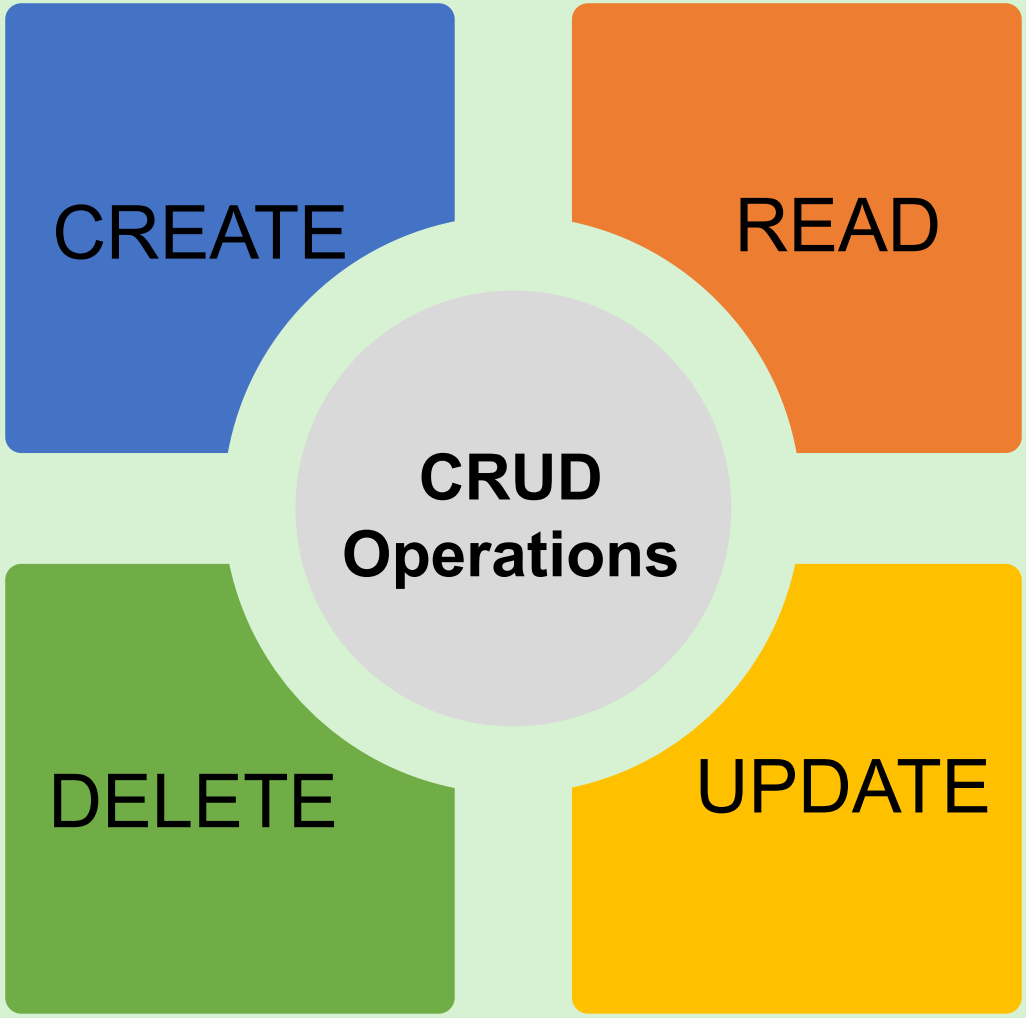
Introduction to Database Connectivity

Relational Databases	Drivers	Non-Relational Daabases	Drivers
MySQL	mysql, mysql2	**MongoDB**	mongodb
PostgreSQL	pg	**Reddit**	ioredis
SQL Lite	sqlite3	**Couchbase**	couchbase
MS SQL Server	mssql	**Cassandra**	cassandra-driver

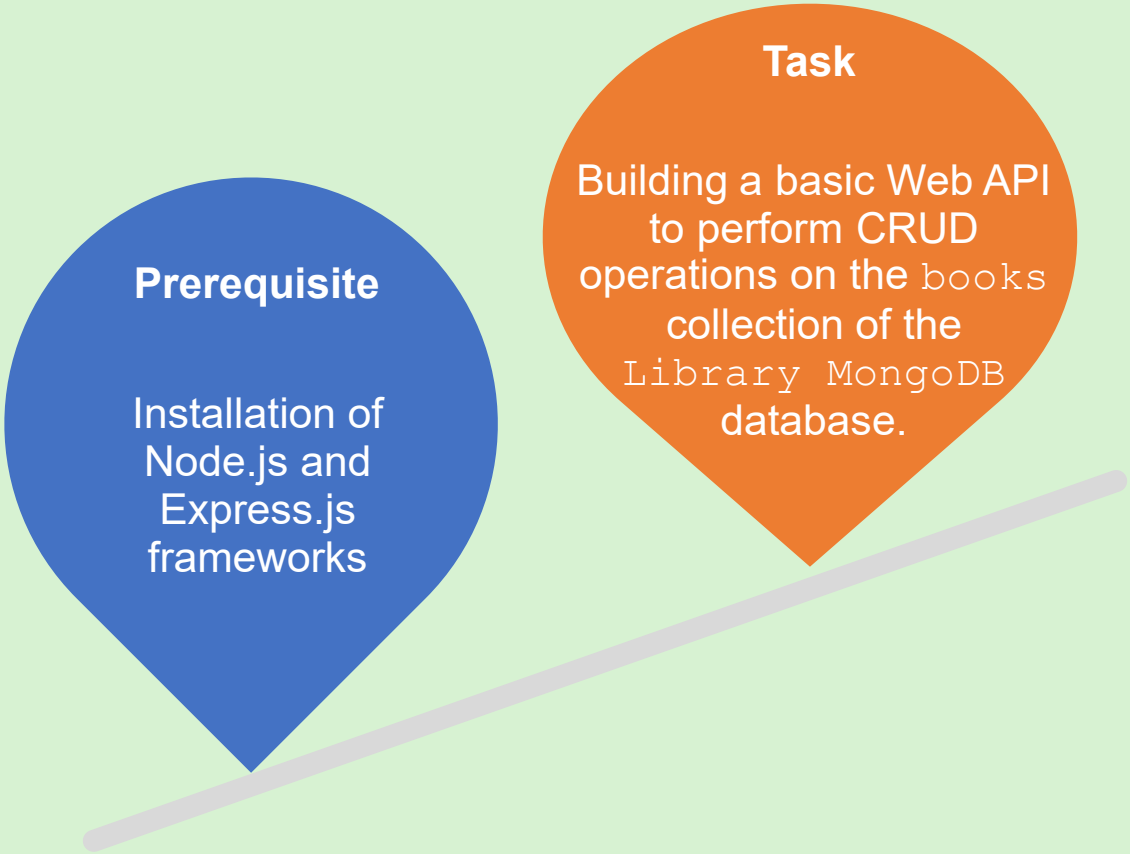
Connecting MongoDB with Node.js



Performing CRUD Operations in Node.js



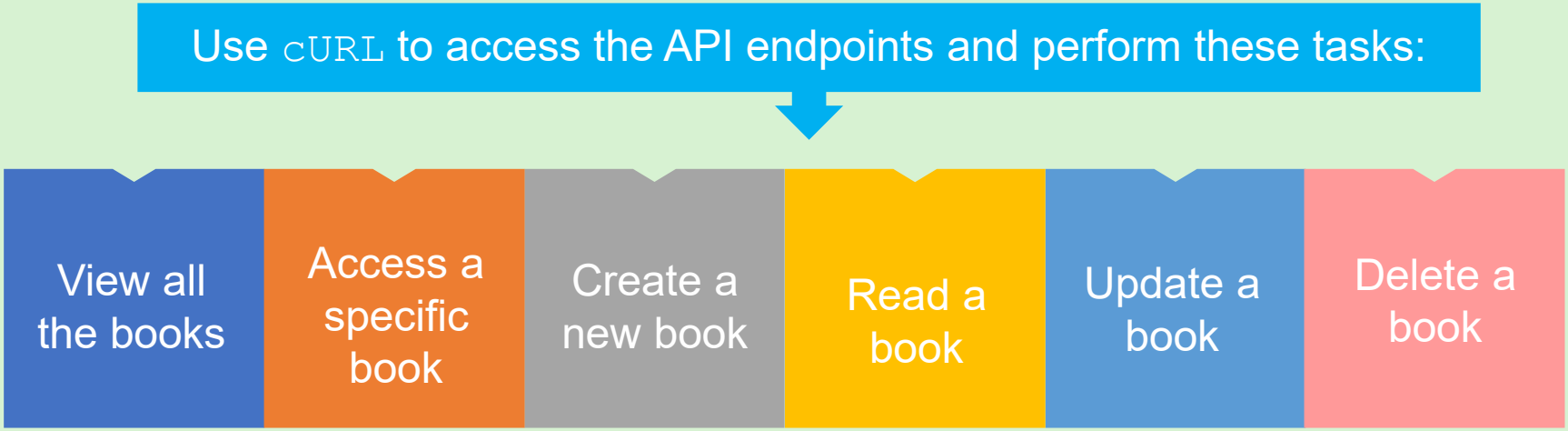
Building CRUD APIs in Node.js



Methods and Properties to Build CRUD APIs

`app.use(express.json())`	`app.listen()`	`app.post()`
`app.get()`	`app.put()`	`app.delete()`
`req.body`	`req.params.id`	`res.status()`

Using cURL for CRUD Operations



Summary

- Using Node.js developers can connect with both relational and non-relational databases.
- MongoDB is a non-relational database.
- Express is a Node.js Web framework that can be used to create Web applications and APIs.
- The MongoDB Node.js Driver is the communication tool that is internally used for communication between Node.js and MongoDB.
- Libraries, such as `mongodb` and `express` can be imported into the code with the `require` keyword.
- Command-line tools, such as `cURL` help in transferring data over network protocols, such as HTTP.

Session: 10

Building Websites Using Node.js

Server-side Development with Node.js

Objectives

- Explain the process of creating a Web application using Node.js
- Describe the steps to connect the Web application to a MongoDB cloud database
- List the steps to upload the Web application to the GitHub repository
- Explain the procedure to deploy the Web application on Render from a GitHub repository

Overview

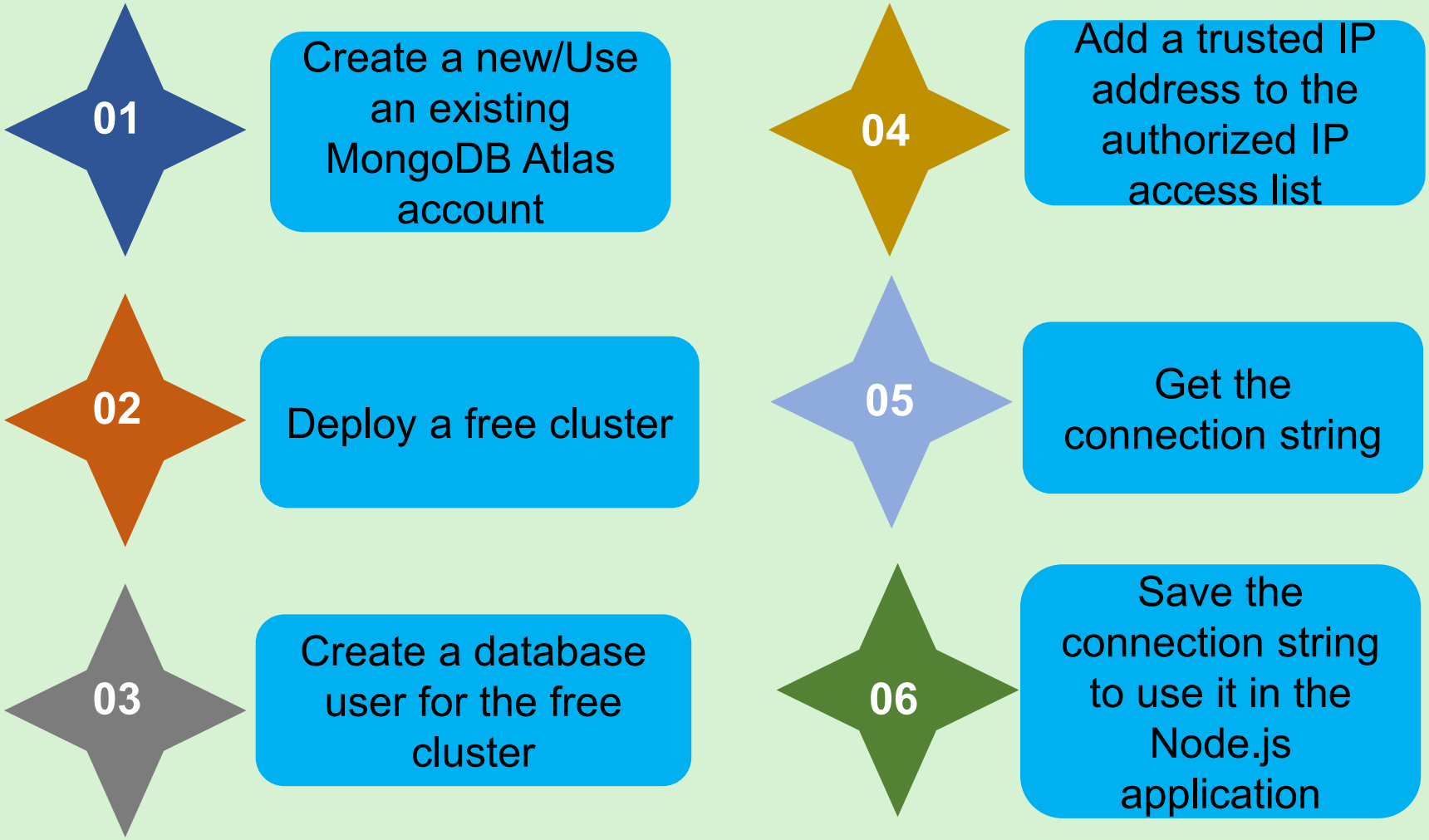
Characteristics of the Library Application

- User Authentication
- New Membership
- Existing Membership
- Administrator Privileges

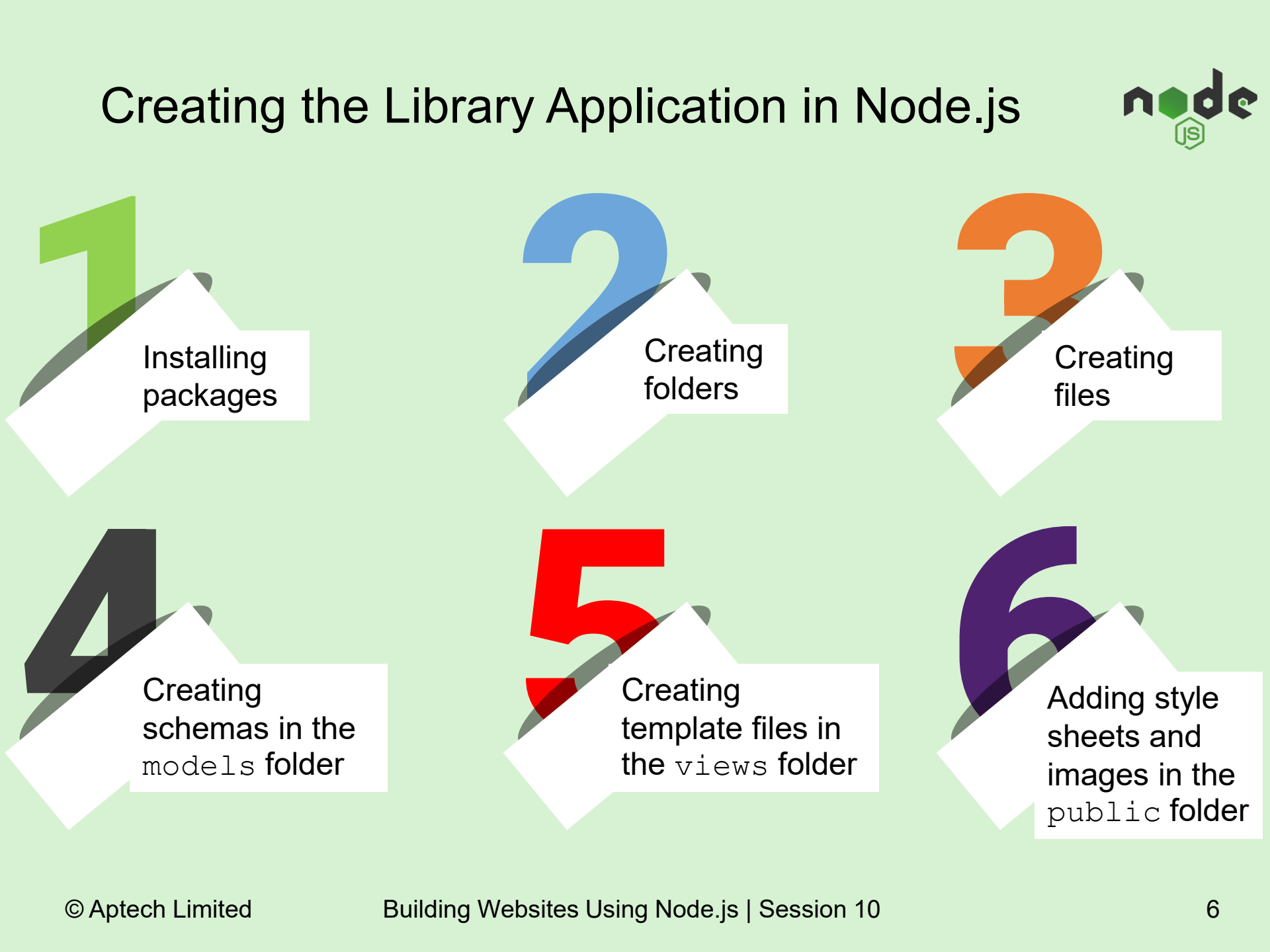
Creating the Library Web Application

- Retrieve the database deployment's connection string.
- Create the library application in Node.js.
- Upload the library application to the GitHub repository.
- Deploy the library application from the GitHub repository on Render.
- Access the library application in the Web browser.

Retrieving the Connection String from MongoDB Cloud



Creating the Library Application in Node.js



Installing Packages in the Project

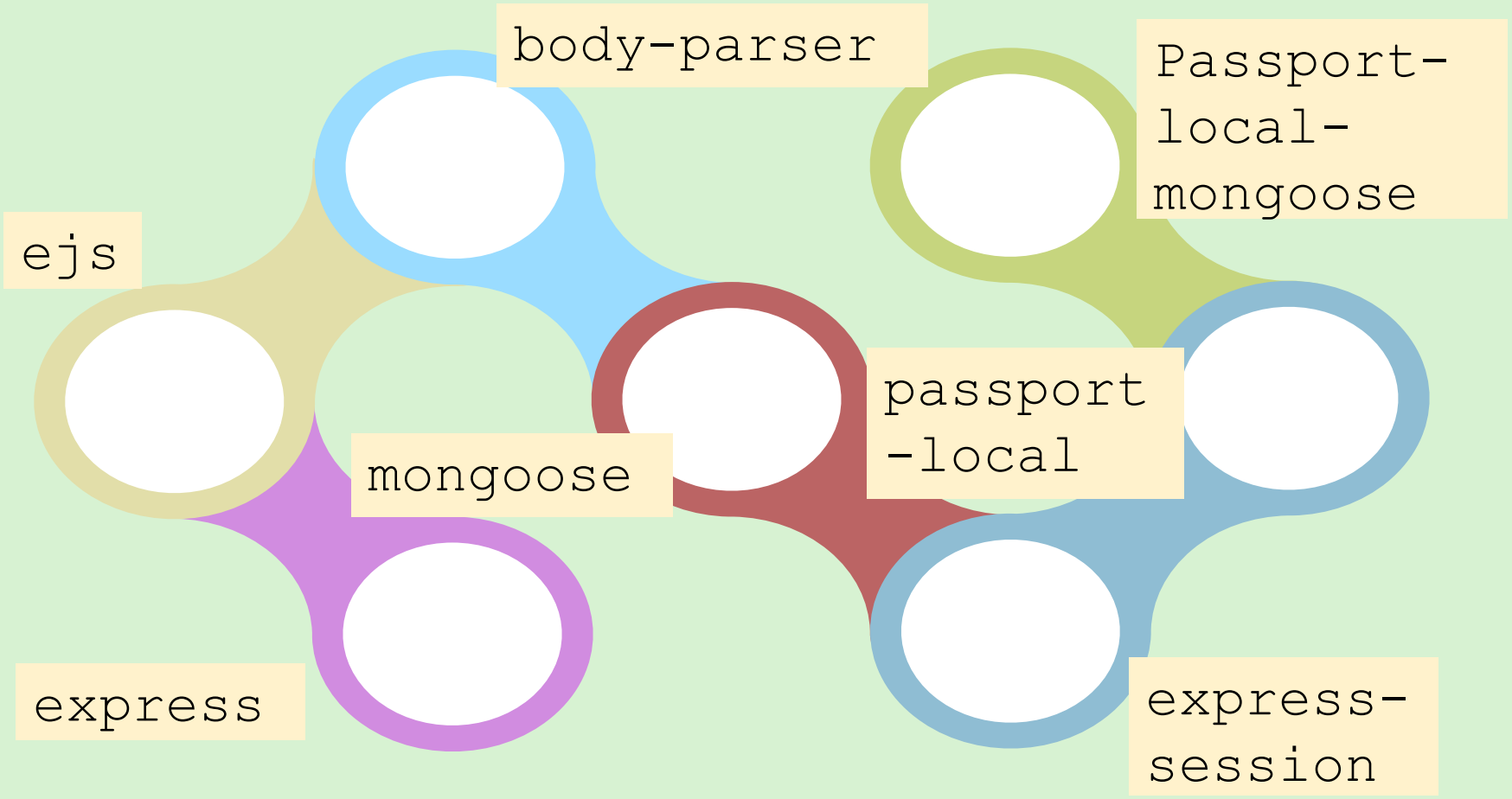
01 Create a directory for the application.

02 Create the `package.json` file to initialize a new Node.js project for the application.

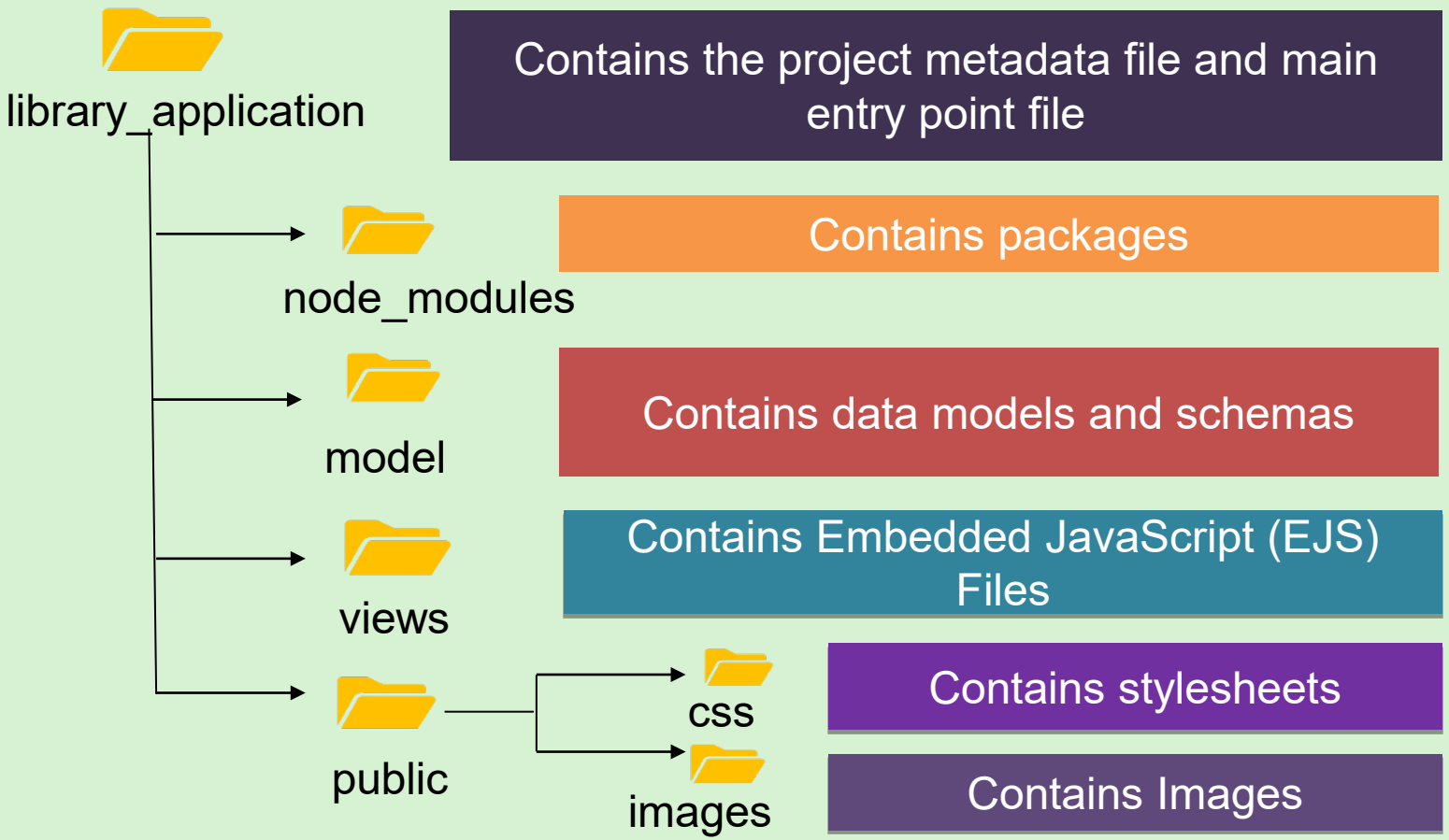
03 Install the required packages in the application's directory.

04 Open the `.json` file in VS code or Notepad.

Packages in the Application Folder



Creating Folders in the Project



Creating Files in the Project

Create schemas in the `model` folder

- user.js
- book.js

Create template files in the `views` folder

- register.ejs
- login.ejs
- home.ejs
- admin-login.ejs
- admin-dashboard.ejs
- booklist.ejs
- error.ejs
- admin-error.ejs

Add stylesheets and images in the `public` folder

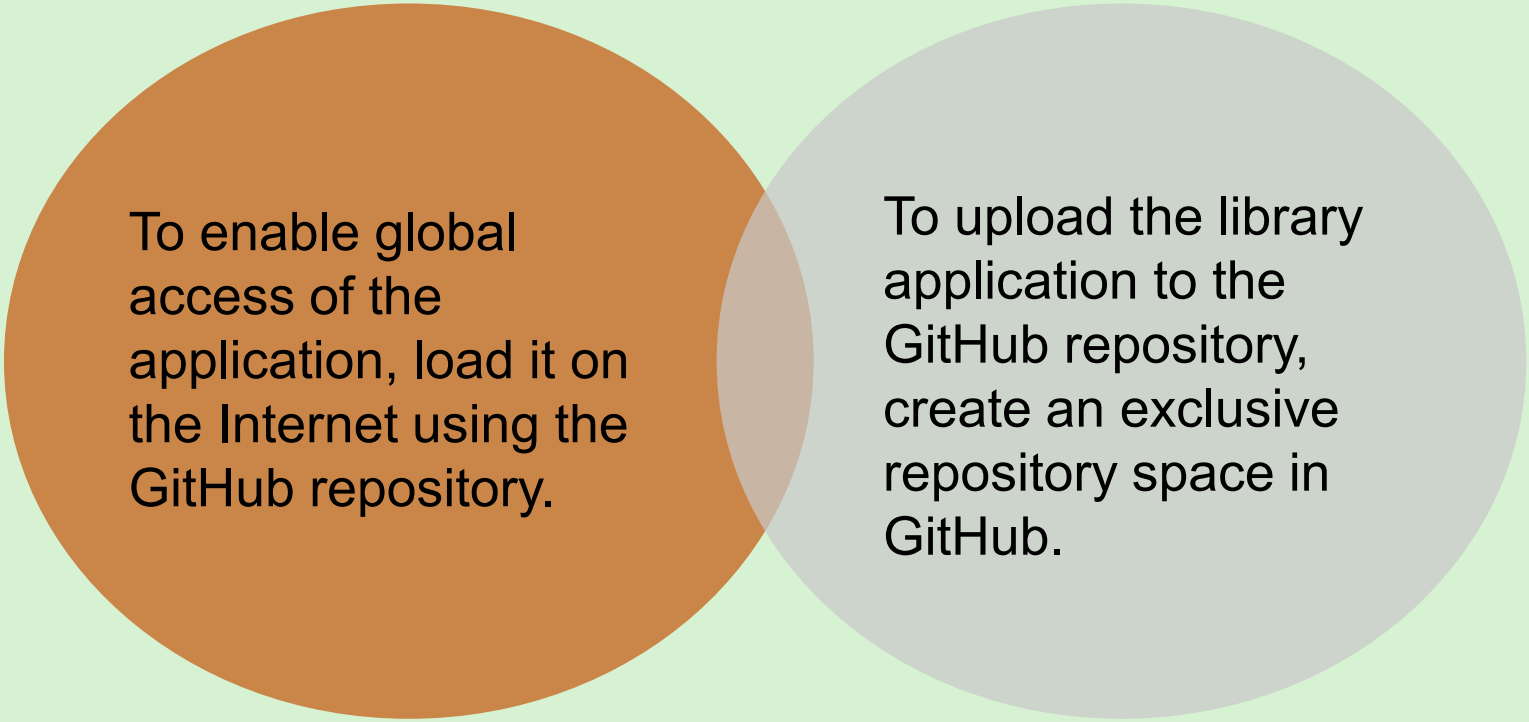
- register.css
- home.css
- error.css
- bookstyle.css
- admindash.css

Operations in the `app.js` File

Importing packages	Importing schemas	Creating express application	Connecting the database
Setting the view engine	Setting up the middleware	Defining the local strategy	Serializing and deserializing user functions
Defining routes	Handling new member registration	Handling existing member login	Handling administrator login and dashboard

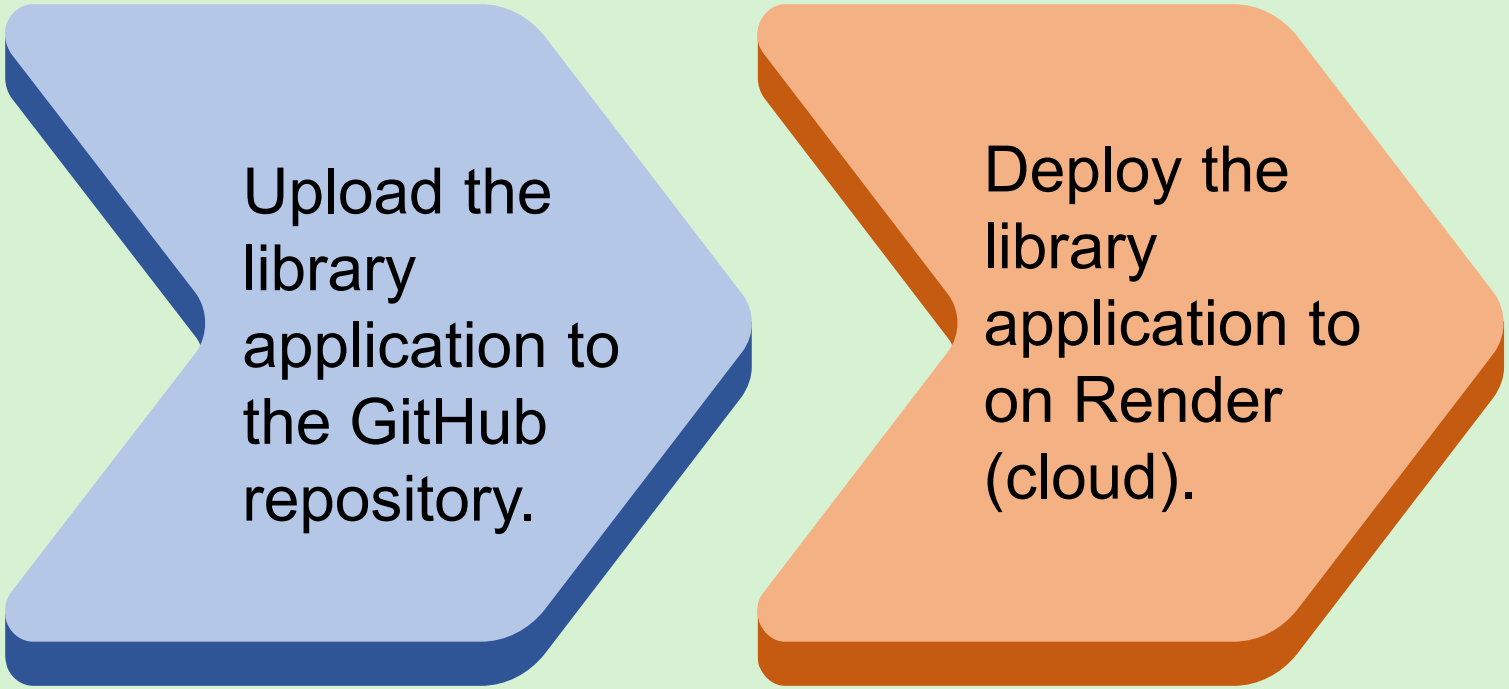
Managing `admin dashboard` and adding books	Managing member/ administrator logout process	Listening for requests

Uploading the Library Application to the GitHub Repository

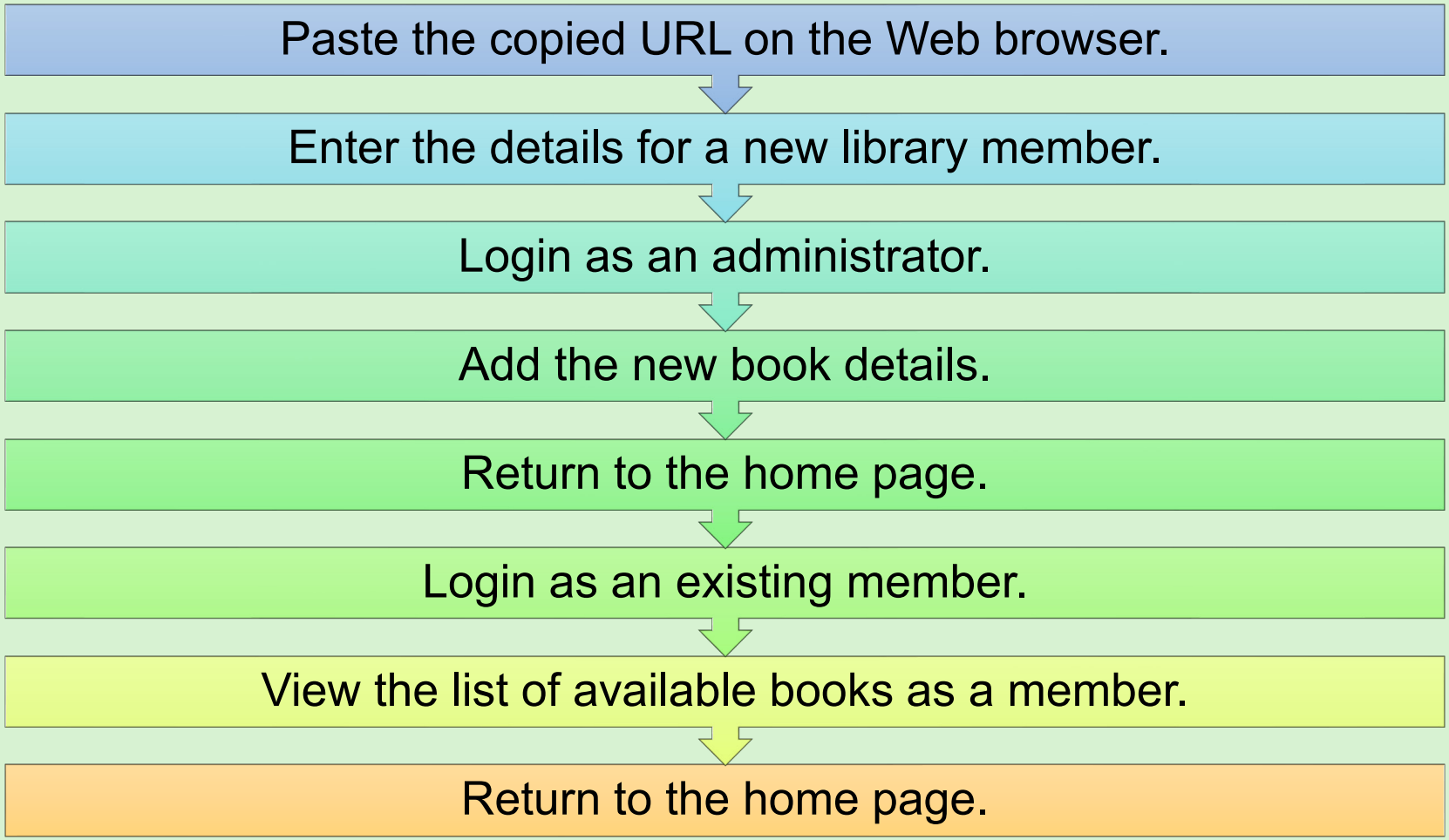


Deploying the Library Application on Render

Developers can use the cloud computing platform, Render to deploy their applications



Accessing the Library Application in Web Browser



Summary

- The connection string created in the MongoDB cloud database connects the Web application to MongoDB cloud.
- The Node.js packages are installed in the Web application using the npm commands.
- Some of the important tasks in Web application development are:
  - ✓ Setting up the view engine and middleware
  - ✓ Defining the local strategy and routes
  - ✓ Serializing and deserializing user functions
  - ✓ Listening to requests
- Schemas and stylesheets should be created in the respective Web application folders.
- The Web application can be uploaded to the GitHub repository for better collaboration and version control.
- Render can be used to deploy Web applications on cloud.

Session: 11

Node.js in Action (Application Development) - I

Server-side Development with Node.js

Objectives

- Explain the process of creating a Web application using Node.js
- Describe the steps to connect the Web application to a MongoDB database

Overview

Requirements to create the Wonderland Widescreen Application

Create the application using Node.js, HTML, and a MongoDB database: `MovieCollectionCluster`.

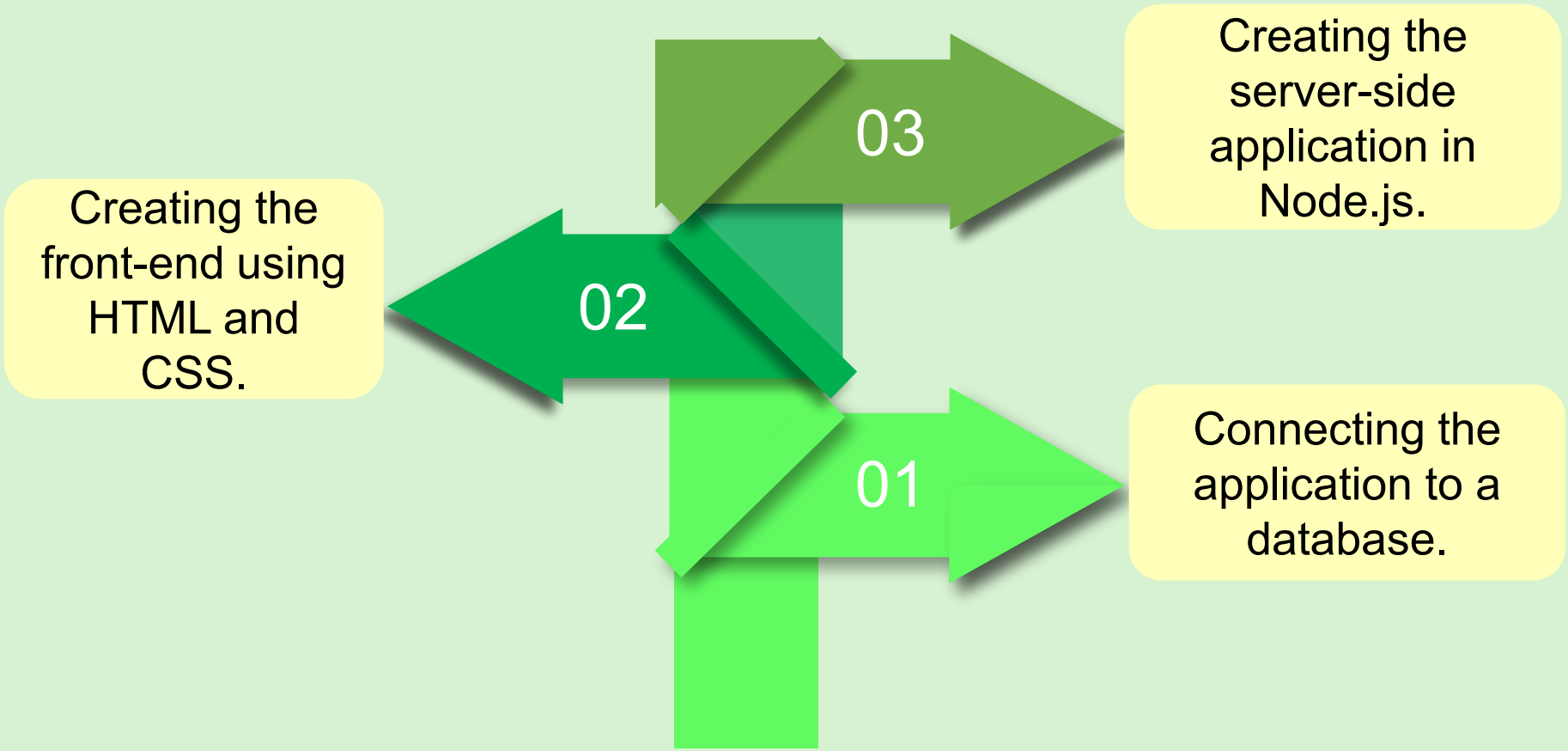
Create a collection named `MovieCollection` in a database `MovieCollectionCluster`.

Characteristics of the Wonderland Widescreen Application

Guests can access the application to view movies and book seats for a movie.

Admin can access the application to view a movie, add a new movie, book seats for a movie, update a movie, and delete a movie.

Listing the Steps to Create the Movie Application

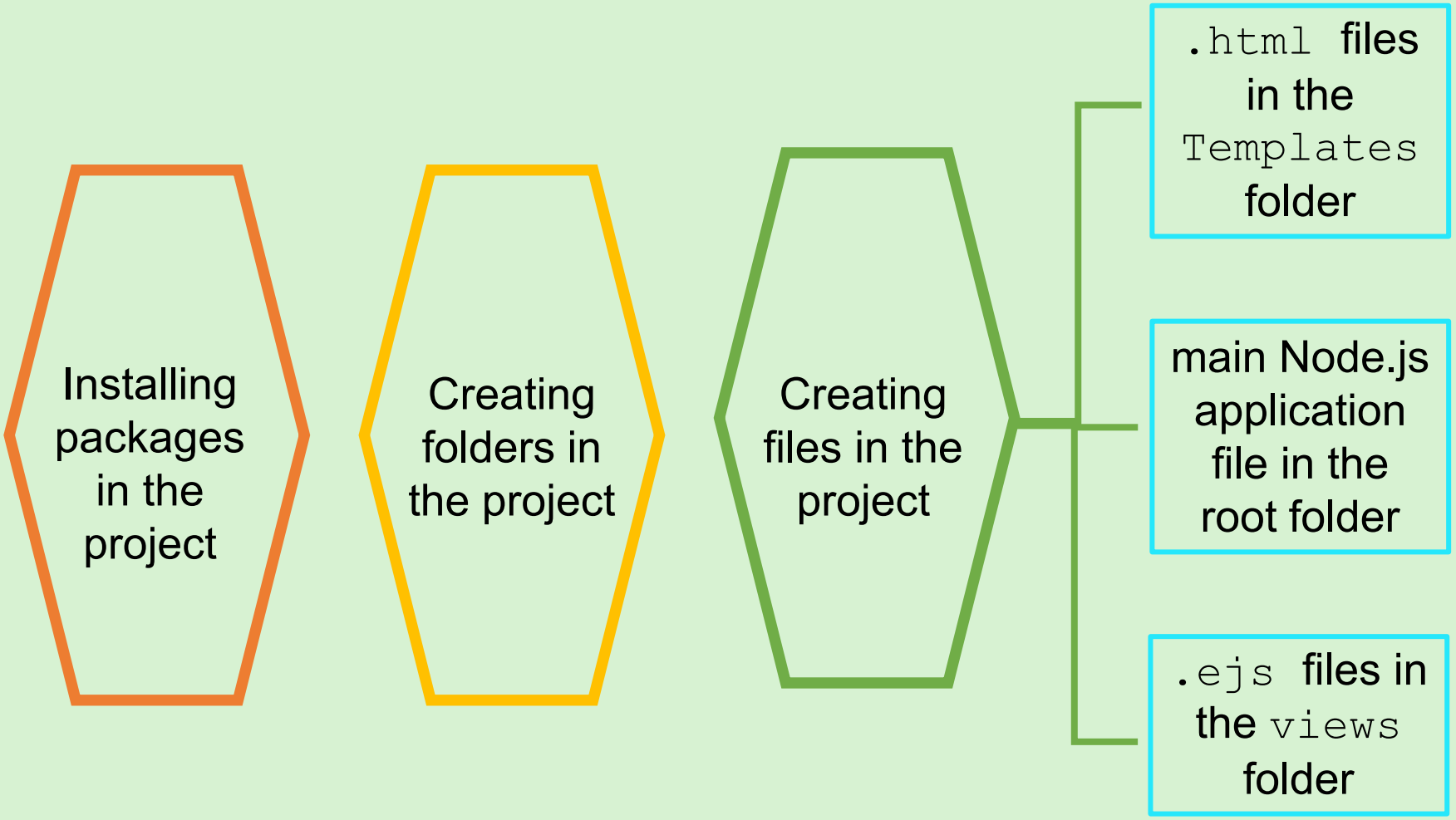


Retrieving the Connection String from MongoDB Cloud

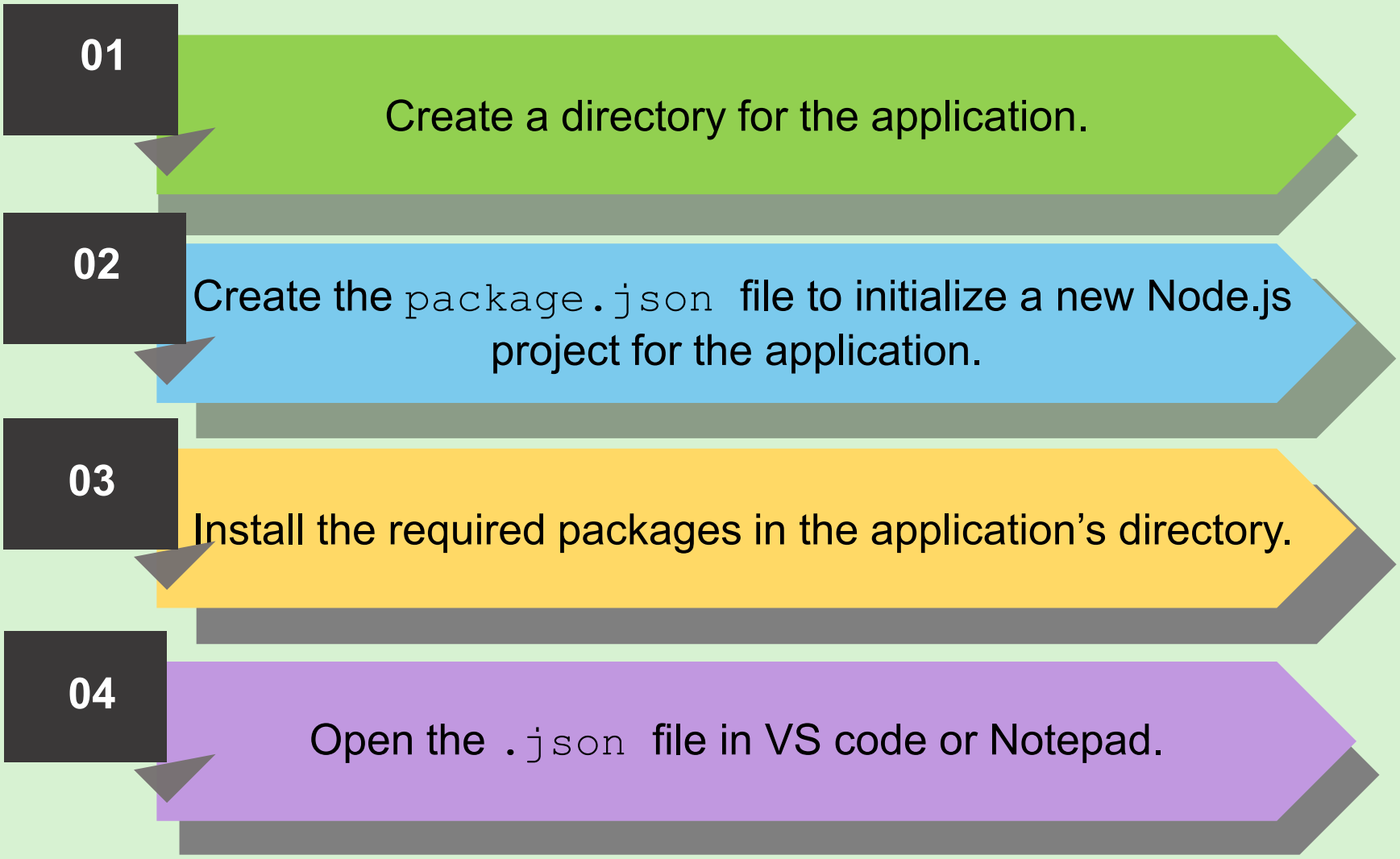
Deploy a database in the MongoDB cloud and retrieve the database deployment's connection string.

Use the connection string to connect the Wonderland Widescreen Node.js application with the cloud database for storing, retrieving, and processing data.

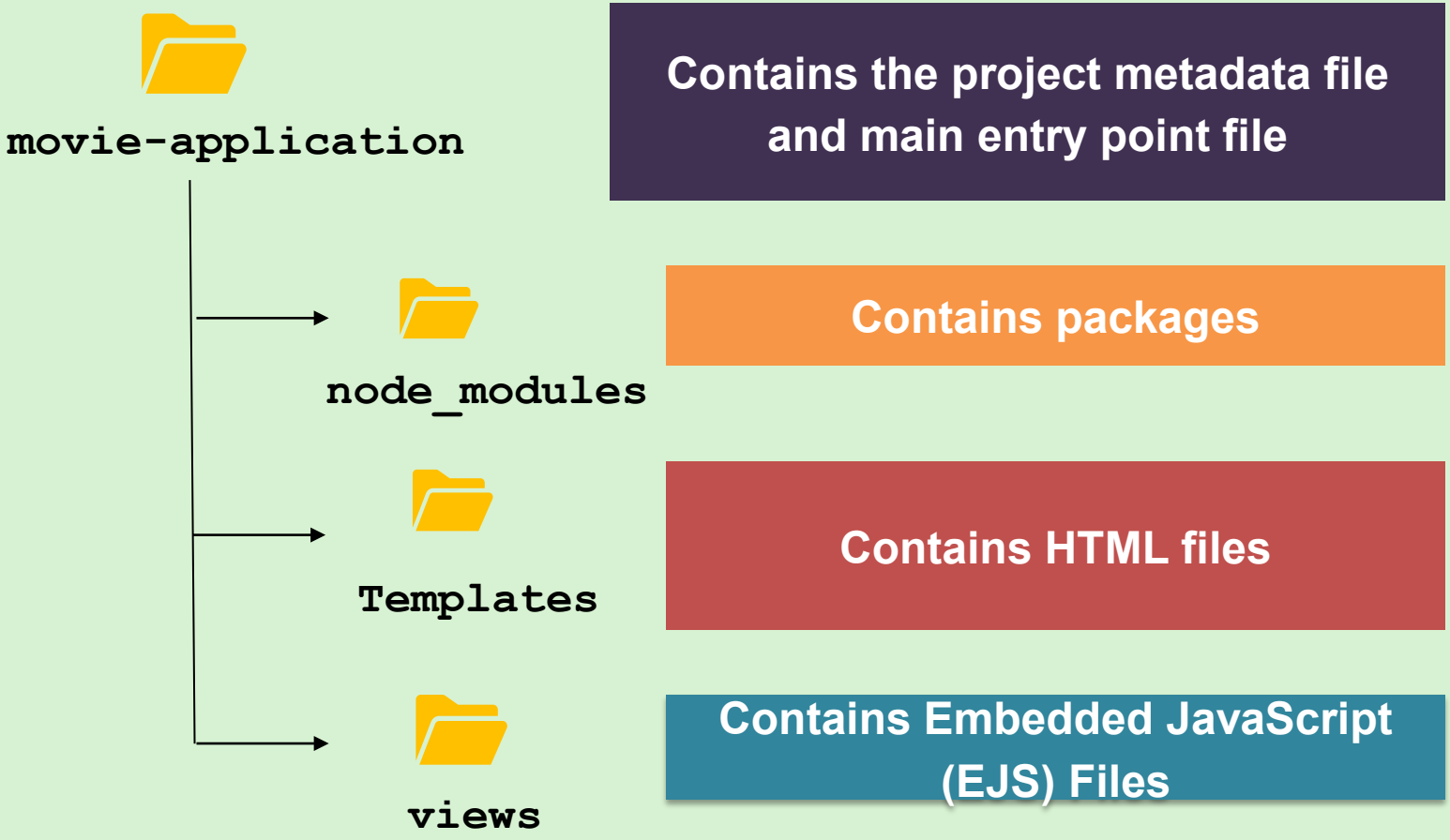
Creating the Movie Application in Node.js



Installing Packages in the Project



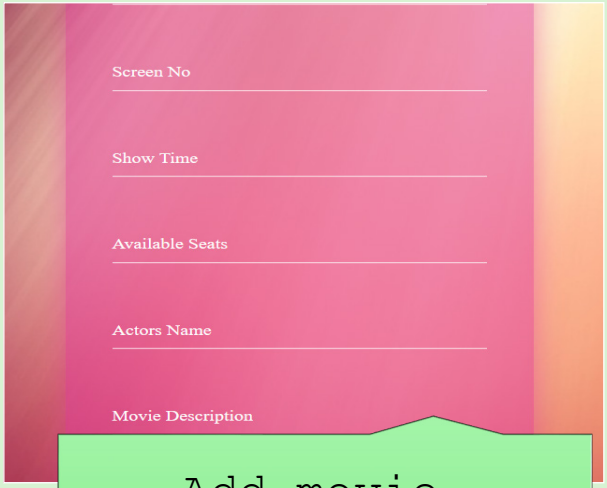
Creating Folders in the Project



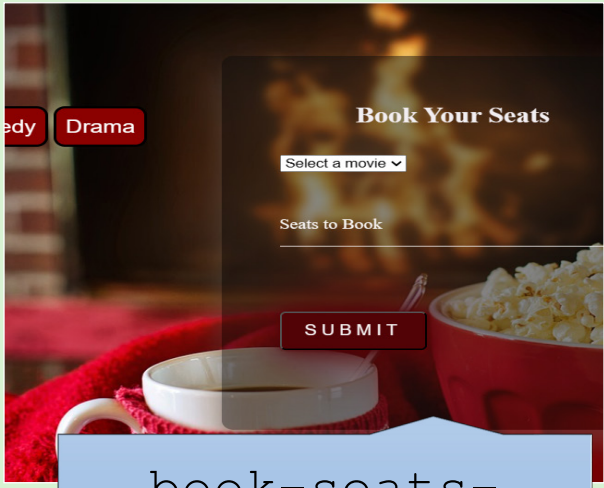
Creating .html Files in the Templates Folder

Movie-list.js

Movie_List.html

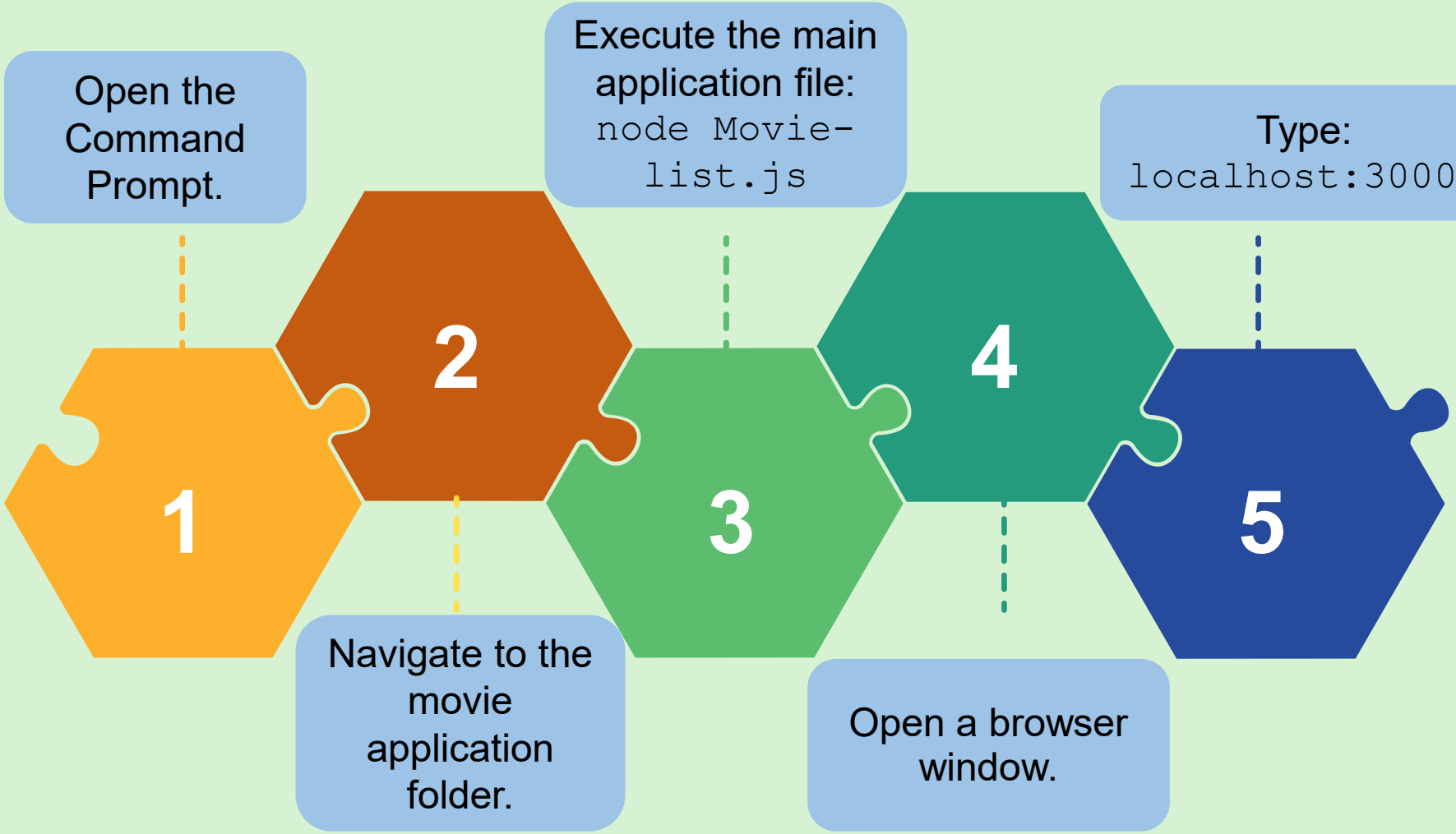


Add_movie-form.html



book-seats-form.html

Running the Application



Summary

- Creation of a complete Web application involves a combination of several different client-side and server-side technologies.
- The connection string created in the MongoDB cloud database connects the Wonderland Widescreen application to the MongoDB cloud.
- Front-end files are created using HTML and JavaScript.
- The main application entry point is a JavaScript file that must be kept in the root folder of the application.
- HTML files are kept in the `Templates` folder under the root folder.
- When Node packages are installed, they are automatically stored in the `node_modules` folder under the root folder.

Session: 12

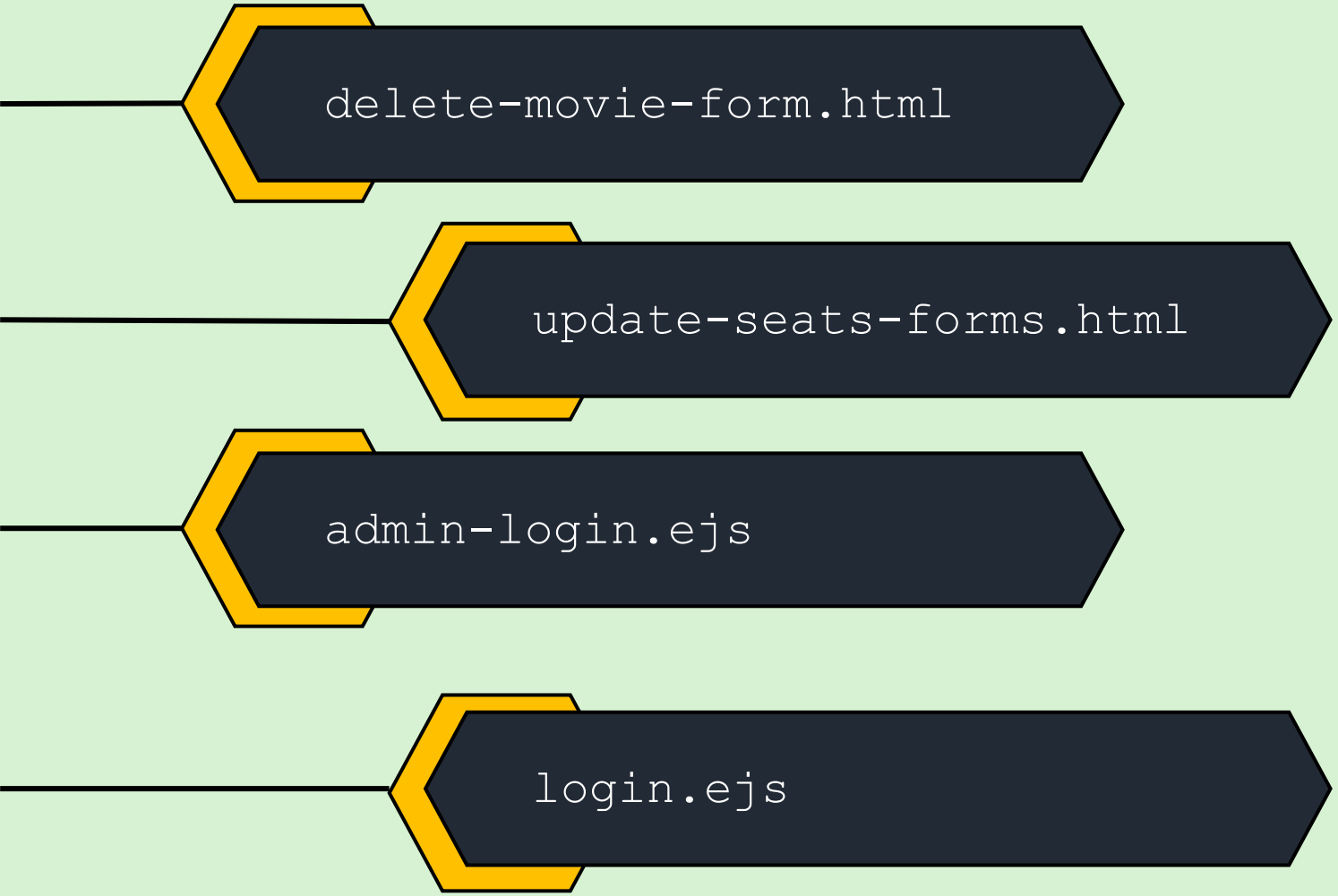
Node.js in Action (Application Development) - II

Server-side Development with Node.js

Objectives

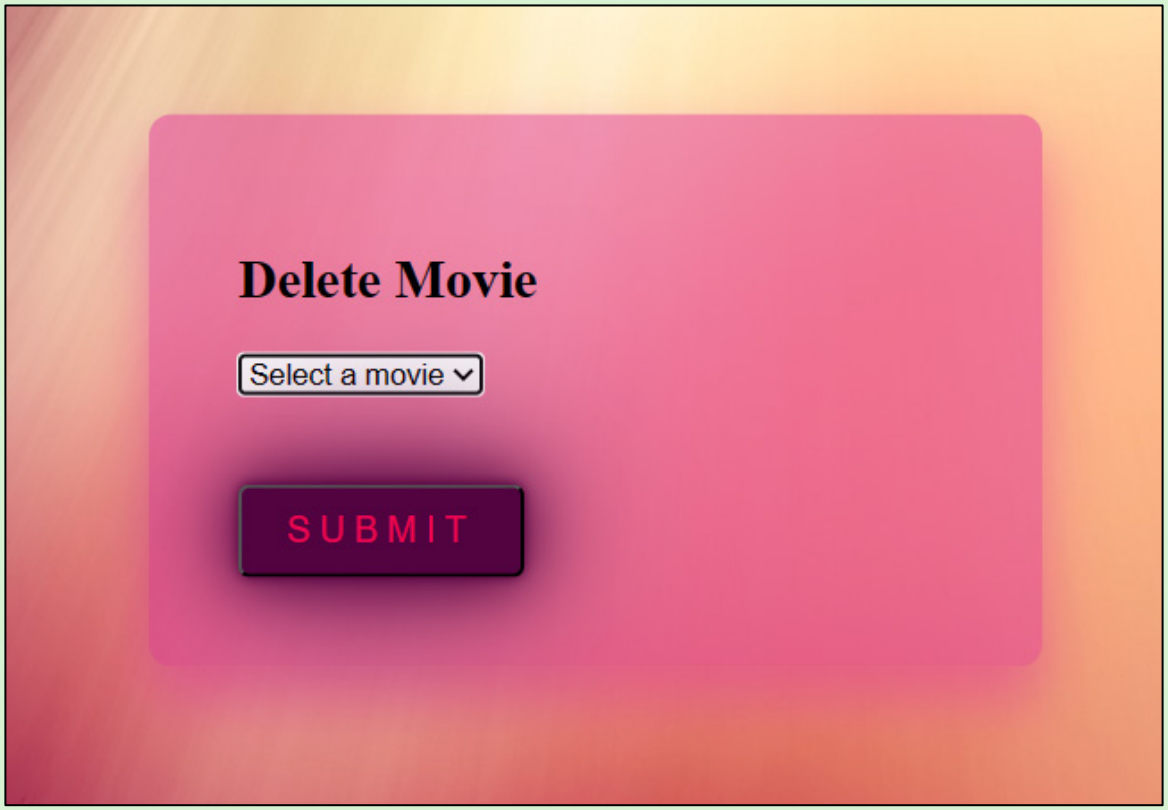
- List the steps to upload the Web application to the GitHub repository
- Explain the procedure to deploy the Web application on Render from a GitHub repository

Creating More Pages for Web Applications



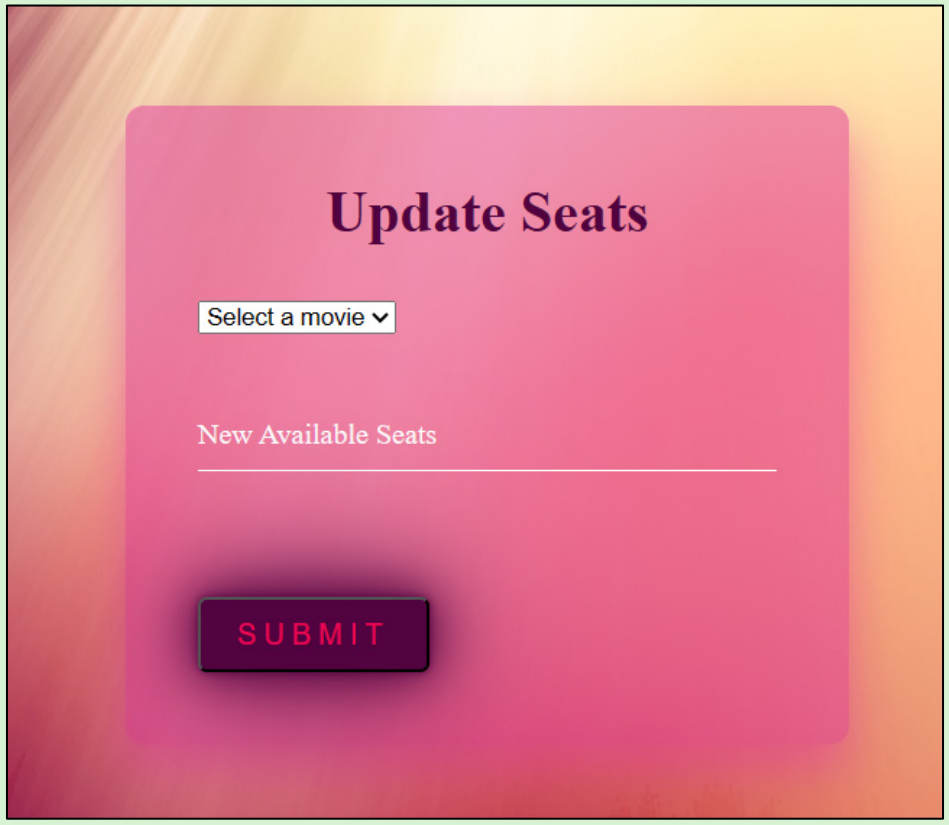
delete-movie-form.html

Design for the `delete-movie-form.html` page



update-seats-forms.html

Design for the update-seats-form.html page



Adding Home Page with User Authentication

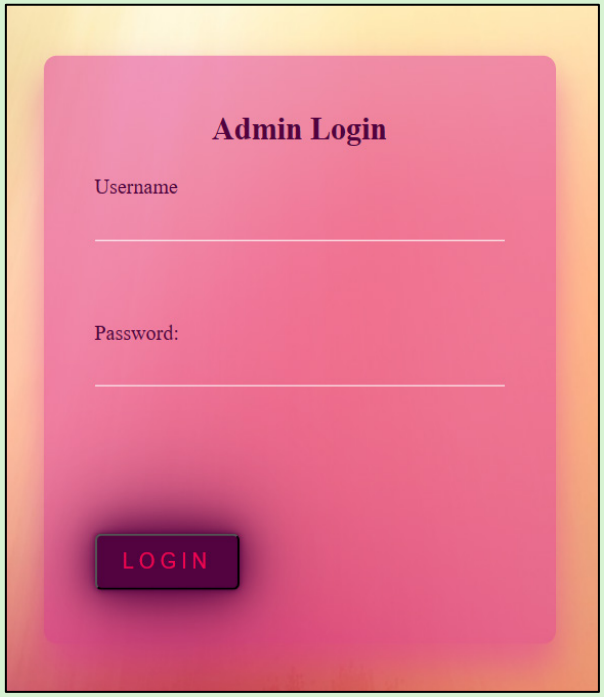
Design of the `Wonderland.html` page

Endpoint: `admin-login.ejs`

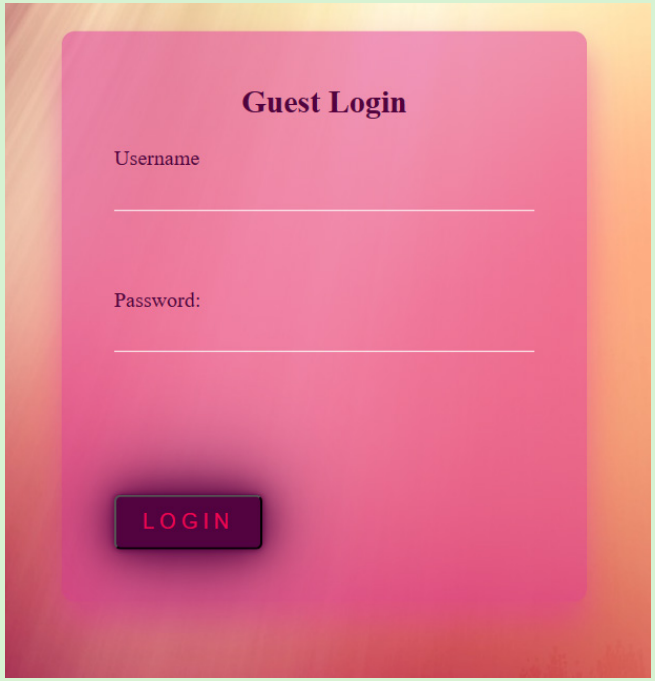
Endpoint: `login.ejs`

Adding Home Page with User Authentication

`Admin-login.ejs` page



`login.ejs` page



Updating the Main Application File

The `movie-list.js` file:

Purpose

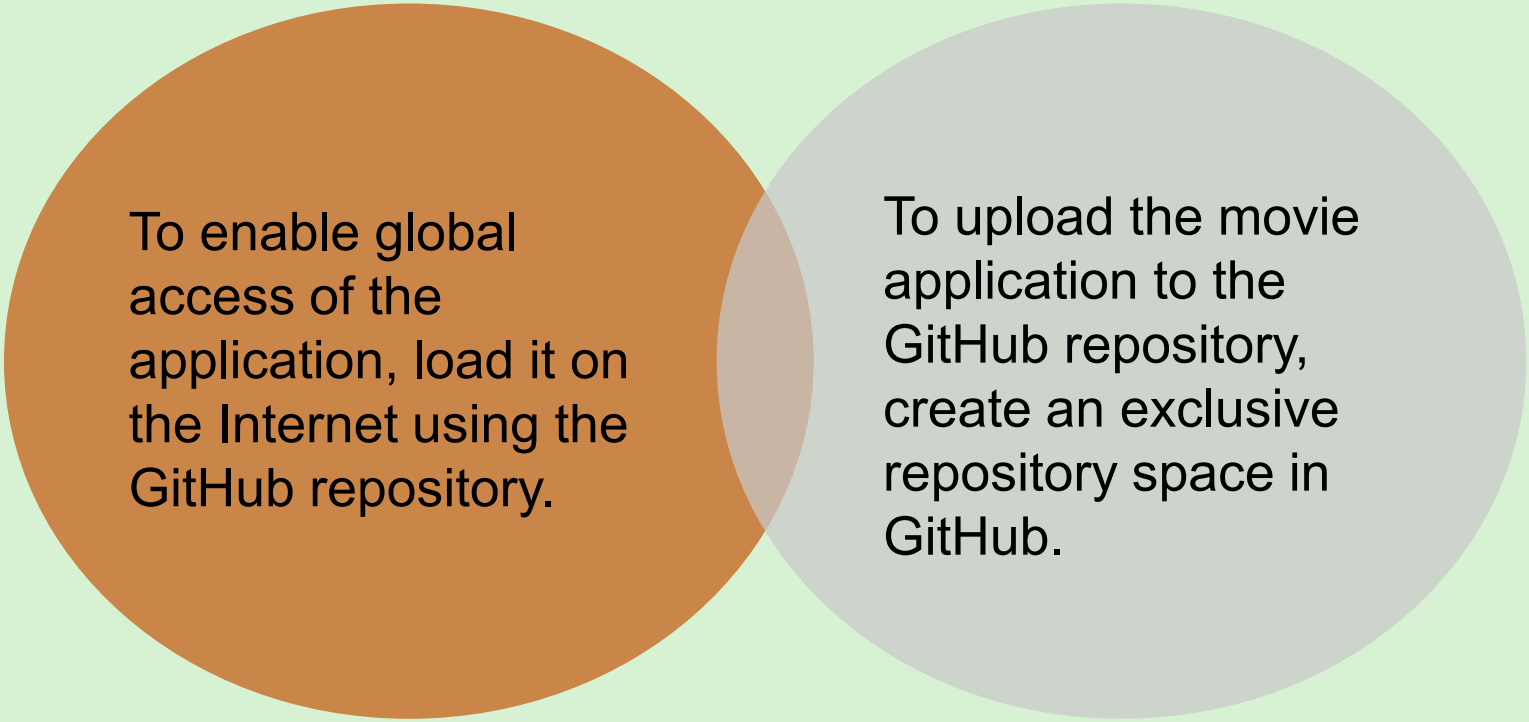
Acts as the main entry point in the movie application

Uses

Handles the core logic of the application

Defines routes and middleware

Uploading the Movie Application to the GitHub Repository



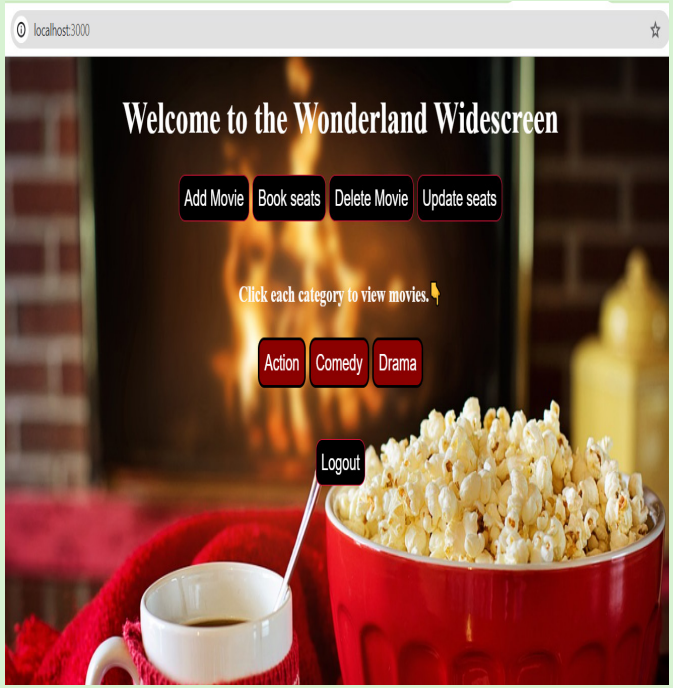
Deploying the Application on Render from GitHub Repository

Render helps to deploy applications on cloud.

- Deploy the movie application uploaded to the GitHub repository using Render.
- When the deployment is complete, copy the URL https://movie-application-o2ed.onrender.com

Accessing the Application in Web Browser

Admin Login



Add Movie

On the dashboard:

1. Click **Add Movie**.
2. Enter the details in the **Add Movie Details** page.
3. Click **SUBMIT**.

Book Seats

On the dashboard:

1. Click **Book seats**.
2. Select a movie and enter the number of seats.
3. Click **SUBMIT**.

Update Seats

On the dashboard:

1. Click **Update seats**.
2. Enter the details in the **Update seats** page.
3. Click **SUBMIT**.

Delete Movies

On the dashboard:

1. Click **Delete Movie**.
2. Enter the details in the **Delete Movies** page.
3. Click **SUBMIT**.

Summary

- Embedded JavaScript files are used to provide the HTML markups. The scripts required are embedded in the main Node.js application file.
- The `.ejs` files are stored in the `views` folder.
- Middleware for maintaining user session and authentication is added to the main Node.js application file using the `passport` middleware.
- `app.use` from the `express-session` package is used to set up session middleware.
- A Web application can be uploaded to the GitHub repository for better collaboration and version control.
- Render can be used to deploy Web applications on the cloud.